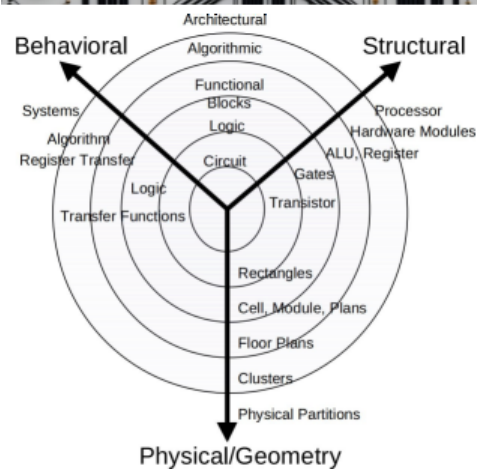


DISEÑO DE SISTEMAS DIGITALES

Carlos Iván Camargo Bareño

Universidad Nacional de Colombia

31 de agosto de 2026



DISEÑO DE SISTEMAS DIGITALES

AUTOR: C. Camargo

E-MAIL: cicamargoba@unal.edu.co

Universidad Nacional de Colombia

<http://www.unal.edu.co>

Se garantiza el permiso para distribuir, y/o modificar este documento
bajo los términos de la licencia Creative Commons CC BY-SA

Publicado por la Universidad Nacional de Colombia

Índice

1	Síntesis de Sistemas Digitales	5
1.1	Niveles de Abstracción	5
1.1.1	Síntesis	6
1.2	Sistema en Silicio - Descripción Estructural a nivel arquitectura	7
1.2.1	Sistemas sobre Silicio SoC	8
1.3	Ejemplo de implementación de una tarea usando un periférico	14
1.3.1	Lectura del periférico	15
1.3.2	Escritura del periférico	16
1.3.3	Implementación de la funcionalidad del periférico usando Lógica de transferencia de registros	16
1.4	Lógica Combinatoria y Secuencial	23
1.4.1	Lógica combinatoria	23
1.4.2	Minimización <i>gráfica</i> utilizando mapas de Karnaugh	27
1.4.3	Herramientas modernas para lógica combinatoria	28
1.4.4	Lenguajes de Descripción de Hardware	28
1.4.5	Descripción de módulos combinatorios usando el HDL Verilog	29
1.4.6	Lógica Secuencial	32
1.4.7	Descripción del Verilog del multiplicador	40
1.5	Implementación de un divisor de n bits sin signo	45
1.5.1	Identificación de componentes del camino de datos e interconexión	47
1.5.2	Unidad de control	48
1.5.3	Diagrama de bloques del divisor	49
1.5.4	Simulación del divisor	50
1.6	Implementación de un algoritmo para la raíz cuadrada	50
1.7	Conversión de Binario a Binario codificado en Decimal	51
1.8	Controlador de Panel LED 8 colores	52
1.8.1	Control de panel LED 12 bits por pixel	55
1.8.2	Control de LED RGB WS2812	56
1.9	Flujo de diseño Hardware - Software	60
1.9.1	simulación	63
1.9.2	Estructura de los ejemplos	63
1.9.3	Aplicación software para utilizar el multiplicador	65
1.10	Programación en lenguaje C	66
2	Ejemplos de Aplicaciones del VHDL	67
2.1	INTRODUCCION	67
2.2	¿QUE ES UNA FSM?	67
2.3	TABLAS Y DIAGRAMAS DE ESTADO	68
2.3.1	Tabla de Estados	68
2.3.2	Diagrama de Estados	69
2.4	SINTESIS DE FSM	70

2.4.1	Estructura de una FSM	70
2.4.2	Proceso de Síntesis	70
2.4.3	Control de Motor de Paso.....	72
2.5	DESCRIPCION VHDL DE UNA FSM	77
Referencias		84

Capítulo 1

Síntesis de Sistemas Digitales

Según el diccionario de la lengua española un Sistema es un "Conjunto de cosas que relacionadas entre sí ordenadamente contribuyen a determinado objeto.". Por lo tanto, un sistema digital es un conjunto de componentes electrónicos que de forma ordenada realizan una determinada función.

1.1 Niveles de Abstracción

Todo sistema digital puede ser descrito de tres diferentes formas utilizando diferentes dominios de descripción: *Comportamental* - describe el funcionamiento del sistema, *Estructural* - describe su estructura lógica y *Físico* - describe la implementación física, estos dominios se representan como ejes en la figura 1.1 que recibe el nombre de Y de Gajski-Kuhn. Dentro de cada dominio es posible realizar diferentes descripciones dependiendo del nivel de abstracción, un nivel de abstracción alto representa una visión de alto nivel del sistema enfocándose en características globales, mientras que un nivel de abstracción bajo implica una descripción más detallada. Existen 5 niveles de abstracción:

- *Arquitectura*: En este nivel se representa el sistema a alto nivel indicando sus principales componentes/funciones, no se detalla cómo están implementadas. En el dominio comportamental se realiza una descripción de funcionalidades unidas a restricciones físicas, en el dominio Estructural se describen los procesadores, memorias y buses de conexión del sistema y en el dominio físico dependiendo de la implementación se puede mostrar un diagrama del Circuito Integrado o la disposición física en una placa de circuito impreso.
- *Algoritmo*: Se realiza una descripción de las funcionalidades descritas en el nivel arquitectura, en el dominio comportamental se proporciona la descripción de todos los algoritmos que implementan la funcionalidad requerida, en el dominio Estructural se muestran los componentes que implementan un determinado algoritmo (camino de datos, control, periféricos y procesador), en el dominio físico se realiza una descripción que incluye información de tiempos de respuesta de los circuitos que implementan la descripción estructural.
- *Bloque Funcional*: En el dominio comportamental se describen los algoritmos como el resultado de una transferencia entre registros, lo que la acerca más a la descripción estructural, uno de los métodos utilizados es el uso de máquinas de estado algorítmicas; en el dominio estructural la descripción se realiza utilizando unidades aritméticas-lógicas (ALUs), sumadores, comparadores, contadores, multiplexores y registros todos ellos controlados por una unidad de control. En el dominio físico se realiza una representación de la localización de los componentes, sus caminos de conexión, teniendo en cuenta consideraciones de espacio y potencia.
- *Lógico*: En el dominio funcional se realiza una descripción a nivel de ecuaciones booleanas y autómatas finitos; el dominio estructural se representa con los componentes básicos Flip-Flops, y compuertas.
- *Circuitos*: En el dominio comportamental se realizan representaciones del comportamiento utilizando ecuaciones diferenciales de voltaje y corriente; en el dominio estructural se realiza la representación de cada componente a nivel de transistores, diodos, resistencias y condensadores dependiendo de la tecnología utilizada; el dominio físico se representa con el diagrama físico de las diferentes capas que forman el circuito integrado indicando las dimensiones exactas de y geometría de cada componente.

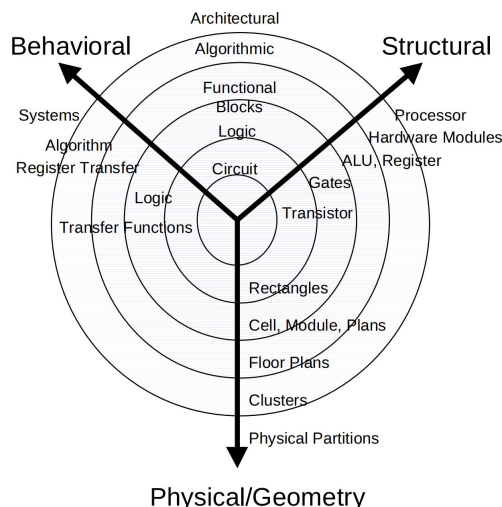


Figura 1.1 Niveles de Abstracción Y de Gajski-Kuhn fuente: D. Gajski, ?Silicon Compilers?, Addison-Wesley, 1987

1.1.1 Síntesis

Pasar de un dominio de descripción a otro recibe el nombre de síntesis, es un proceso realizado con ayuda de herramientas de diseño (CAD), las cuales pueden ser lenguajes de programación o de descripción de hardware (verilog, vhdl, systemC, etc) o herramientas gráficas. En la gráfica 1.2 se muestra un flujo de diseño que comienza en una descripción comportamental de alto nivel con la funcionalidad requerida (1), continúa con una descripción estructural a nivel de arquitectura (2), sigue con una descripción comportamental a nivel algorítmico (3), a continuación se realiza una descripción estructural a nivel funcional (4), después se muestra una descripción estructural a nivel lógico (5) y finaliza con una representación física del circuito integrado que implementa la funcionalidad especificada en 1.

Procesos de síntesis se pueden observar del paso (3) al (4) donde una herramienta tipo CAD a partir de una descripción comportamental basada en un diagrama de flujo y un diagrama de estados genera una representación en el dominio estructural con componentes básicos (flip-flops, compuertas y registros). Otro ejemplo de síntesis se obtiene cuando se pasa de una descripción **estructural** a nivel lógico (5) a una descripción **física** a nivel de circuito. Estas tareas se realizaban manualmente en el pasado, lo que generaba muchos errores o reprocesos y en algunos casos tomaban mucho tiempo, gracias a los avances de las herramientas computacionales, y a la proliferación de herramientas para este fin, estas tareas se pueden realizar de forma automatizada aumentando la confiabilidad de los diseños al tiempo que se reduce el tiempo para hacerlas y se permite trabajar con circuitos más complejos.

1.2 Sistema en Silicio - Descripción Estructural a nivel arquitectura

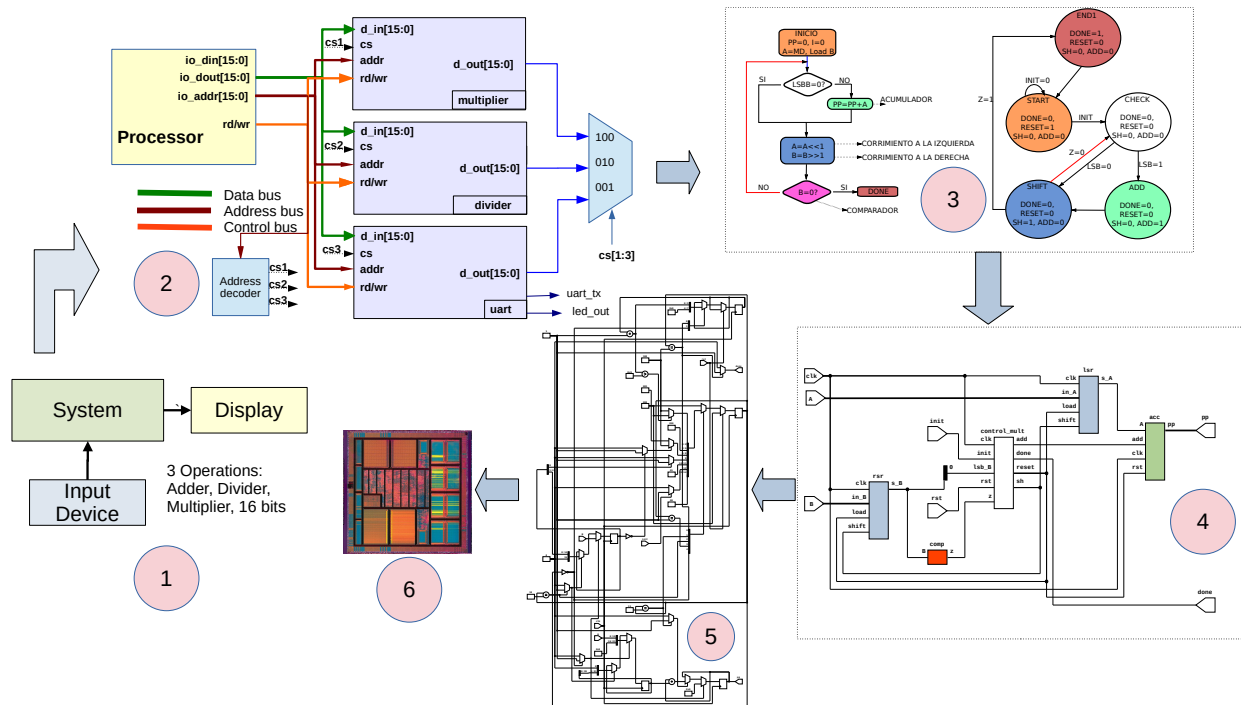


Figura 1.2 Flujo de diseño con diferentes niveles de abstracción y dominios de descripción

1.2 Sistema en Silicio - Descripción Estructural a nivel arquitectura

En el pasado, en ausencia de computadores y herramientas para el diseño, la experiencia del diseñador era clave a la hora de abordar la creación de un nuevo dispositivo digital, decenas de años de experiencia acumulada en el equipo de diseño permitía reducir de forma considerable el tiempo a salir al mercado del producto (time to market), sin embargo, adquirir esta experiencia no era fácil ya que requería mucho tiempo para lograrlo, cuando una persona quiere diseñar un sistema que resuelva un determinado problema, no es factible esperar muchos años para poder hacerlo, lo mismo sucede con los estudiantes en los cursos de diseño digital (que en total en el mejor de los casos suman 2 años), entonces para que estos estudiantes salgan al mundo productivo con las habilidades necesarias ¿Qué se debe hacer?.

- 1- Alargar las carreras para que los estudiantes adquieran las habilidades necesarias como en el pasado o
2. Utilizar las herramientas modernas y cambiar la forma de trabajo en estos cursos centrándose en las tareas que realizamos mejor los humanos y dejar los procesos tediosos a las máquinas.

Algo en lo que somos buenos los seres humanos es en pensar, esto hasta el momento (2023) no lo ha podido realizar ninguna máquina. Por esta razón, es necesario cambiar la forma de diseño que se realizaba en el pasado, la cual consistía en una metodología en la que a partir de componentes discretos se llegaba a la funcionalidad deseada, de nuevo, solo diseñadores con mucha experiencia podían realizar esta tarea. Si se invierte esta metodología partiendo de las especificaciones del sistema y de la funcionalidad que se desea, utilizando los más altos niveles de abstracción para hacer la descripción del sistema y haciendo uso de las herramientas tipo CAD para bajar el nivel de abstracción y para cambiar el dominio de descripción, la experiencia del diseñador no será tan determinante y se podrán formar profesionales con las habilidades que la industria requiere.

Esto debe estar acompañado de un cambio en la metodología de enseñanza ya que la forma tradicional (quiz, parcial, tareas para la casa) no es aplicable a la hora de adquirir habilidades en diseño, para aprender a diseñar se deben enfrentar problemas reales y dar soluciones a problemas reales (lo que no es posible en 2 horas de un parcial o un quiz), es necesario trabajar en equipo de forma efectiva (apoyando a los demás para lograr el objetivo, realizando consensos y articulando el trabajo entre los diferentes miembros o equipos), la teoría debe estar sincronizada con el momento en el flujo de diseño para que se experimente en la práctica logrando aprendizajes significativos.

En este libro se utilizará la descripción a nivel arquitectura asumiendo que todo sistema digital se implementará utilizando procesadores, periféricos y memorias, se utilizarán diferentes procesadores (LM32, RISC-V) y se trabajará en la creación de periféricos dedicados utilizando el lenguaje de descripción de hardware Verilog (aunque no se realizará una presentación de este lenguaje ya que en la web se encuentra mucho material de buena calidad).

Para el primer curso del área de sistemas digitales se trabajará con un procesador, el cuál será tratado como una caja negra con un conjunto de funcionalidades que puede ser programado usando el lenguaje C. Esto se hace para que se conozca perfectamente la arquitectura de los procesadores modernos donde siempre se incluye una CPU con un grupo de periféricos, al tiempo que se trabajan las habilidades en programación usando un lenguaje de programación de nivel adecuado.

En este primer curso se realizará la descripción de periféricos utilizando máquinas de estado algorítmicas, las cuales, reproducen la arquitectura basada en una unidad que implementa un algoritmo y unos bloques que realizan funciones básicas aritméticas o lógicas.

1.2.1 Sistemas sobre Silicio SoC

Un Sistema en silicio (SoC) es la unión de una unidad de procesamiento central (CPU) y un grupo de periféricos (multiplicador, divisor, uart, unidad aritmética de punto flotante, etc); la CPU está encargada de leer, decodificar y ejecutar una lista de instrucciones que implementan una determinada tarea, esta arquitectura permite la ejecución de cualquier algoritmo con solo cambiar las instrucciones que deben ser ejecutadas.

En la figura 1.3 se muestra el diagrama de bloques de un Sistema en Silicio basado en la unidad de procesamiento central RISC-V, el cual fue desarrollado por Bruno Levy ¹.

¹ <https://github.com/BrunoLevy>

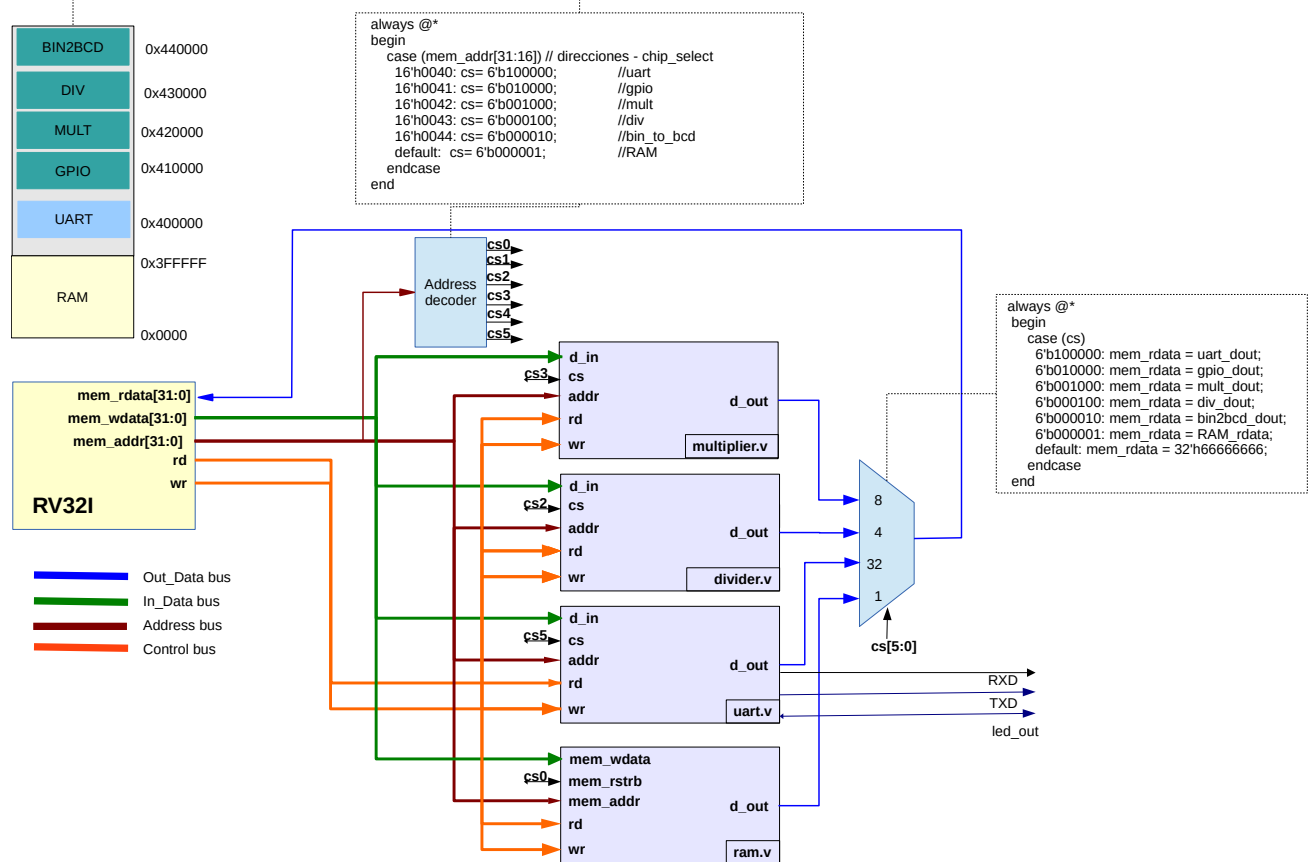


Figura 1.3 Diagrama de bloques del procesador RISC-V

Procesador RISC-V

La CPU es un módulo que realiza tres tareas de forma continua. *Fetch* lee una instrucción de la memoria de programa, *Decode*, reconoce la instrucción y *Execute*, realiza las acciones definidas para la instrucción detectada. En este capítulo se utilizará una CPU como una caja negra, es decir, un módulo que ejecuta una serie de instrucciones y nos permite controlar otros módulos, los cuales, a su vez implementan una determinada función.

En la actualidad existen una gran variedad de procesadores tanto abiertos como propietarios, sin embargo, el procesador RISC-V, ha sido adoptado por una gran cantidad de personas en todo el mundo (que hacen parte de diferentes mundos, industrial, académico, pasatiempo), y gracias a esto se han desarrollado una gran cantidad de herramientas que facilitan su uso, en específico, se cuenta con compiladores en diferentes lenguajes, librerías, sistemas operativos y aplicaciones.

Parte de su popularidad se debe a su carácter libre y abierto, esto es, puede utilizarse sin restricciones por quien lo desee. El proyecto inició en la Universidad de Berkeley con la colaboración de voluntarios y diseñadores externos. Este procesador como se puede deducir de su nombre es del tipo **Reduced Instruction Set Computer**, es decir, posee un conjunto de instrucciones reducido, lo que disminuye su complejidad, permite aumentar la velocidad de desempeño al permitir la segmentación (no es necesario esperar a que una instrucción se ejecute para adquirir la siguiente instrucción).

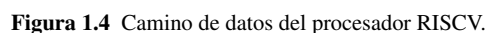
Set de Instrucciones

En la tabla 1.1 se puede apreciar la lista de las instrucciones del RV32I con su respectivo código y función. Estas instrucciones pueden ser vistas como bloques constructores, y serán parte de los algoritmos implementados para ser ejecutados por el procesador.

Inst	Name	FMT	Opcode	funct3	funct7	Description (C)
add	ADD	R	0110011	0x0	0x00	rd = rs1 + rs2
sub	SUB	R	0110011	0x0	0x20	rd = rs1 - rs2
xor	XOR	R	0110011	0x4	0x00	rd = rs1 ^ rs2
or	OR	R	0110011	0x6	0x00	rd = rs1 — rs2
and	AND	R	0110011	0x7	0x00	rd = rs1 & rs2
sll	Shift Left Logical	R	0110011	0x1	0x00	rd = rs1 << rs2
srl	Shift Right Logical	R	0110011	0x5	0x00	rd = rs1 >> rs2
sra	Shift Right Arith*	R	0110011	0x5	0x20	rd = rs1 >> rs2
slt	Set Less Than	R	0110011	0x2	0x00	rd = (rs1 < rs2)?1:0
sltu	Set Less Than (U)	R	0110011	0x3	0x00	rd = (rs1 < rs2)?1:0
addi	ADD Immediate	I	0010011	0x0		rd = rs1 + imm
xori	XOR Immediate	I	0010011	0x4		rd = rs1 ^ imm
ori	OR Immediate	I	0010011	0x6		rd = rs1 — imm
andi	AND Immediate	I	0010011	0x7		rd = rs1 & imm
slli	Shift Left Logical Imm	I	0010011	0x1	imm[5:11]=0x00	rd = rs1 << imm[0:4]
srlr	Shift Right Logical Imm	I	0010011	0x5	imm[5:11]=0x00	rd = rs1 >> imm[0:4]
srai	Shift Right Arith Imm	I	0010011	0x5	imm[5:11]=0x20	rd = rs1 >> imm[0:4]
slti	Set Less Than Imm	I	0010011	0x2		rd = (rs1 < imm)?1:0
sltiu	Set Less Than Imm (U)	I	0010011	0x3		rd = (rs1 < imm)?1:0
lb	Load Byte	I	0000011	0x0		rd = M[rs1+imm][0:7]
lh	Load Half	I	0000011	0x1		rd = M[rs1+imm][0:15]
lw	Load Word	I	0000011	0x2		rd = M[rs1+imm][0:31]
lbu	Load Byte (U)	I	0000011	0x4		rd = M[rs1+imm][0:7]
lhu	Load Half (U)	I	0000011	0x5		rd = M[rs1+imm][0:15]
sb	Store Byte	S	0100011	0x0		M[rs1+imm][0:7] = rs2[0:7]
sh	Store Half	S	0100011	0x1		M[rs1+imm][0:15] = rs2[0:15]
sw	Store Word	S	0100011	0x2		M[rs1+imm][0:31] = rs2[0:31]
beq	Branch ==	B	1100011	0x0		if(rs1 == rs2) PC += imm
bne	Branch !=	B	1100011	0x1		if(rs1 != rs2) PC += imm
blt	Branch <	B	1100011	0x4		if(rs1 < rs2) PC += imm
bge	Branch ≥	B	1100011	0x5		if(rs1 ≥ rs2) PC += imm
bltu	Branch < (U)	B	1100011	0x6		if(rs1 < rs2) PC += imm
bgeu	Branch ≥ (U)	B	1100011	0x7		if(rs1 ≥ rs2) PC += imm
jal	Jump And Link	J	1101111			rd = PC+4; PC += imm
jalr	Jump And Link Reg	I	1100111	0x0		rd = PC+4; PC = rs1 + imm
lui	Load Upper Imm	U	0110111			rd = imm << 12
auipc	Add Upper Imm to PC	U	0010111			rd = PC + (imm << 12)
ecall	Environment Call	I	1110011	0x0	imm=0x0	Transfer control to OS
ebreak	Environment Break	I	1110011	0x0	imm=0x1	Transfer control to debugger

Cuadro 1.1 Set de Instrucciones del RV32I. fuente: <https://github.com/jameslzh/riscv-card/tree/master>

En la Figura 1.4 se muestra el diagrama del camino de datos del procesador, en ella se aprecian los componentes típicos de esta unidad, un banco de registros, una Unidad Aritmética y Lógica (ALU) y los buses de datos (*mem_wdata*, *mem_rdata*) y dirección (*mem_addr*) que van al exterior.



Para que un algoritmo tenga utilidad en la vida real debe permitir la entrada de datos desde el exterior, así como la salida de los resultados para que sean utilizados externamente, para esto, los procesadores deben contar con dispositivos de entrada/salida que permitan el intercambio de información o realicen tareas específicas, estos dispositivos reciben el nombre de *periféricos*, en la actualidad existe una gran variedad para diversas aplicaciones, lo que permite utilizar diferentes protocolos de comunicación (serial, SPI, I2C, USB, Ethernet, PCI, etc), dispositivos de interfaz con humanos (teclados, monitores, LCDs, etc). Estos periféricos se conectan a la CPU utilizando 3 buses (agrupación de 8, 16, 32, 64, señales) que realizan las siguientes funciones:

- **Datos:** Transmite y recibe información entre el procesador y el periférico, esta información representa valores de variables de un determinado algoritmo, valores para configurar periféricos o valores que se intercambian desde y hacia el exterior.
- **Dirección:** Transmite información que permite seleccionar el periférico con el que se desea comunicar el procesador.
- **Control:** Indica el tipo de operación que va a realizar la CPU, lectura o escritura.

Gracias a estos tres buses la CPU puede acceder a la memoria externa del procesador, esto es, los diferentes valores que puede tomar el bus de direcciones, cada valor del bus de direcciones corresponde a un dato de N bits del bus de datos; en la figura 1.5 se muestra este concepto, allí podemos observar que existen dos buses para el bus de datos uno para la entrada y otra para la salida de datos (esto se hace ya que los Dispositivos Lógicos Programables no poseen buses tri-estado internamente). El número de señales del bus de direcciones determina el rango de direcciones que puede tomar la memoria externa, en este ejemplo son 16 señales, por lo que este bus puede tomar los valores 0 a

65536, lo que corresponde a un bus de 16 bits. Las señales del bus de control se activan cuando el procesador realiza una operación de lectura (activando la señal *rd*) o de escritura (activando la señal *wr*).

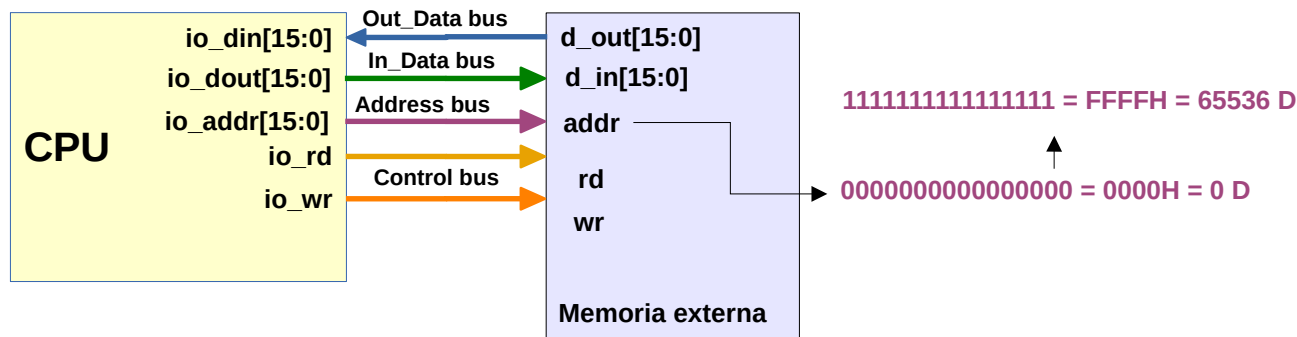


Figura 1.5 Buses del procesador RISC-V y concepto de memoria externa

Periféricos

Como se dijo anteriormente, los periféricos realizan tareas fijas y permiten adicionar funcionalidad a la CPU, en la figura 1.3 se muestra un SoC con tres periféricos: *multiplier* y *divider* posibilitan operaciones de multiplicación y división, mientras que el periférico *uart* permite la transmisión de información utilizando un protocolo serial asíncrono, el cuál, es el más utilizado en aplicaciones de microcontroladores.

La comunicación entre los periféricos y la CPU se realiza a través de los buses de *datos*, *direcciones* y *control*. La CPU monopoliza el control de las operaciones de lectura y escritura, los periféricos nunca realizarán operaciones a la CPU de forma autónoma, siempre responden a peticiones de la CPU; este proceso centralizado permite coordinar las acciones de los periféricos y cede el control de la ejecución del algoritmo a la CPU, esto no significa que los periféricos no puedan realizar tareas complejas, solo que ellos realizan tareas ordenadas por la CPU.

Para diferenciar con qué periférico se está comunicando la CPU, se debe asignar un rango de las direcciones externas a cada uno de los periféricos que forman el SoC, el rango de direcciones asignado depende del tipo de periférico, así pues, una memoria externa de 1 kB requerirá 1024 B, mientras que un periférico sencillo con 16 registros (de configuración, control y estatus) requerirá 16 posiciones de memoria. En SoCs con pocos periféricos (entre 1-10) es normal asignar el mismo rango de memoria (número de bits) a los periféricos así estos no requieran estas posiciones. En la figura 1.3 podemos observar que se asignan las regiones 0x400000 a 0x40FFFF para la UART, 0x410000 a 0x41FFFF para los GPIOs, 0x420000 a 0x42FFFF para el multiplicador, 0x430000 a 0x43FFFF para el divisor y 0x440000 a 0x44FFFF para el BIN2BCD. Las direcciones 0-0x3FFFFFF son asignadas a la memoria de programa.

Este listado de rango de direcciones recibe el nombre de *mapa de memoria*, y el módulo que realiza su implementación física recibe el nombre de *decodificador de direcciones* (address decoder), el cual (ver figura 1.3), tiene como entrada las señales del bus de direcciones y proporciona seis salidas (una por cada periférico) *cs0* - *cs6* (*cs* por chip select), estas señales se activan de acuerdo al mapa de memoria establecido: *cs0* se activa (toma un valor lógico alto 1) en el rango 0x000000 a 0x3FFFFFF, *cs1* en el rango 0x400000 a 0x40FFFF y *cs2* para el rango 0x410000 a 0x41FFFF, etc. En el recuadro de la figura se muestra el código en Verilog que implementa este decodificador, se trata de un bloque combinatorio (el valor de las salidas dependen únicamente del valor de las entradas) que tiene como entradas las señales del bus de direcciones *mem_addr[31:16]* (bits 31 al 16), esto porque las señales *mem_addr[15:0]* varían de 0000 a FFFF tomando cualquiera de estos valores y no son determinantes. La salida es el bus *cs[5:0]* formado por las seis señales de selección descritas anteriormente. La sentencia *case* examina el valor de *mem_addr* y asigna el valor a *cs* de acuerdo al mapa de memoria, estos valores son arbitrarios y pueden cambiar a criterio del diseñador, lo único que debe cumplirse es que **no se pueden asignar la misma dirección de memoria a más de un periférico**.

Ejemplo de comunicación CPU-periférico

Las señales de control *wr* y *rd*, indican el tipo de transmisión; cuando el procesador escribe a un periférico activa la señal *wr* y cuando lee información activa la señal *rd*, dicha información será transmitida por el bus de datos de salida y de entrada respectivamente. Las señales *cs1*, *cs2* ... *cs6* (chip select) activan al periférico con el que se desea realizar la comunicación, y únicamente el que se encuentre activado (solo se activa uno) participará en el intercambio de información con la CPU.

El bus de datos de salida de la CPU se conecta al bus de datos de entrada de todos los periféricos, y sólo el que sea seleccionado (activando su señal *cs*) procesará esa información. El bus de datos de salida de los periféricos se conecta a un *multiplexor* (módulo combinatorio que conecta una de múltiples entradas a la salida dependiendo de la señal de selección) que controla la conexión de estos al bus de datos de entrada del procesador, las señales *cs1*, *cs2* y *cs3* determinan que periférico es conectado. de esta forma cuando se activa la señal de activación del periférico se crea un camino entre el periférico y la CPU.

Para aclarar este proceso, considere que desea escribir los datos 0x0005 en la dirección 0x420004, 0x0003 en la 0x420008 y 0x0001 en la 0x42000C; estas direcciones se encuentran en el rango del multiplicador por lo que la señal *cs3* se activará (1) y las señales *cs0*, *1,2,4,5* desactivará (0) a los otros periféricos.

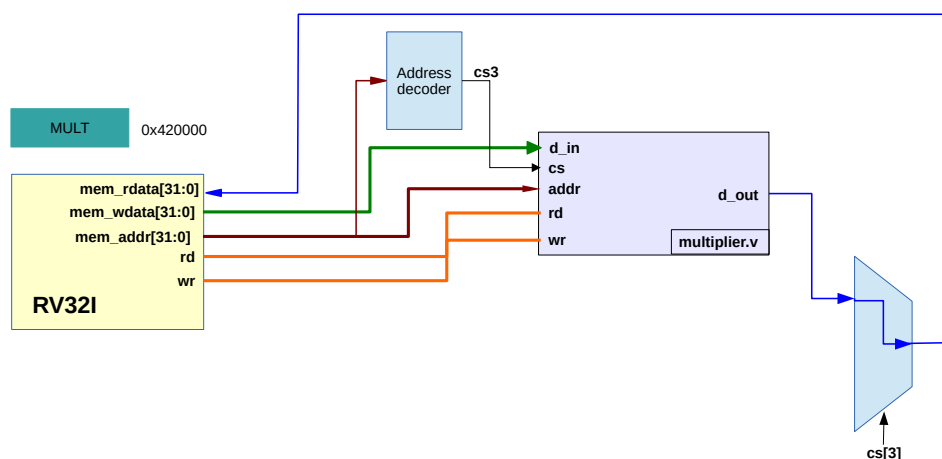


Figura 1.6 SoC J1 - Comunicación con un periférico

En la figura 1.6 se muestra el circuito equivalente al seleccionar el rango de direcciones del multiplicador, los otros periféricos se desactivan lo que para fines prácticos equivale a eliminarlos, con lo que la CPU se encuentra totalmente conectada al multiplicador.

Escritura a un periférico

En la figura 1.7 se muestran las formas de onda de las señales asociadas a un ciclo de escritura desde la CPU a las direcciones 0x420004, 0x420008 y 0x42000C. Cuando se realiza una operación de escritura, la CPU coloca la dirección en el bus de direcciones *mem_addr*, lo que hace que la señal *cs3* sea igual a 1; activa la señal *wr* (1) del bus de control y coloca el dato a escribir en el bus de datos de salida *mem_wdata*. Para las señales mostradas en la figura escribe 0x0005 en la dirección 0x420004, 0x0003 en la dirección 0x420008 y 0x0001 en la 0x42000C.

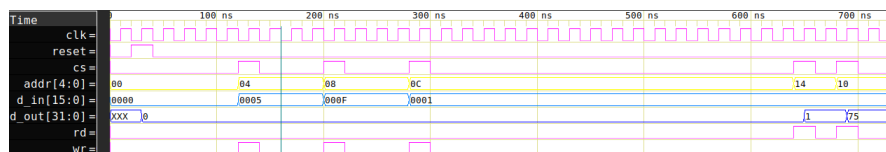


Figura 1.7 SoC J1 - Formas de onda de la escritura un periférico

Lectura a un periférico

En la figura 1.8 se muestran las formas de onda de las señales asociadas a un ciclo de lectura desde la CPU a las direcciones 0x420014 y 0x420010. Cuando se realiza una operación de lectura, la CPU coloca la dirección en el bus de direcciones *mem_addr*, lo que hace que la señal *cs1* sea igual a 1; activa la señal *rd* del bus de control y lee el bus de datos de entrada *mem_rdata*. Para las señales mostradas en la figura lee 0x0001 en la dirección 0x420014, y 0x000F en la 0x420010.

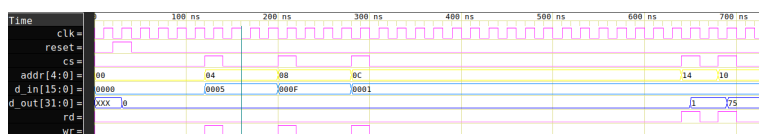


Figura 1.8 SoC J1 - Formas de onda de la lectura un periférico

1.3 Ejemplo de implementación de una tarea usando un periférico

En esta sección mostraremos la estructura interna de los periféricos para permitir la comunicación con la CPU. En la figura 1.9 se muestra el diagrama de caja negra para el multiplicador; este módulo tiene como entradas los operandos de 16 bits **A** y **B** y la señal **init** (que da comienzo al algoritmo de multiplicación) y como salida el resultado de 32 bits **PP** y la señal que indica que ya se realizó la operación **done**.

Se deben crear mecanismos que permitan suministrar los operandos y obtener el resultado de la operación a través de los buses de datos conectados a la CPU, para esto, el primer paso, es identificarlos y asignarles una dirección de memoria a cada uno de ellos para que la CPU pueda leerlos o escribirlos. En este caso (la asignación puede variar a criterio del diseñador) el primer operando se encuentra en la dirección $BASE + 00$, (donde $BASE$ es la dirección de memoria del periférico en el mapa de memoria, 0x6700 en este caso), el segundo operando a la dirección $BASE + 02$, la señal *init* a la dirección $BASE + 04$, la señal *done* a la dirección $BASE + 06$, las parte alta y baja del resultado a las direcciones $BASE + 08$ y $BASE + 0A$ respectivamente y. En total 6 registros, por lo que solo se necesitarán las primeras 4 líneas del bus de direcciones (*io_addr[3:0]*) para seleccionar a qué registro se quiere acceder.

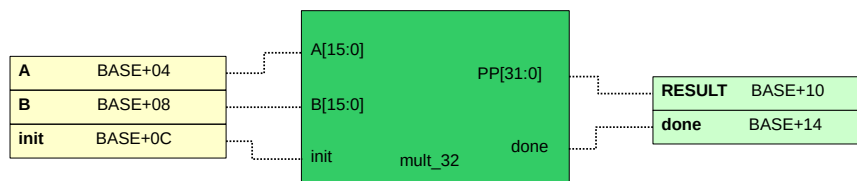


Figura 1.9 Mapa de memoria para el multiplicador

Una vez definida la dirección de memoria para la información de entrada y salida del periférico se debe adaptar el diseño para que permita el intercambio de información con la CPU. En la figura 1.10 se muestra el diagrama de bloques del periférico multiplicador, en él podemos identificar dos multiplexores conectados a los buses de datos (*dout* y *din*) encargados de seleccionar a qué señal se escribirá/leerá la señal desde/hacia la CPU.

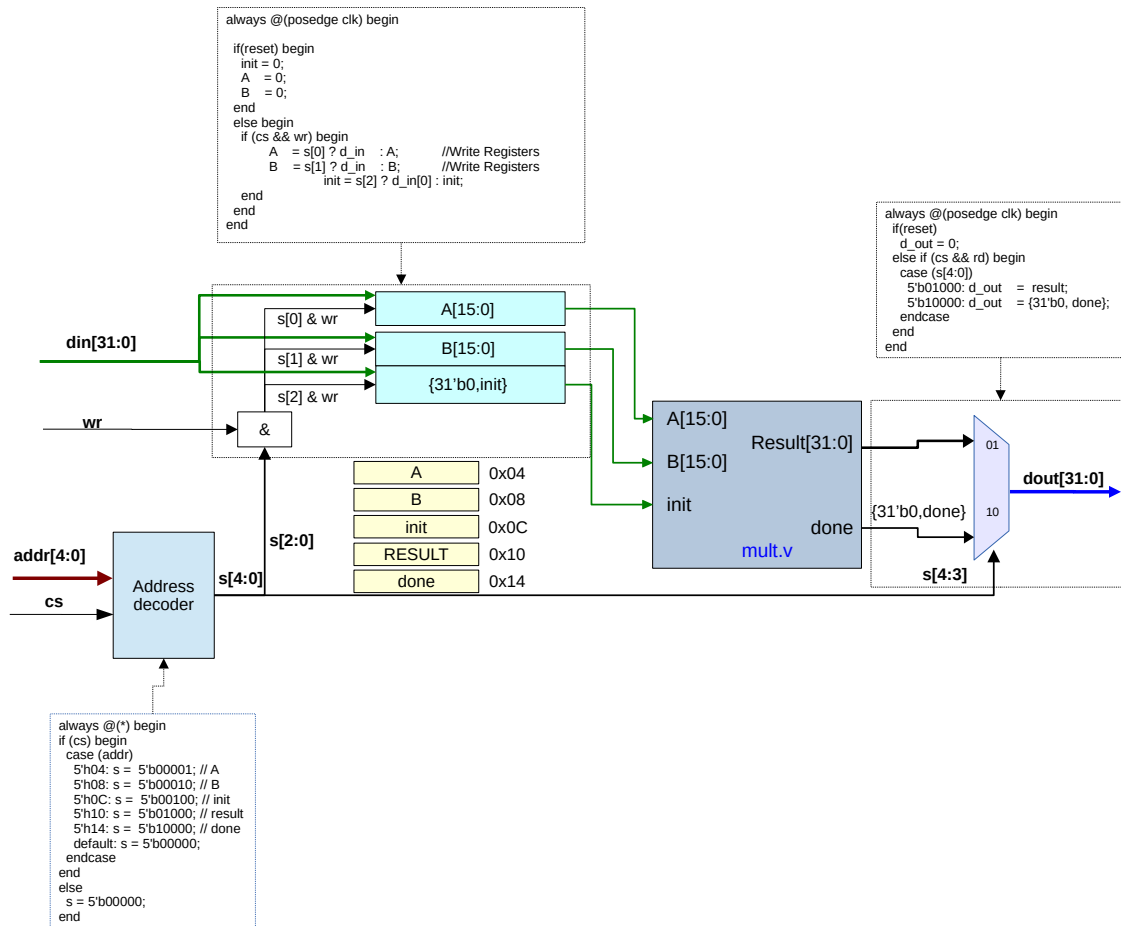


Figura 1.10 Diagrama de bloques del periférico multiplicador

1.3.1 Lectura del periférico

En la figura 1.11 se muestra el diagrama de bloques simplificado para la lectura. En ella podemos observar que la señal de control `s[5:3]` selecciona lo que se enviará por el bus de datos de salida `dout`, es decir, `PP[31:0]` o `done`, el decodificador de direcciones interno activa una de estas señales de acuerdo con el mapa de memoria del dispositivo tal como se indica en el código en verilog de la figura 1.11. Este decodificador es un bloque combinatorio que activa las señales de selección `s[5:3]` siempre y cuando las señales `cs` (periférico seleccionado) y `rd` (señal de lectura del procesador) estén activas.

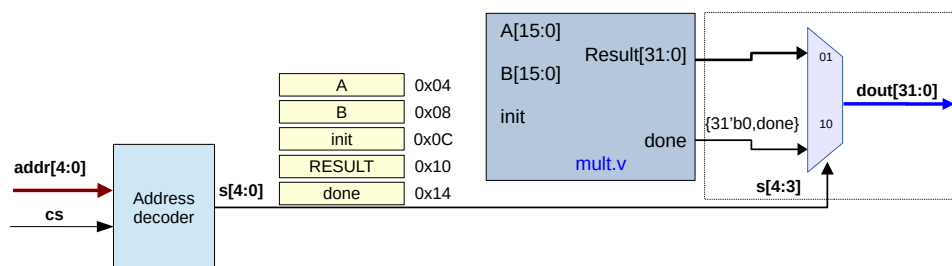


Figura 1.11 Diagrama de bloques para la lectura del multiplicador

1.3.2 Escritura del periférico

En la figura 1.12 se muestra el diagrama de bloques simplificado para la escritura; de forma similar a la lectura de datos desde la CPU a los periféricos, el bus de datos de entrada (*din*) está conectado a todos los registros *A*, *B* e *init*, el codificador de direcciones controla la activación de dichos registros; en este caso se utilizan registros ya que es necesario mantener los operandos en el valor deseado durante la ejecución del algoritmo de multiplicación. Se muestra el código del módulo combinatorio que realiza la decodificación de las direcciones de escritura el cual activa las señales *s[0:2]* siempre y cuando las señales *cs* y *wr* estén activas lo que indica un ciclo de escritura al periférico.

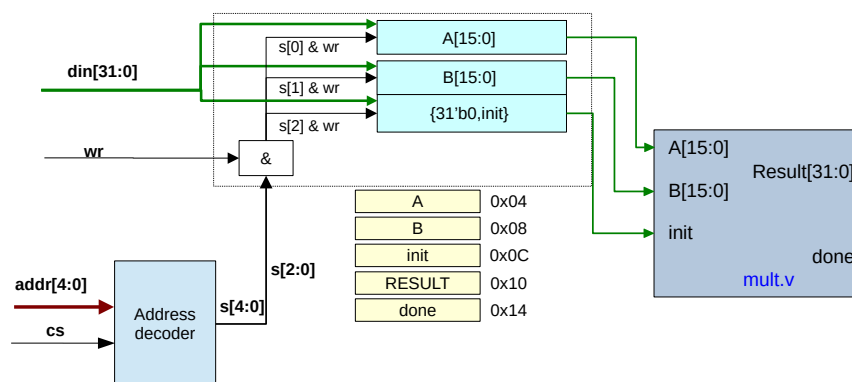


Figura 1.12 Diagrama de bloques para la escritura del multiplicador

En la figura 1.13 podemos observar las formas de onda de un ciclo completo de operación del multiplicador, en este ejemplo realizamos la multiplicación $0x5 * 0xF$ ($5 * 16$), el bus de datos de entrada al periférico se resaltan en azul, las direcciones en amarillo; aquí podemos observar que cuando las señales *cs* = 1 y *wr* = 1, el bus de direcciones toma el valor **0x04** y el de datos 0x0005 indicando la escritura del primer operando; en el siguiente ciclo de escritura el bus de direcciones toma el valor de **0x08** y el de datos 0x0F con lo que se carga el segundo operando, en la tercera operación de escritura el bus de direcciones es **0x0C** y el dato 1, con lo que se inicia el algoritmo de multiplicación con estos dos operandos, en este momento inicia la ejecución del algoritmo de multiplicación (lo que demora un determinado número de ciclos de reloj); al finalizar debemos leer el estado del flag **done** en la dirección de memoria **0x14** (*cs* = *rd* = 1), si su valor es igual a 1 se realiza la lectura (*cs* = *rd* = 1) del resultado en la dirección de memoria **0x10**, que en este caso es el valor 75 (0x4B).

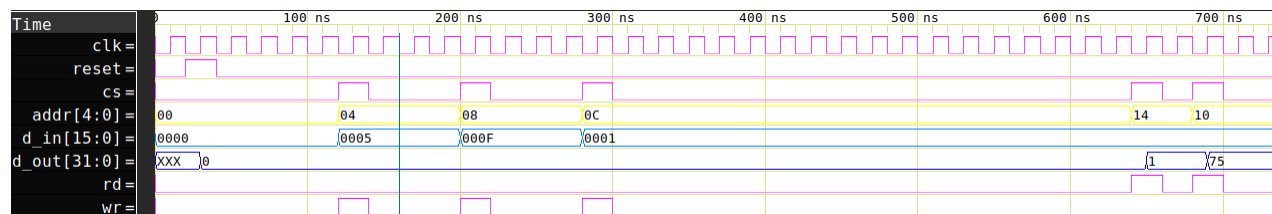


Figura 1.13 Simulación de la multiplicación $0x5 * 0xF$

1.3.3 Implementación de la funcionalidad del periférico usando Lógica de transferencia de registros

La lógica de transferencia de registros es una descripción comportamental a nivel funcional, que puede ser sintetizada a una descripción estructural utilizando registros, una unidad que realiza operaciones aritméticas/lógicas y una máquina de control que maneja la transferencia o envío de datos de un registro a otro. La secuencia o flujo de datos al realizar ese

proceso de transferencia: registros, unidad aritmético-lógica, registros; es lo que permite que se generen los algoritmos. Así, todo sistema secuencial (algoritmo) puede ser visto como la unión de un camino de datos (registros y operaciones) y una unidad de control que determina cuándo y cuáles operaciones se deben realizar, al igual que las transferencia o movimiento de datos entre registros.

En este nivel de descripción los diseñadores abordan la solución de un problema a nivel de operaciones y de transferencia de datos entre registros, lo cual ocurre durante muchos ciclos de reloj [REF GAJSKI BUSCAR]. En este nivel de abstracción no se aborda el problema desde la perspectiva de las dimensiones geométricas o conexiones físicas concretas del sistema, sino que se aborda desde el nivel comportamental desarrollado por cada uno de los pasos que permiten realizar esa manipulación de datos almacenados en registros, hasta la ejecución de operaciones aritmético lógicas; finalizando nuevamente en registros en donde se van guardando los resultados. Es este el punto de partida para abordar el desarrollo estructurado de soluciones en hardware, que para problemas de mediana y baja complejidad hacen que sea una solución acertada.

Una primera alternativa para llegar a diseñar un sistema digital nace de las propias necesidades que entregan las especificaciones del problema, en donde se busca realizar la integración de diferentes bloques, elementos de librería o periféricos (p.e decodificadores, multiplexores, sumadores, restadores, contadores, registros de desplazamiento) que conlleven a tener una arquitectura del sistema. Esta arquitectura puede llegar a tener elementos netamente combinatoriales (el valor de las salidas depende únicamente del valor de las entradas), o secuenciales (el valor de las salidas depende de su valor actual y del valor de las entradas), o la combinación de ambos.

En el caso más frecuente de sistemas digitales como lo es el síncrono, se requiere de elementos secuenciales que siempre están gobernados bajo la misma señal de reloj, la cual tiene una frecuencia de operación muy superior a cualquiera de las frecuencias de operación de las demás señales de entrada del sistema. Adicionalmente uno de los métodos más conocidos para el diseño de circuitos digitales de cierta complejidad se basa en la utilización de máquinas de estado finito “FSM” (en inglés Finite State Machine), a donde llegan todas las señales de control las cuales controlan o manipulan los demás bloques o elementos de librería que componen dicha arquitectura.

Las máquinas de estado finito utilizan una representación (1.14) por medio de círculos los cuales representan los estados o momentos de operación en determinado instante de tiempo, definido por condiciones de operación que controlan los demás bloques en dicho instante y flechas que interconectan los diferentes estados, las cuales representan los condiciones de cambio entre estados o instantes de tiempo.

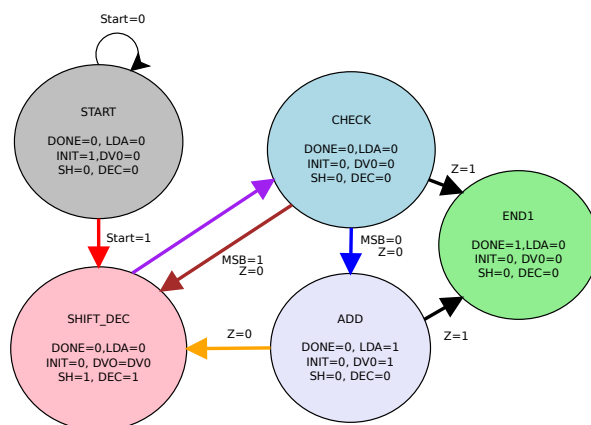


Figura 1.14 Diagrama de estados de una máquina de estado finito

En el caso de máquinas de estados complejas con gran cantidad de estados y señales, la representación con círculos y flechas puede llegar a ser confusa y muy larga. Es por esto que se hace necesario acompañar siempre cada descripción y de hecho comenzar la etapa de diseño utilizando los algoritmos que permitan manipular con rigor y consistencia las operaciones matemáticas necesarias para la ejecución de los diversos cálculos implicados. Para esto se suelen emplear grafos que muestren dependencia entre operandos y flujos que comunican los datos del algoritmo. A este tipo de notación se le conoce como máquinas de estado algorítmicas, en las cuales se tiene una representación gráfica intuitiva de las operaciones del circuito en cada ciclo de reloj.

Máquinas de Estados Algorítmicas (ASM)

La lógica de transferencia de registros es una técnica ampliamente utilizada para la descripción a alto nivel del comportamiento de circuitos secuenciales; los cuales pueden ser vistos como un grupo de registros y operaciones aritméticas y/o lógicas que transfieren datos de un registro a otro. Todo sistema secuencial puede ser visto como la unión de un camino de datos (entre registros y la unidad aritmético lógica) y una unidad de control que determina en qué momento se deben realizar las operaciones y las transferencias de los datos entre registros. En la Figura 1.15 se puede ver la estructura fundamental de una máquina de estados algorítmica. Nótese que los datos nunca pasan por la unidad de control y que la comunicación entre el bloque de Control y el bloque Camino de datos se lleva a cabo por señales simples que deciden qué operación se debe realizar (señal control) y señales que dan información al Bloque de control sobre el estado de la operación, lo que llamaremos banderas. Estas señales dan información al control como por ejemplo si el resultado de una operación da un número negativo, el resultado da cero, hay overflow o se genera un acarreo al realizarse la operación entre otras.

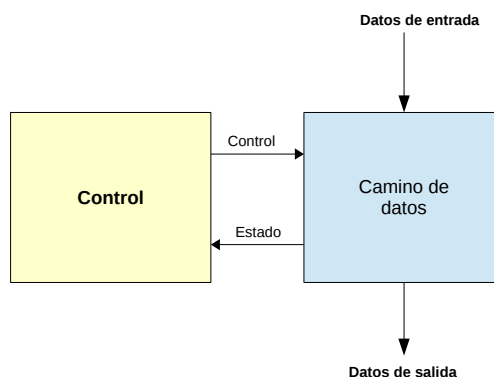


Figura 1.15 Estructura de una máquina de estados algorítmica.

A la unión de una unidad de control y un camino de datos le daremos el nombre de máquina de estados algorítmica (ASM), para diferenciarla de la máquina de estados finitos y para indicar la posibilidad de implementación de cualquier tipo de algoritmo. Los pasos que se realizan para el diseño e implementación de una máquina de estados algorítmica son los siguientes:

1. Elaboración de un diagrama de flujo que describa la funcionalidad deseada ya sea a nivel gráfico o en texto.
2. Identificación de los componentes del camino de datos.
3. Identificación de las señales necesarias para controlar el camino de datos e interconexión.
4. Especificación de la unidad de control utilizando diagramas de estado.
5. Implementación de los componentes del camino de datos y de la unidad de control utilizando lenguajes de descripción de hardware.
6. Simulación y pruebas.

ASM para la multiplicación de números binarios

El algoritmo de multiplicación que se implementará se basa en productos parciales (PP); el primer producto parcial siempre es cero (ver Figura 1.16), se realiza la multiplicación iniciando con el bit menos significativo (bit de la derecha) del multiplicador, el resultado de la multiplicación se suma al primer producto parcial y se obtiene el segundo producto parcial; si el bit del multiplicador es '0' no se afecta el contenido de PP, por lo que no se realiza la suma. A continuación se realiza la multiplicación del siguiente bit (a la izquierda del LSB) y el resultado se suma al producto parcial pero corrido un bit a la izquierda, esto para indicar que la potencia del siguiente bit tiene un grado más; este corrimiento se debe realizar ya que si un número binario se multiplica por 2 el resultado es el mismo número corrido a la izquierda; este proceso continúa hasta completar todos los bits del multiplicador y el último producto parcial es el resultado final.

$$\begin{array}{r}
 1010 \\
 * 0101 \\
 \hline
 0000 \leftarrow \text{Primer producto parcial} \\
 1010 \\
 \hline
 1010 \leftarrow \text{Segundo producto parcial} \\
 0000 \\
 \hline
 01010 \leftarrow \text{Tercer producto parcial} \\
 1010 \\
 \hline
 110010 \leftarrow \text{Cuarto producto parcial} \\
 0000 \\
 \hline
 110010 \leftarrow \text{Resultado}
 \end{array}$$

Figura 1.16 Multiplicación de números binarios usando productos parciales.

Descripción comportamental a nivel algoritmo

En la Figura 1.17 se muestra un diagrama de flujo del algoritmo que implementa la multiplicación por productos parciales descrita anteriormente. El primer paso para realizar la multiplicación es hacer que el producto parcial (PP) sea igual a cero, a continuación se realiza una verificación del bit menos significativo del multiplicador, esto se hace para sumar únicamente los resultados que no son parcialmente cero. En este caso se utiliza un corrimiento a la izquierda para obtener el siguiente bit del multiplicador, si por ejemplo al número *1010* se le realiza un corrimiento a la derecha se obtiene el número *0101*, con lo que el bit menos significativo corresponde al segundo bit de *1010*, si se realiza otro corrimiento a la derecha se obtiene *0010* y de nuevo el bit menos significativo corresponde al tercer bit de *1010*, al realizar de nuevo un corrimiento se obtiene *0001*, con lo que tendríamos todas las cifras del multiplicador de forma consecutiva en el bit menos significativo. Cuando se realiza un nuevo corrimiento el resultado es *0000* lo que indica que el producto parcial no puede cambiar y podemos terminar el algoritmo. Este método para finalizar el algoritmo produce que el número de iteraciones depende del valor del multiplicador; otra forma de terminar el algoritmo sin que dependa del valor del multiplicador se obtiene al contar el número de bits del multiplicador y realizar el corrimiento n veces, donde n es el número de bits del multiplicador.

Para indicar que cada vez que se toma un bit del multiplicador, este tiene una potencia mayor que el bit anterior, debemos multiplicar el resultado por la base, la cual es 2 en este caso; como se mencionó anteriormente, multiplicar por 2 equivale a realizar un corrimiento a la izquierda, por lo que siempre que se tome un nuevo bit del multiplicador debemos correr a la izquierda el multiplicando.

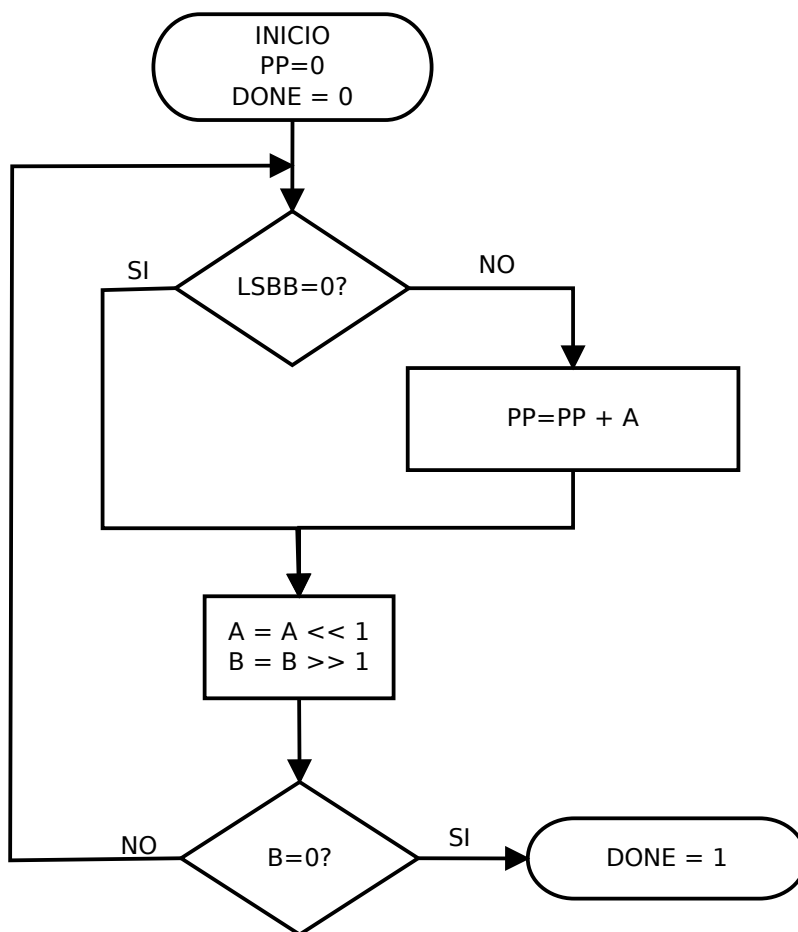


Figura 1.17 Diagrama de flujo para la multiplicación de números binarios.

Una vez conocido el funcionamiento del sistema se procede a realizar el diagrama de caja negra de entradas y salidas. En la Figura 1.18 se muestra el multiplicando y el multiplicador (A y B), señales de m bits cada una, el resultado de la multiplicación PP (Bus de 2m Bits), la señal de reloj (CLOCK). Las señales INIT y DONE se utilizan para iniciar el proceso de multiplicación e indicar que el resultado está disponible respectivamente; es importante que todo sistema digital posea la forma de interactuar con el exterior, ya que sin ello el sistema carecería de utilidad.

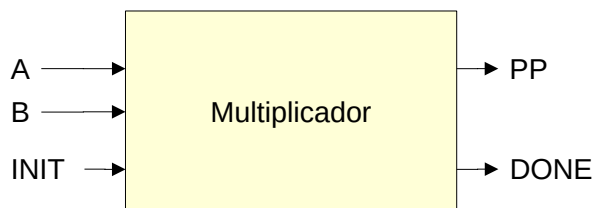


Figura 1.18 Diagrama de caja negra para el multiplicador de números binarios.

Identificación de los componentes del camino de datos

Para identificar los componentes del camino de datos, se recorre el algoritmo para encontrar las operaciones que se realizan, el resultado se aprecia en la figura 1.19, en ella se resaltan las operaciones que se deben realizar para la correcta operación del algoritmo; la primera es una operación de acumulación correspondiente a $PP = PP + A$; la

segunda operación que encontramos son los dos corrimientos a la izquierda y derecha del multiplicando (A) y el multiplicador (B) respectivamente, el último módulo es un comparador que indica que el multiplicador es igual a cero, indicando que el algoritmo puede finalizar.

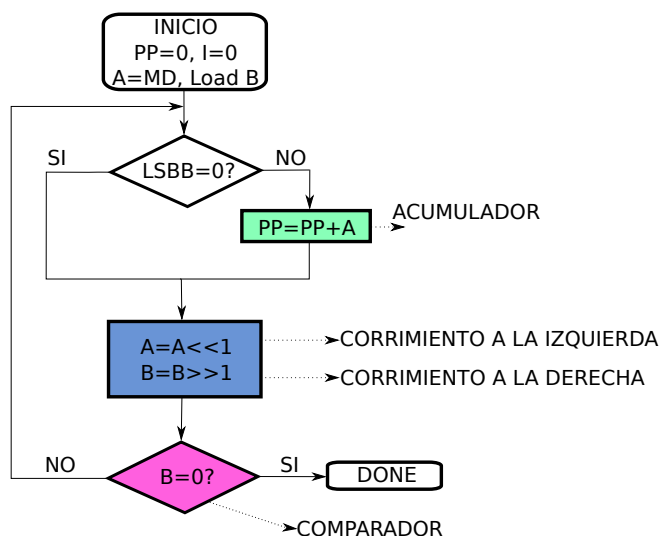


Figura 1.19 Identificación de los componentes del camino de datos para el multiplicador de números binarios.

Identificación de las señales de control e interconexión del camino de datos

En la figura 1.20 se muestra la interconexión de los componentes del camino de datos y las señales que lo controlan.

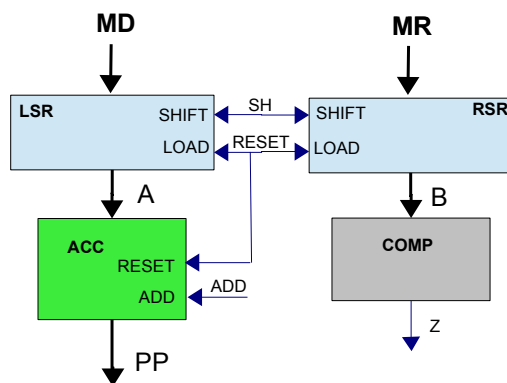


Figura 1.20 Identificación de las señales de control e interconexión del camino de datos.

En este punto se realiza una lista de acciones sobre las variables identificadas en el paso anterior. Sobre la variable PP se realizan dos acciones $PP = 0$ y $PP = PP + A$, por lo que su bloque debe tener señales que permitan realizarlas, estas son: RESET ($PP = 0$) y ADD ($PP = PP + A$), adicionalmente, $PP = PP + A$, nos indica que la entrada a este acumulador debe ser la variable A, por lo que debe existir una conexión entre la variable A y el acumulador (ACC).

Aunque es posible que la máquina de control genere muchas señales, es mejor agruparlas de acuerdo a su activación; esto es, si una operación se activa al mismo tiempo que otra, se puede utilizar una señal que las controle a ambas; para esto, se utiliza el diagrama de flujo y se observa en qué momento se realizan las operaciones. Ilustremos esto con un ejemplo: en el diagrama de flujo se observa que en el primer paso del algoritmo, se cargan los valores de las variables A y B y se inicializa en cero el acumulador, estas operaciones no se vuelven a realizar después, por este

motivo utilizaremos la misma señal (*RESET*) para realizar estas tareas. La señal que controla el momento en que el acumulador se incrementa es única, ya que solo aparece en ese punto del algoritmo y recibe el nombre de *ADD*;

A las variables A y B también se les realiza dos operaciones: se les asigna un valor (operandos) y se realizan corrimientos a la izquierda y a la derecha respectivamente. Por esta razón, ambos módulos poseen las señales *LOAD* y *SHIFT*, estas dos señales son comunes a los dos bloques ya que las tareas se realizan al mismo tiempo². Por esto, se puede utilizar la señal *SH* para generar el corrimiento.

La variable B es comparada con cero en cada ciclo para determinar la finalización del algoritmo, por lo que la entrada del comparador es la variable B. Por último, el comparador debe proporcionar una señal que indique que el valor de su entrada es igual a cero, Z en este caso.

De lo anterior se obtienen las señales de control *SH*, *RESET*, *ADD* y *DONE*, las que deben ser generadas por el control. Para determinar las entradas del módulo control, se identifican las señales involucradas en los bloques de decisión del diagrama de flujo. En este caso tenemos tres: *INIT* detecta el inicio del algoritmo, *LSBB* determina si se suma o no y Z detecta la finalización del algoritmo.

Con lo anterior podemos relizar el diagrama de bloques que aparece en la figura 1.42 que implementa este diagrama de flujo, lo que implica una síntesis (cambio de dominio de descripción) del dominio funcional al estructural.

Especificación de la unidad de control utilizando diagramas de estado

Una vez que se conoce el camino de datos, las señales que lo controlan y las señales que ayudarán a la unidad de control a tomar decisiones, se procede con la especificación de la unidad de control, la cual, se implementa con una máquina de estados finitos, por lo que la mejor forma de especificarla es utilizando un diagrama de estados; en la figura 1.21 se muestra la relación entre el diagrama de flujo y el diagrama de estados.

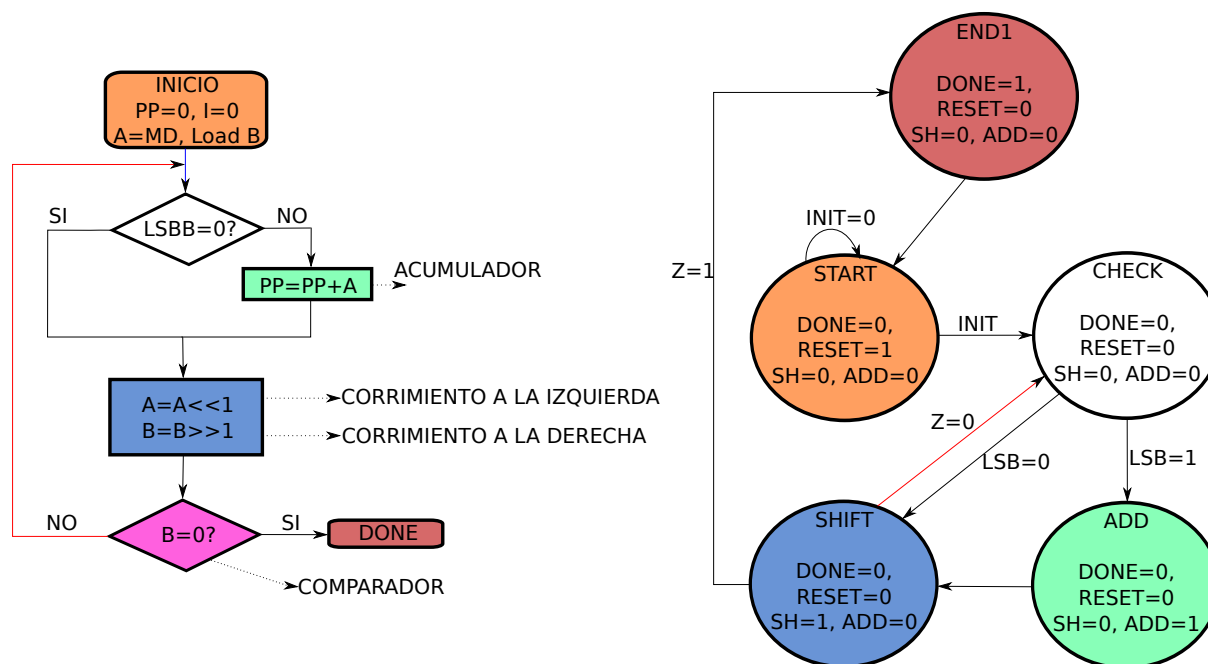


Figura 1.21 Diagrama de estados de la unidad de control del multiplicador binario.

Como se puede observar, existe una relación muy estrecha entre el diagrama de estados y el diagrama de flujo, cada operación del diagrama de flujo corresponde a un estado de la máquina de control y las transiciones entre ellos son idénticas, observe las líneas del mismo color en la figura 1.21

² A diferencia de los lenguajes de programación donde una CPU ejecuta instrucciones de forma secuencial, la implementación a nivel RTL permite realizar tareas en paralelo, por lo que el corrimiento a la derecha y a la izquierda se realizan al tiempo

La máquina de estados debe iniciar en START y se queda en este estado siempre que la señal INIT tenga un valor de '0'. En el estado START la señal RESET = '1', con lo que el valor del acumulador se hace cero y se cargan las variables A y B. Cuando INIT = '1', entramos al estado CHECK, en el que evalúa la señal LSB, si LSB = '0', no se debe realizar la suma de A, pero si se debe realizar un corrimiento para obtener el siguiente bit del multiplicador y realizar el corrimiento necesario en A. Si LSB = '1' se debe sumar el valor de A al valor del acumulador, y después se debe realizar un corrimiento. En el estado ADD se hace la salida ADD = '1' para que el valor a la entrada del acumulador se sume al valor almacenado en él. En el estado SHIFT se realiza el corrimiento de A y B haciendo el valor de la señal SH = 1. Es necesario declarar el valor de todas las salidas en todos los estados.

Implementación de los componentes del camino de datos y de la unidad de control

Hasta el momento hemos podido realizar la parte más importante de la definición de un sistema digital su concepción, modelamiento utilizando un diagrama de flujo y su síntesis a un diagrama de bloques. En este punto nos enfrentamos a la pregunta: ¿Cómo implemento estos bloques en el mundo real?. Para contestar esta pregunta dejaremos un momento este ejercicio para profundizar en la lógica combinatoria y secuencial, las que nos ayudarán en esta implementación.

1.4 Lógica Combinatoria y Secuencial

Cualquier bloque necesario para construir una máquina de estados algorítmica puede ser clasificado de dos formas: Requiere memoria o NO requiere memoria. Expresiones como: $A = B + C$, $D = A \& B$, $A \neq B$, el resultado depende únicamente del valor de los parámetros de entrada, mientras que expresiones como $A = A + 1$; $B = B \gg 1$, $C = C \ll 1$, $Z = Z + R$, su resultado depende del valor anterior que tenía la variable. Escrito más formal: $O = f(I)$ (sin memoria), $O^* = f(O, I)$ (con memoria).

Las expresiones que no requieren memoria reciben el nombre de circuitos combinatorios, y las que requieren memoria secuenciales.

1.4.1 Lógica combinatoria

Históricamente el desarrollo del diseño digital utilizaba la lógica combinatoria para optimizar la implementación de ecuaciones booleanas, se desarrollaron métodos que permitían obtener dichas ecuaciones a partir de tablas de verdad para su posterior minimización, es decir, reducir estas ecuaciones a su mínima expresión, esto con el fin de utilizar el menor número de compuertas lógicas (dispositivos electrónicos que realizan una función lógica) para su implementación.

En la figura 1.22 y en la tabla ?? se muestra el diagrama de caja negra y la tabla de verdad de un semisumador:

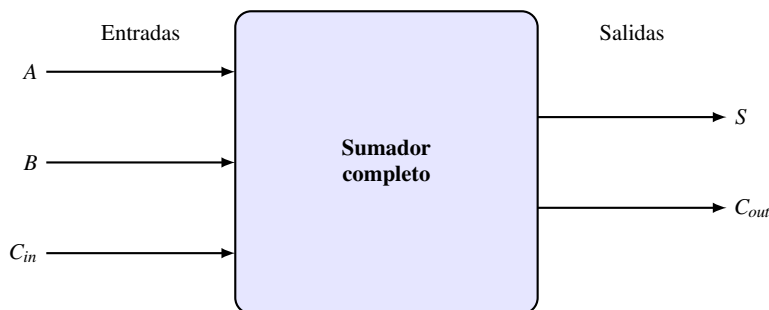


Figura 1.22 Caja negra del sumador completo

A	B	C_{in}	S	C_{out}
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

Cuadro 1.2 Tabla de verdad del sumador completo

Cuando no existían computadores, ni programas de ayuda el diseño, los ingenieros debían obtener las ecuaciones que implementaban esta tabla, esto se puede hacer de dos formas: Sumando los ‘1s’ o multiplicando los ‘0s’

A	B	C_{in}	S	C_{out}	Mintérmino	Maxitérmino
0	0	0	0	0	$m_0 = \bar{A} \cdot \bar{B} \cdot \bar{C}_{in}$	$M_0 = A + B + C_{in}$
0	0	1	1	0	$m_1 = \bar{A} \cdot \bar{B} \cdot C_{in}$	$M_1 = A + B + \bar{C}_{in}$
0	1	0	1	0	$m_2 = \bar{A} \cdot B \cdot \bar{C}_{in}$	$M_2 = A + \bar{B} + C_{in}$
0	1	1	0	1	$m_3 = \bar{A} \cdot B \cdot C_{in}$	$M_3 = A + \bar{B} + \bar{C}_{in}$
1	0	0	1	0	$m_4 = A \cdot \bar{B} \cdot \bar{C}_{in}$	$M_4 = \bar{A} + B + C_{in}$
1	0	1	0	1	$m_5 = A \cdot \bar{B} \cdot C_{in}$	$M_5 = \bar{A} + B + \bar{C}_{in}$
1	1	0	0	1	$m_6 = A \cdot B \cdot \bar{C}_{in}$	$M_6 = \bar{A} + \bar{B} + C_{in}$
1	1	1	1	1	$m_7 = A \cdot B \cdot C_{in}$	$M_7 = \bar{A} + \bar{B} + \bar{C}_{in}$

Cuadro 1.3 Tabla de verdad del sumador completo con mintérminos y maxitérminos

Mintérmino

Para una función de n variables, el mintérmino m_i es un producto lógico (AND) de todas las variables, donde cada variable aparece en forma directa si su valor es 1, o complementada si su valor es 0:

$$m_i = \prod_{j=0}^{n-1} x_j^{(b_j)}, \quad x_j^{(b_j)} = \begin{cases} x_j & \text{si el bit } b_j = 1 \\ \bar{x}_j & \text{si el bit } b_j = 0 \end{cases}$$

donde $b_{n-1} \dots b_1 b_0$ es la representación binaria del índice i . El mintérmino m_i vale 1 únicamente en la fila i de la tabla de verdad y 0 en todas las demás. Cualquier función lógica puede expresarse como suma de mintérminos (forma canónica SOP):

$$f(x_{n-1}, \dots, x_0) = \sum m(\text{filas donde } f = 1)$$

Maxitérmino

El maxitérmino M_i es una suma lógica (OR) de todas las variables, donde cada variable aparece en forma directa si su valor es 0, o complementada si su valor es 1:

$$M_i = \sum_{j=0}^{n-1} x_j^{(\bar{b}_j)}, \quad x_j^{(\bar{b}_j)} = \begin{cases} x_j & \text{si el bit } b_j = 0 \\ \bar{x}_j & \text{si el bit } b_j = 1 \end{cases}$$

El maxitérmino M_i vale 0 únicamente en la fila i y 1 en todas las demás. Cualquier función lógica puede expresarse como producto de maxitérminos (forma canónica POS):

$$f(x_{n-1}, \dots, x_0) = \prod M(\text{filas donde } f = 0)$$

Relación entre mintérmino y maxitérmino

Mintérmino y maxitérmino del mismo índice son complementos entre sí:

$$M_i = \overline{m_i}$$

Por lo tanto, las filas de mintérminos y maxitérminos siempre son complementarias y juntas cubren los 2^n renglones de la tabla de verdad.

A partir de la tabla de verdad, las salidas S y C_{out} se expresan:

Forma canónica SOP (suma de mintérminos)

$$\begin{aligned} S &= \sum m(1, 2, 4, 7) = m_1 + m_2 + m_4 + m_7 \\ &= \bar{A} \cdot \bar{B} \cdot C_{in} + \bar{A} \cdot B \cdot \bar{C}_{in} + A \cdot \bar{B} \cdot \bar{C}_{in} + A \cdot B \cdot C_{in} \end{aligned} \quad (1.1)$$

$$\begin{aligned} C_{out} &= \sum m(3, 5, 6, 7) = m_3 + m_5 + m_6 + m_7 \\ &= \bar{A} \cdot B \cdot C_{in} + A \cdot \bar{B} \cdot C_{in} + A \cdot B \cdot \bar{C}_{in} + A \cdot B \cdot C_{in} \end{aligned} \quad (1.2)$$

Forma canónica POS (producto de maxitérminos)

$$\begin{aligned} S &= \prod M(0, 3, 5, 6) = M_0 \cdot M_3 \cdot M_5 \cdot M_6 \\ &= (A + B + C_{in})(A + \bar{B} + \bar{C}_{in})(\bar{A} + B + \bar{C}_{in})(\bar{A} + \bar{B} + C_{in}) \end{aligned} \quad (1.3)$$

$$\begin{aligned} C_{out} &= \prod M(0, 1, 2, 4) = M_0 \cdot M_1 \cdot M_2 \cdot M_4 \\ &= (A + B + C_{in})(A + B + \bar{C}_{in})(A + \bar{B} + C_{in})(\bar{A} + B + C_{in}) \end{aligned} \quad (1.4)$$

Minimización algebraica del sumador completo (mintérminos)

$$S = \bar{A}\bar{B}C_{in} + \bar{A}B\bar{C}_{in} + A\bar{B}\bar{C}_{in} + ABC_{in} \quad (1.5)$$

$$= C_{in}(\bar{A}\bar{B} + AB) + \bar{C}_{in}(\bar{A}B + A\bar{B}) \quad (\text{factor } C_{in} \text{ y } \bar{C}_{in}) \quad (1.6)$$

Ningún par de términos difiere en una sola variable, por lo tanto **no es posible reducir** el número de términos. La expresión factorizada equivale a ($\bar{A}\bar{B} + \bar{A}B$ equivale a $A \oplus B$):

$$S = C_{in}(\overline{A \oplus B}) + \bar{C}_{in}(A \oplus B) \quad (1.7)$$

$$\boxed{S = A \oplus B \oplus C_{in}} \quad (1.8)$$

Minimización de C_{out} — SOP (mintérminos)

$$C_{out} = \bar{A}BC_{in} + A\bar{B}C_{in} + AB\bar{C}_{in} + ABC_{in} \quad (1.9)$$

$$= \bar{A}BC_{in} + ABC_{in} + A\bar{B}C_{in} + ABC_{in} + AB\bar{C}_{in} + ABC_{in} \quad (\text{idempotencia: } X + X = X) \quad (1.10)$$

$$= BC_{in}(\bar{A} + A) + AC_{in}(\bar{B} + B) + AB(\bar{C}_{in} + C_{in}) \quad (\text{factor común}) \quad (1.11)$$

$$= BC_{in} \cdot 1 + AC_{in} \cdot 1 + AB \cdot 1 \quad (\text{complemento: } X + \bar{X} = 1) \quad (1.12)$$

$$= BC_{in} + AC_{in} + AB \quad (1.13)$$

$$\boxed{C_{out} = AB + AC_{in} + BC_{in}} \quad (1.14)$$

Minimización de S — POS (maxitérminos)

$$S = (A + B + C_{in})(A + \bar{B} + \bar{C}_{in})(\bar{A} + B + \bar{C}_{in})(\bar{A} + \bar{B} + C_{in}) \quad (1.15)$$

Se intentan agrupar pares que difieran en una sola variable. Ningún par cumple esta condición (patrón de tablero de ajedrez), por lo tanto **no es posible reducir** el número de factores. La expresión equivale a:

$$\boxed{S = A \oplus B \oplus C_{in}} \quad (1.16)$$

4. Minimización de C_{out} — POS (maxitérminos)

Se aplica la identidad dual: $(X + a)(X + \bar{a}) = X$. Primero se expande M_0 por idempotencia para formar tres pares:

$$C_{out} = (A + B + C_{in})(A + B + \bar{C}_{in})(A + \bar{B} + C_{in})(\bar{A} + B + C_{in}) \quad (1.17)$$

$$= \underbrace{(A + B + C_{in})(A + B + \bar{C}_{in})}_{\text{par } M_0 \cdot M_1} \cdot \underbrace{(A + B + C_{in})(A + \bar{B} + C_{in})}_{\text{par } M_0 \cdot M_2} \cdot \underbrace{(A + B + C_{in})(\bar{A} + B + C_{in})}_{\text{par } M_0 \cdot M_4} \quad (\text{idempotencia}) \quad (1.18)$$

Par $M_0 \cdot M_1$ — difieren en C_{in} :

$$\begin{aligned} (A + B + C_{in})(A + B + \bar{C}_{in}) &= (A + B) + C_{in}\bar{C}_{in} && (\text{distributiva}) \\ &= (A + B) + 0 = A + B && (\text{complemento}) \end{aligned} \quad (1.19)$$

Par $M_0 \cdot M_2$ — difieren en B :

$$\begin{aligned} (A + B + C_{in})(A + \bar{B} + C_{in}) &= (A + C_{in}) + B\bar{B} && (\text{distributiva}) \\ &= (A + C_{in}) + 0 = A + C_{in} && (\text{complemento}) \end{aligned} \quad (1.20)$$

Par $M_0 \cdot M_4$ — difieren en A :

$$\begin{aligned} (A + B + C_{in})(\bar{A} + B + C_{in}) &= (B + C_{in}) + A\bar{A} && (\text{distributiva}) \\ &= (B + C_{in}) + 0 = B + C_{in} && (\text{complemento}) \end{aligned} \quad (1.21)$$

Combinando los tres resultados:

$$\boxed{C_{out} = (A + B)(A + C_{in})(B + C_{in})} \quad (1.22)$$

Resumen

Salida	SOP mínima	POS mínima
S	$A \oplus B \oplus C_{in}$	$A \oplus B \oplus C_{in}$
C_{out}	$AB + AC_{in} + BC_{in}$	$(A + B)(A + C_{in})(B + C_{in})$

1.4.2 Minimización gráfica utilizando mapas de Karnaugh

Como puede verse en este ejemplo sencillo, la minimización manual implica una serie de pasos que dependen mucho de la experiencia del diseñador y se pueden obtener resultados poco optimizados o erróneos. Para facilitar este proceso, hace muchos años idearon un método gráfico que permitía obtener de forma rápida estas ecuaciones, este método llamado *mapas de Karnaugh*.

En la figura 1.23 se muestra el diagrama de Karnaugh para nuestro ejemplo. En este diagrama se deben colocar los unos y ceros de la función a minimizar.

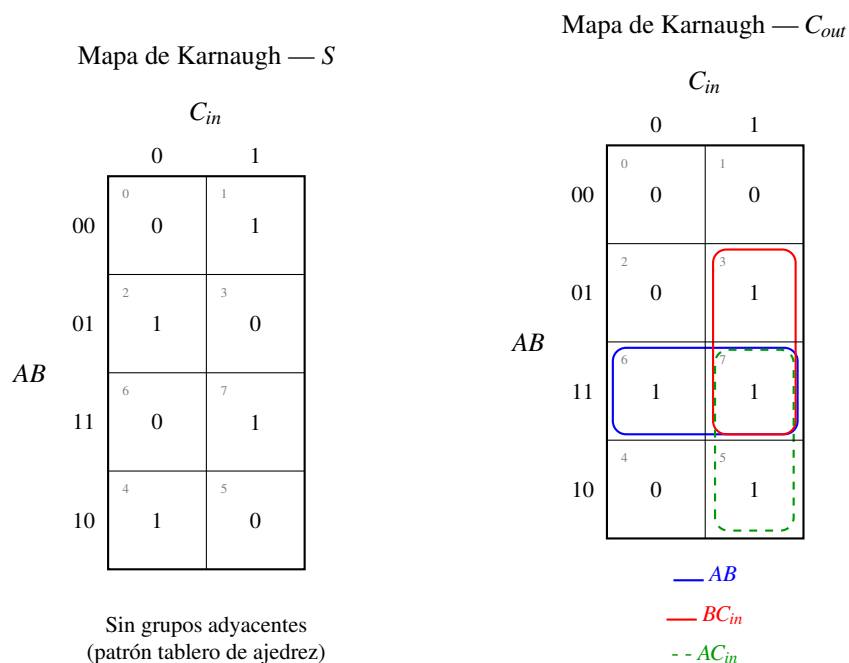


Figura 1.23 Mapas de Karnaugh del sumador completo (mintérminos)

En un mapa de Karnaugh, dos celdas son **adyacentes** si difieren en exactamente **una** variable. Esta condición es precisamente la que permite aplicar el teorema de complemento del álgebra booleana:

$$X \cdot \bar{X} = 0 \quad \text{y} \quad X + \bar{X} = 1 \quad (1.23)$$

Del mapa de Karnaugh de C_{out} , el grupo $m_6 + m_7$:

$$m_6 = A \cdot B \cdot \bar{C}_{in}, \quad m_7 = A \cdot B \cdot C_{in}$$

Estos dos mintérminos difieren únicamente en C_{in} , por lo que:

$$m_6 + m_7 = ABC_{in} + ABC_{in} \quad (1.24)$$

$$= AB(\bar{C}_{in} + C_{in}) \quad (\text{factor común } AB) \quad (1.25)$$

$$= AB \cdot 1 \quad (\text{complemento}) \quad (1.26)$$

$$= AB \quad (C_{in} \text{ eliminada}) \quad (1.27)$$

La variable C_{in} se elimina porque en las dos celdas agrupadas toma todos sus posibles valores (0 y 1), lo que significa que el resultado AB es **independiente** de C_{in} .

Generalización para grupos de 2^k celdas

Este mismo principio se extiende a grupos más grandes:

- Un grupo de 2 celdas elimina **1** variable.
- Un grupo de 4 celdas elimina **2** variables.
- Un grupo de 8 celdas elimina **3** variables.
- Un grupo de 2^k celdas elimina k variables.

En general, para una función de n variables, un grupo de 2^k celdas produce un término con $n - k$ variables:

$$\underbrace{2^k}_{\text{celdas agrupadas}} \longrightarrow \underbrace{n - k}_{\text{variables restantes en el término}} \quad (1.28)$$

Por esta razón, cuanto mayor es el grupo, más sencillo es el término resultante, y la minimización consiste en encontrar los grupos más grandes posibles que cubran todos los minterminos de la función.

1.4.3 Herramientas modernas para lógica combinatoria

El proceso de minimización mostrado anteriormente se realizaba hace más de 30 años cuando no existían computadores, afortunadamente, el mundo ha cambiado y hoy en día existen un sin número de aplicaciones que aplican algoritmos modernos de minimización. Una de estas aplicaciones es el proyecto *Digital*³. Esta herramienta permite análisis y síntesis de circuitos combinacionales, se puede generar un circuito a partir de una tabla de verdad y la minimización se hace automáticamente por el método de Quine-McCluskey.

En la figura 1.24 se puede ver como a partir de una tabla de verdad *Digital* genera el mapa de Karnaugh y las ecuaciones minimizadas, en esta figura se muestra la posibilidad que tiene la herramienta para simular a nivel de compuertas al tiempo que muestra las formas de onda de esta simulación.

1.4.4 Lenguajes de Descripción de Hardware

Los métodos de implementación presentados anteriormente no son utilizados a nivel industrial, son útiles para mostrar la importancia de la minimización en el proceso de diseño y con fines didácticos. La industria electrónica ha avanzado mucho en los últimos años y los dispositivos digitales son cada vez más complejos y están formados por procesadores y periféricos, pensar en implementar estos sistemas con herramientas del siglo pasado sería una tarea titánica e inútil, ya que en la actualidad se cuenta con herramientas que permiten implementar sistemas complejos de forma fácil. Una de estas herramientas son los *Lenguajes de Descripción de Hardware*, los que utilizando una descripción *similar* a un programa en C, implementan la funcionalidad de los componentes que hacen parte de un sistema digital.

EL lenguaje en sí es un grupo de reglas de sintaxis que permiten describir lógica combinatoria y secuencial en un archivo de texto, el cual, es la entrada de un programa que lo convierte en un netlist, es decir, una representación de una conexión de compuertas básicas. Este programa recibe el nombre de *sintetizador*, existen diferentes sintetizadores

³ <https://github.com/hneemann/Digital>

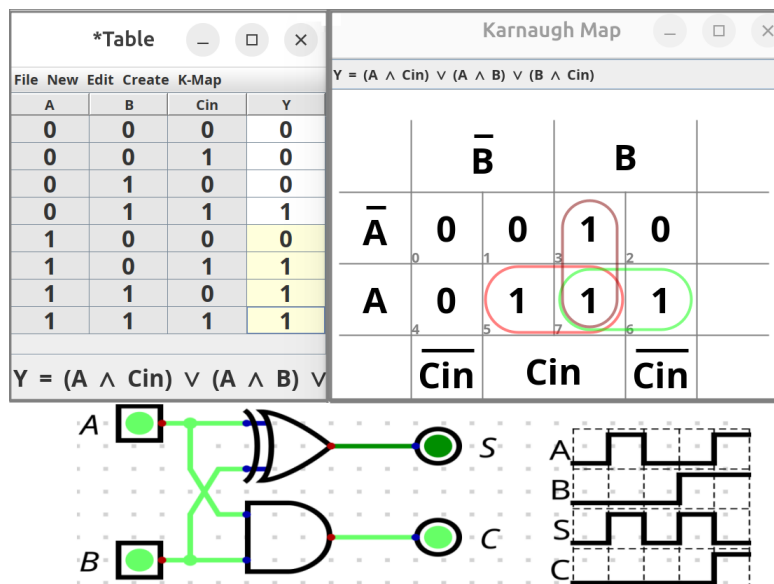


Figura 1.24 Uso de la herramienta Digital

proporcionados por los diferentes fabricantes de Chips (Altera, AMD, Efinix, Lattice, GoWin, etc). El proyecto **Yosys** es una opción abierta que será utilizada en las primeras implementaciones en este libro, yosys permite generar netlists para varias FPGAs (Lattice, Efinix, Xilinx, Intel y GoWin) y puede generar netlists para procesos de fabricación de chips. Todo esto hace que Yosys sea la Herramienta perfecta para este curso.

La figura 1.25 muestra el flujo de diseño en yosys, en el primer nivel se encuentran los LDH que recibe: Verilog, SystemVerilog y VHDL; En el segundo nivel aparecen las diferentes herramientas que posee y que permiten convertir estas entrada en las salidas del tercer nivel, netlist para FPGAs y para flujo de diseño ASIC. La salida de Yosys es utilizada por simuladores como Icarus Verilog (iverilog) y Symbi Yosys.

1.4.5 Descripción de módulos combinatorios usando el HDL Verilog

A manera de Ejemplo realizaremos la descripción del sumador implementado con el método antiguo. En la Figura 1.26 se muestra el proceso de síntesis y simulación de la descripción en verilog del modulo semi-sumador. El archivo fuente *half_adder.v* implementa la funcionalidad utilizando un módulo (module). Lo primero que debemos definir son las entradas y salidas del módulo, esto se hace con declarando unas señales como **input** u **output**; las salidas son asignadas dependiendo del valor de las entradas (*a*, *b*, *cin*), la forma de implementar una tabla de verdad en verilog es utilizando la sentencia **case**, para este ejemplo la sentencia evalúa el valor de la entrada {cin, a, b}, aquí los corchetes denotan concatenación, es decir los bits individuales forman una señal de 3 bits que será evaluada para determinar el valor de las señales de salida.

La mayoría de aplicaciones que utilizaremos en este libro se ejecutan en una consola de texto en la que se deben pasar parámetros de entrada y salida, estos comandos a menudo son largos y no es fácil recordarlos, por esta razón, se utiliza el script *Makefile* que se ejecuta con la herramienta *make*. El Makefile del ejemplo tiene tres opciones que puede ejecutar: *make sim* realiza la simulación del circuito utilizando *icarus verilog* (iverilog) y el visor de formas de onda *gtkwave*.

Banco de pruebas en verilog (TestBench)

En verilog existen dos tipos de archivos y se dividen en sintetizables, esto es, se van a convertir en un circuito y los no sintetizables, es decir, no se usan para describir circuitos; Estos últimos se usan para generar un banco de pruebas en el

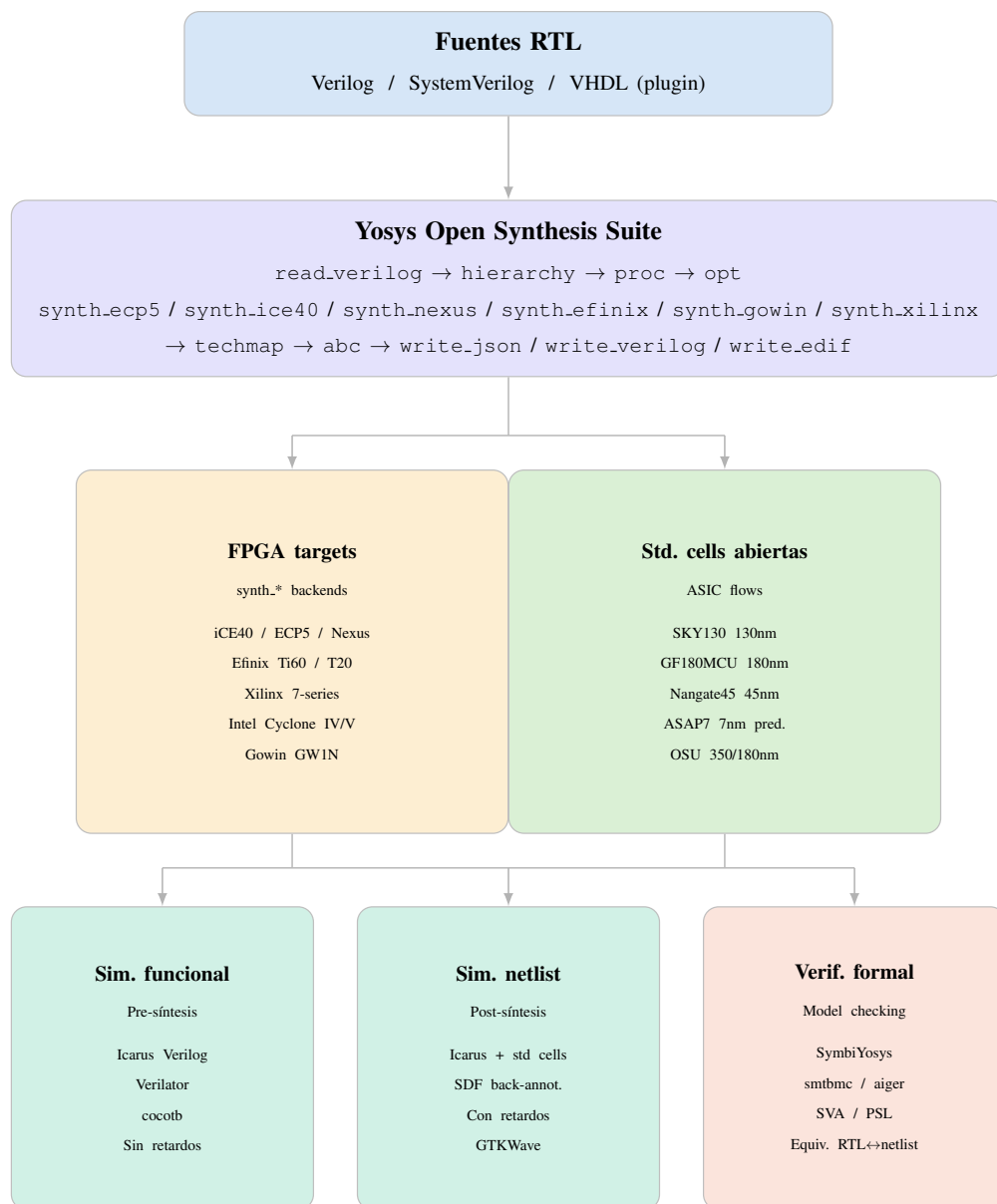


Figura 1.25 Flujo de Yosys: RTL → síntesis → netlists (FPGA / Std. cells) → simulación

que se aplican vectores de puebas al módulo que se ve a sintetizar. En el archivo *half_adder_TB.v* se declara un módulo llamado *half_adder_TB*, que a diferencia de un módulo sintetizable no tiene puertos de entrada o salida (debido a que no se convertirá en un circuito), en lugar de los puertos se deben declarar las señales que generarán las señales que se aplicarán al circuito bajo prueba (*UUT*). A continuación se realiza la instanciación de la unidad bajo prueba:

```
half_adder uut(a(a), .b(b), .cin(cin));
```

Aquí: **half_adder** es el nombre del módulo a simular, **uut** el nombre dado a esta instancia, entre paréntesis va la asignación de puertos del módulo a las señales que generan los vectores de prueba. la señal con el punto "." debe estar dentro de la lista de señales del módulo y entre paréntesis se coloca la señal a la que se conecta, en este caso las señales que generan los vectores y las señales del módulo tienen el mismo nombre. La generación de vectores de prueba se realiza en un bloque delimitado por *initial begin end*:

```
#10 a = 0; b = 0; cin = 0;
```

Acá: #10 indica que después de 10 unidades de tiempo haga $a = b = cin = 0$, acá se asignan las 8 posibles señales que pueden tomar las tres variables de entrada. El comando: `iverilog half_adder_TB.v half_adder.v`

Le indica al simulador icarus verilog que realice la simulación del archivo `half_adder.v` utilizando los vectores de prueba generados en `half_adder_TB.v`, como resultado de esta simulación se genera el archivo **a.out**, que al ser pasado como parámetro a la aplicación **vvp** genera el archivo en un formato compatible con el visor de formas de onda **gtkwave**, el nombre del archivo se indica en el testbench (`dumpfile("half_adder_TB.vcd");`);

Al ejecutar el comando `make sim` en el directorio donde se encuentran los archivos `half_adder_TB.v`, y `half_adder.v`, la herramienta `make` ejecuta los pasos necesario para realizar la simulación y abrir la aplicación `gtkwave` con los vectores resultado de esta simulación.

Al ejecutar `make svg`, `yosys` genera un archivo gráfico con formato `svg` que muestra el circuito sintetizado y minimizado, como podemos ver: $carry = (A + B * Cin) * (B + Cin)$

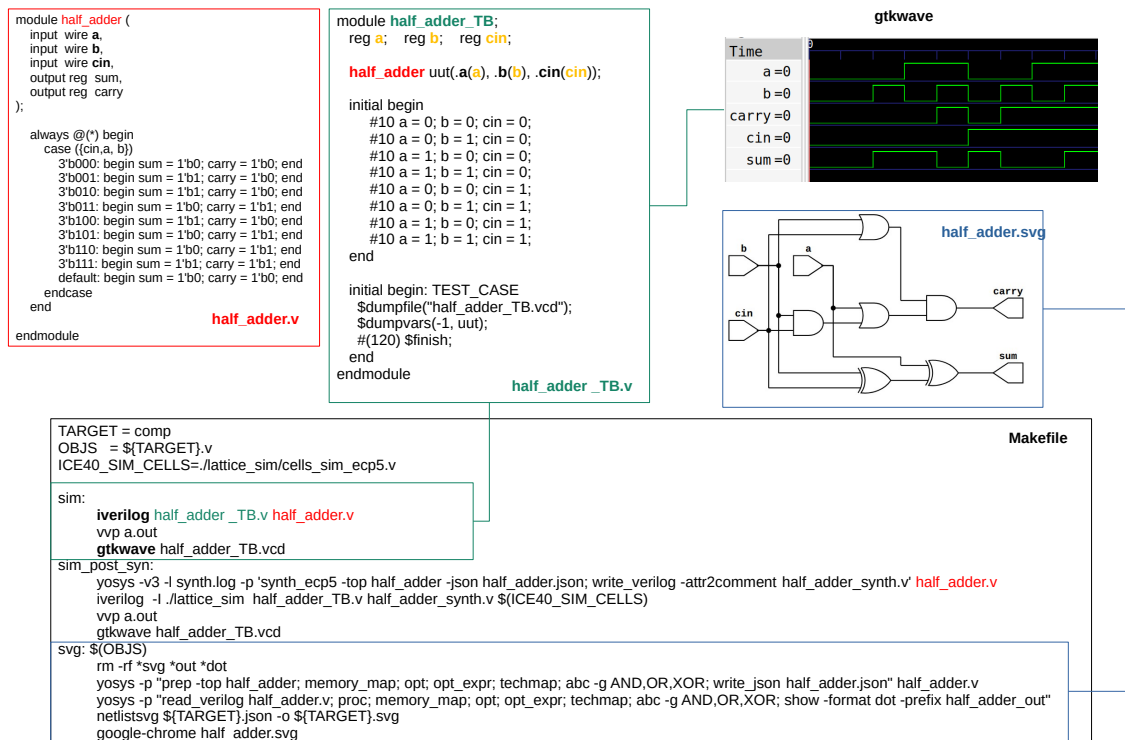


Figura 1.26 Síntesis de un semi-sumador

En el repositorio que acompaña este libro⁴ podemos encontrar estos archivos en el directorio: *digital_UN/basic_circuits/Verilog*. Para poder utilizar este repositorio es necesario instalar las herramientas de síntesis y simulación⁵ en una distribución de Linux⁶ (se recomienda utilizar versiones basadas en ubuntu, debian).

Una vez clonado el repositorio e instaladas las herramientas se debe ejecutar el comando:

`make sim TARGET=half_adder`. Esto abrirá una sesión de `gtkwave` donde se podrán seleccionar las señales de entrada y salida. El Make file de este directorio permite simular varios circuitos solo cambiando el nombre del TARGET, por ejemplo, para simular un comparador se debe ejecutar: **`make sim TARGET=comp`**

⁴ https://github.com/cicamargoba/digital_UN

⁵ <https://github.com/johnnycubides/digital-electronic-1-101/blob/main/installTools/conda-and-tools.md>

⁶ <https://github.com/johnnycubides/digital-electronic-1-101/blob/main/installTools/how-install-linux.md>

1.4.6 Lógica Secuencial

Los circuitos secuenciales son aquellos que requieren memoria, el valor siguiente de la salida depende de su valor actual y de las entradas. Intuitivamente se piensa en una realimentación de las salida a la entrada del circuito. En esta sección se realizará una descripción de los elementos de memoria de forma progresiva hasta llegar al elemento más utilizado para este fin el Flip-Flop. Estos circuitos fueron simulados en la aplicación *Digital* y se pueden encontrar en el repositorio del libro⁷.

Latch Set - Reset

Al realimentar la salida de dos compuertas NOR como se muestra en la figura 1.27 se obtiene el *latch Set - Reset*, este circuito permite almacenar un bit al fijar las señales S-R en el valor deseado. De las formas de onda de esta figura podemos deducir que si $R = 1$ y $S = 0$, Q es igual a 0; si $R = 0$ y $S = 1$, Q es igual a 1; cuando $R = 0$ y $S = 0$, la salida conserva el estado. (la señal $\bar{Q}$ es el negado de la señal Q)

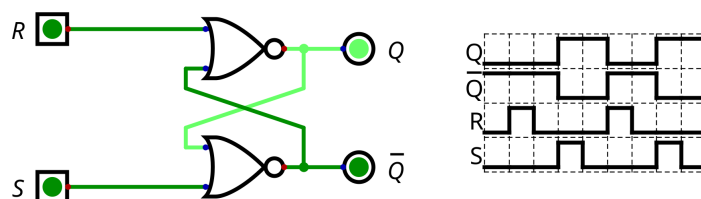


Figura 1.27 Latch Set Reset

Latch Set - Reset con habilitador

Es deseable que el elemento de control realice el almacenamiento únicamente cuando una señal de activación lo permita, esto facilitará la coordinación de módulos. Para esto se implementa el circuito mostrado en la figura 1.28, las compuertas and hacen que las entradas del latch sean 0, cuando $C = 0$; y sean iguales a R y S cuando $C = 1$. Como se ve en las formas de onda, el comportamiento es igual al circuito anterior cuando $C = 1$; cuando $C = 0$, el circuito conserva el estado.

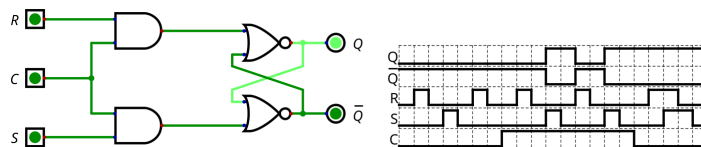


Figura 1.28 Latch Set Reset con habilitación

Latch tipo D con habilitador

El latch tipo D simplifica el circuito anterior reduciendo el número de entradas necesarias para definir el valor a almacenar, como se vé en las formas de onda de la figura 1.29 cuando la señal $C = 1$, $Q = D$, cuando $C = 0$ se conserva el estado almacenado.

⁷ digital_UN/basic_circuits/Digital

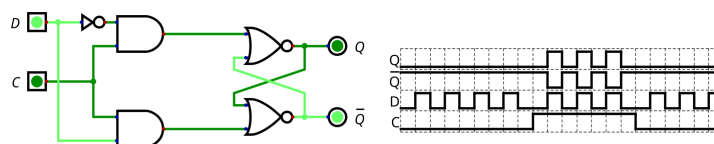


Figura 1.29 Latch tipo D con habilitación

Inferencia de latch por descripción incompleta en verilog

Los latches aparecen cuando una descripción en verilog no es del todo correcta, cuando no se especifican todos los posibles casos en un bloque de decisión *if* o en un *case*. La figura 1.30 muestra el circuito sintetizado cuando se infieren latches (derecha) y cuando no (izquierda). En la descripción de la derecha no se especifica el valor de la salida *q* cuando la señal *en* = 0, el sintetizador entonces conserva el estado previamente asignado para no alterar la descripción, para esto utiliza un latch tipo D.

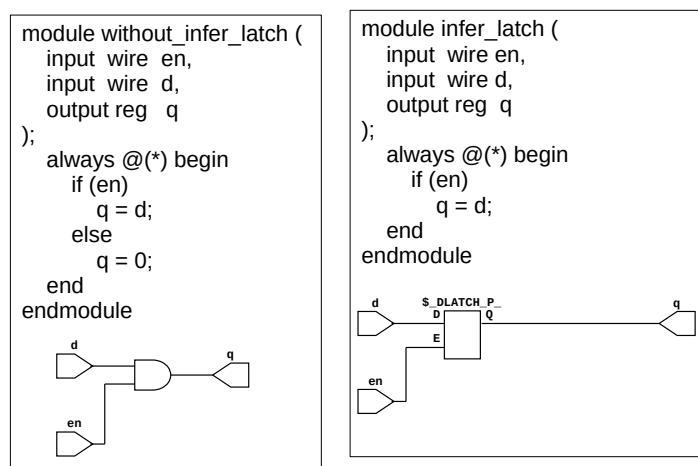


Figura 1.30 Inferencia de Latch

Flip - Flop tipo D

En los sistemas secuenciales modernos, todos los componentes utilizan la misma señal para implementar su funcionalidad (almacenar, contar, hacer corrimientos, etc), esto se asemeja a una orquesta sinfónica donde el instante en que se ejecuta una nota debe ser preciso, el metrónomo indica cuando se deben ejecutar estos cambios, los cuales deben ocurrir en un instante de tiempo exacto. Adicionalmente, no se podrían implementar registros de corrimiento, ya que al habilitar todos los latches con la misma señal la señal se propagaría de forma inmediata al último latch de la cadena y no se obtendría el compoortamiento deseado.

El flip-flop convierte el tiempo continuo en eventos discretos y predecibles. El latch deja el tiempo "abierto" durante medio ciclo, rompiendo las garantías sobre las que está construido todo el flujo de diseño síncrono.

En la figura 1.31 puede verse la forma de implementar un flip flop tipo D, esta estructura recibe el nombre de configuración Master - Slave, y como puede verse en la simulación, los cambios en la señal *Q* solo se presentan cuando hay una transición de "1" a "0" en la señal *C*, es decir, en un flanco descendente de la señal de reloj. En estos flancos *Q* se hará igual a *D* y mantendrá el estado en el período comprendido entre dos flancos seguidos, aún si la señal de *D* cambia.

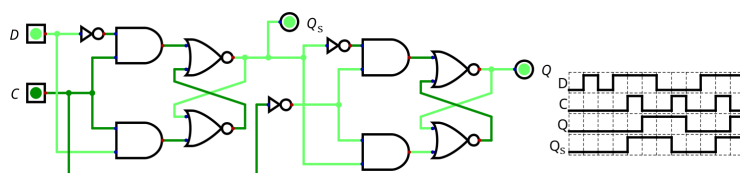


Figura 1.31 Flip Flop tipo D

Concepto de Meta-estabilidad

Cuando el flanco de reloj llega, el FF "toma una foto" de D y la inyecta como condición inicial al primer latch. Si D llevaba suficiente tiempo estable, la condición inicial está claramente cerca de 0V o de Vcc?, y el circuito converge rápido hacia el estado correcto. Sin embargo, cuando D cambia justo en el momento del flanco, la lógica interna recibe una tensión ambigua, cercana a Vcc/2 y el latch converge cerca del punto de equilibrio inestable. La salida no es ni 0 ni 1, es un nivel analógico intermedio que decae exponencialmente hacia uno de los dos estados. Este proceso puede durar mucho más de un ciclo de reloj, y el estado final (0 ó 1) es impredecible porque depende del ruido térmico de los transistores. En la figura 1.32 se ilustra esta situación.

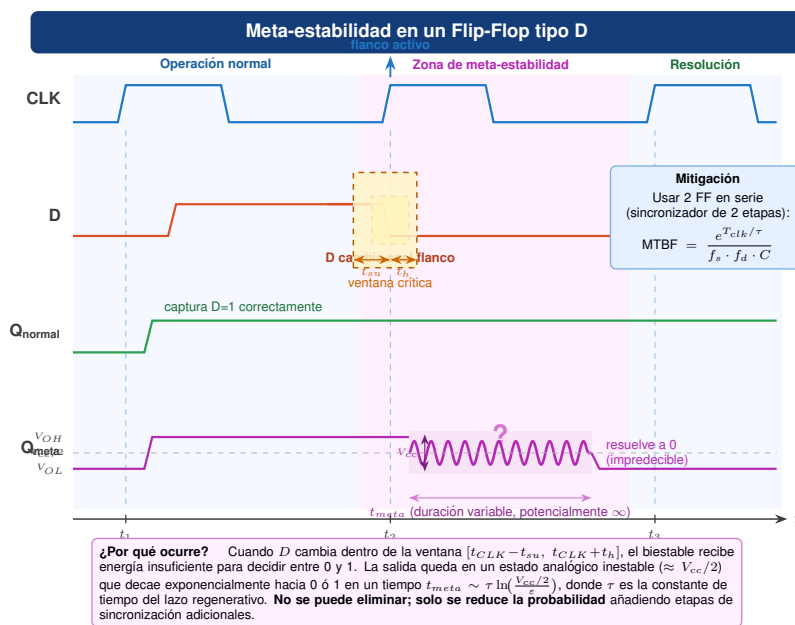


Figura 1.32 Concepto de Metaestabilidad

Para eliminar esta situación se debe garantizar que la señal D del flip-flop sea constante antes (t_s) y después (t_h) del flanco de la señal de reloj, ver figura 1.33

Circuitos Secuenciales básicos a nivel de compuertas

En la figura 1.34 se muestra el diagrama de compuertas y la simulación de un registro de 5 bits, como puede verse la señal de reloj *clk* es común a todos los Flip Flops, únicamente cuando se presenta un cambio de "0" a "1" las salidas toman el valor de las entradas. En esta simulación se observa que los cambios no ocurren de forma inmediata, esto, debido a que toda compuerta lógica tiene un tiempo de respuesta dado por las características de los transistores internos.

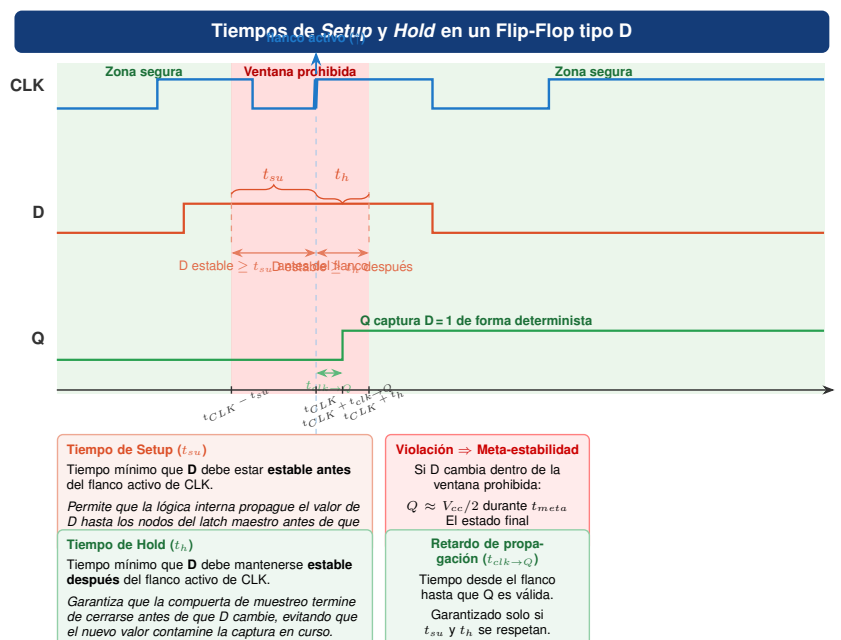


Figura 1.33 Tiempos de setup y hold

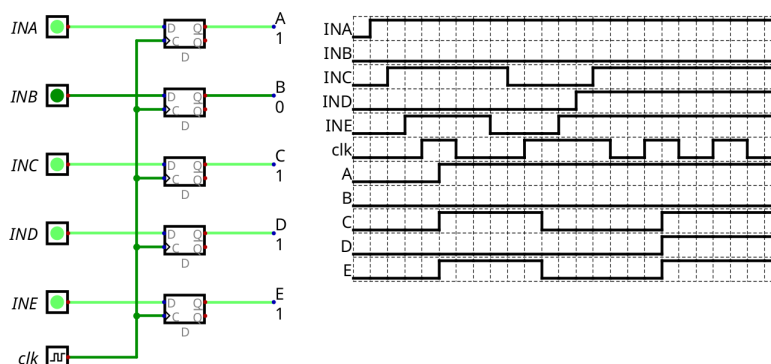


Figura 1.34 Registro de 5 bits

Registro de corrimiento a la izquierda

En la figura 1.35 se muestra la conexión de un registro de corrimiento a la izquierda, aquí, la entrada DIN está conectada a la entrada del Flip-Flop de la derecha, y la salida de este Flip-Flop se conecta a la entrada del Flip-Flop de la izquierda y la salida de este al siguiente Flip-Flop a la izquierda. En las formas de onda vemos como el pulso de la entrada pasa primero por: ESL - DSL - CSL - BSL - ASL⁸.

Registro de corrimiento a la derecha

En la figura 1.36 se muestra la conexión de un registro de corrimiento a la derecha, aquí, la entrada DIN está conectada a la entrada del Flip-Flop de la izquierda, y la salida de este Flip-Flop se conecta a la entrada del Flip-Flop de la derecha y la salida de este al siguiente Flip-Flop a la derecha. En las formas de onda vemos como el pulso de la entrada pasa por: ASR - BSR - CSR - DSR - ESR⁹.

⁸ https://github.com/cicamargoba/digital.UN/basic_circuits/Digital/lr.dig

⁹ https://github.com/cicamargoba/digital.UN/basic_circuits/Digital/rsr.dig

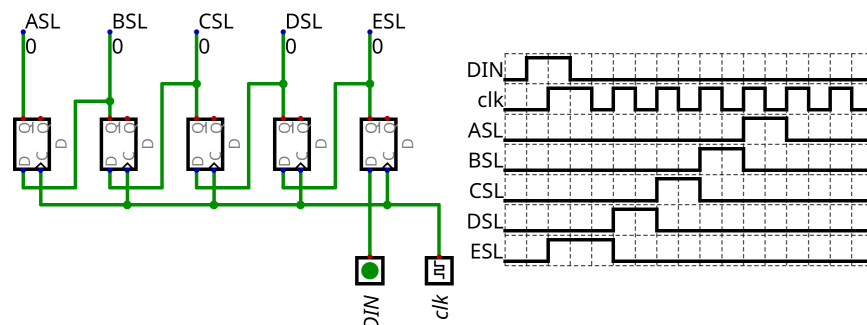


Figura 1.35 Registro de corrimiento a la izquierda

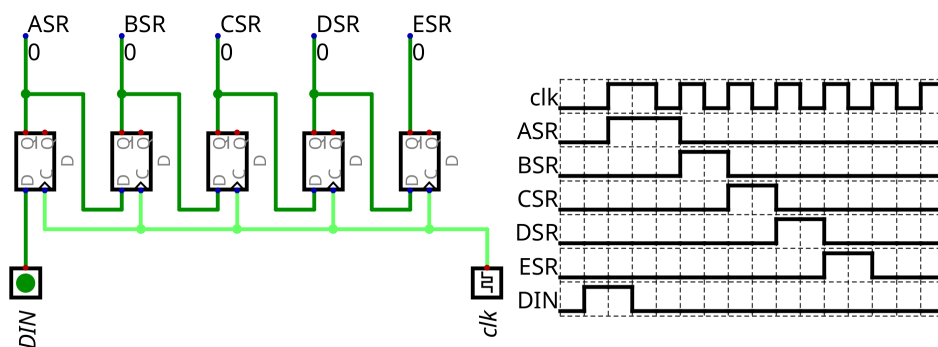


Figura 1.36 Registro de corrimiento a la derecha

Registro de corrimiento a la derecha con precarga ¹⁰.

En la figura 1.37 se muestra la conexión de un registro de corrimiento a la derecha con pre-carga, aquí, la señal **LOAD** controla a un bloque combinatorio que recibe el nombre de multiplexor. Este multiplexor posee dos entradas, una salida y una señal de selección, este módulo "conecta" una de las entradas a la salida dependiendo de la señal de selección.

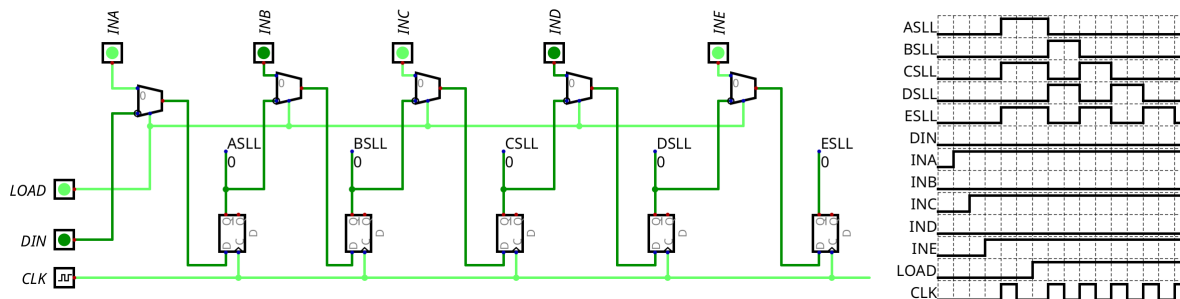


Figura 1.37 Registro de corrimiento a la derecha con pre-carga

En la figura 1.38 se muestran los circuitos resultantes cuando la señal **LOAD** = 0 (parte superior) y cuando **LOAD** = 1 (parte inferior); el circuito superior es un registro que almacena los datos de entrada, mientras que el circuito inferior es un registro de corrimiento a la izquierda; de esta forma debemos hacer **LOAD** = 0 para cargar el dato a correr y después hacer **LOAD** = 1 para permitir en corrimiento, como puede verse en la forma de ondas de la figura 1.37

¹⁰ https://github.com/cicamargoba/digital_UN/basic_circuits/Digital/rsr_charge.dig

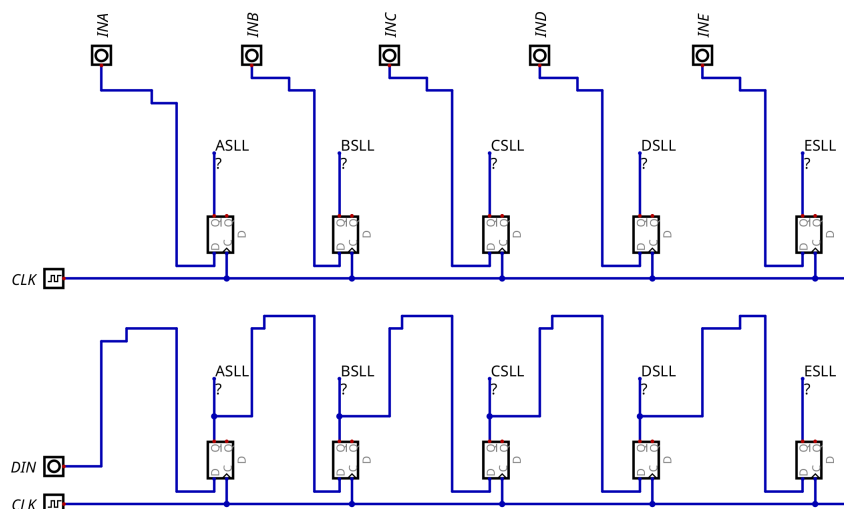


Figura 1.38 Acción del multiplexor sobre el registro de corrimiento con pre-carga

Combinación de circuitos secuenciales y combinatorios: Acumulador ¹¹.

En la figura 1.39 se muestran los circuitos involucrados en el circuito acumulador; el circuito en la parte superior izquierda es un sumador de 4 bits, acá tomamos el circuito implementado anteriormente y se conecta para sumar palabras de 4 bits, aquí vemos como la señal **cin**, del bit menos significativo se deja como una señal de entrada, la señal **cout** del bit menos significativo se conecta a la señal **cin** del siguiente bit y así sucesivamente, con lo que formamos nuestro sumador de 4 bits.

El circuito central superior implementa un restador, acá se utiliza la representación en complemento a 2 para realizar la operación $A - B$

Complemento a 2 (Ca2)

En un sistema de n bits el rango representable es:

$$-2^{n-1} \leq x \leq 2^{n-1} - 1 \quad (1.29)$$

Dado un número positivo x , su negativo en Ca2 se obtiene con:

$$\text{Ca2}(x) = \bar{x} + 1 \quad (1.30)$$

donde $\bar{x}$ es el complemento a 1 (invertir todos los bits).

Pasos:

1. Escribir $|x|$ en binario con n bits.
2. Invertir cada bit ($0 \rightarrow 1$, $1 \rightarrow 0$).
3. Sumar 1 al resultado.

Ejemplo: -5 en 4 bits

$$+5 = 0101_2$$

¹¹ https://github.com/cicamargoba/digital_UN/basic_circuits/Digital/accum.dig

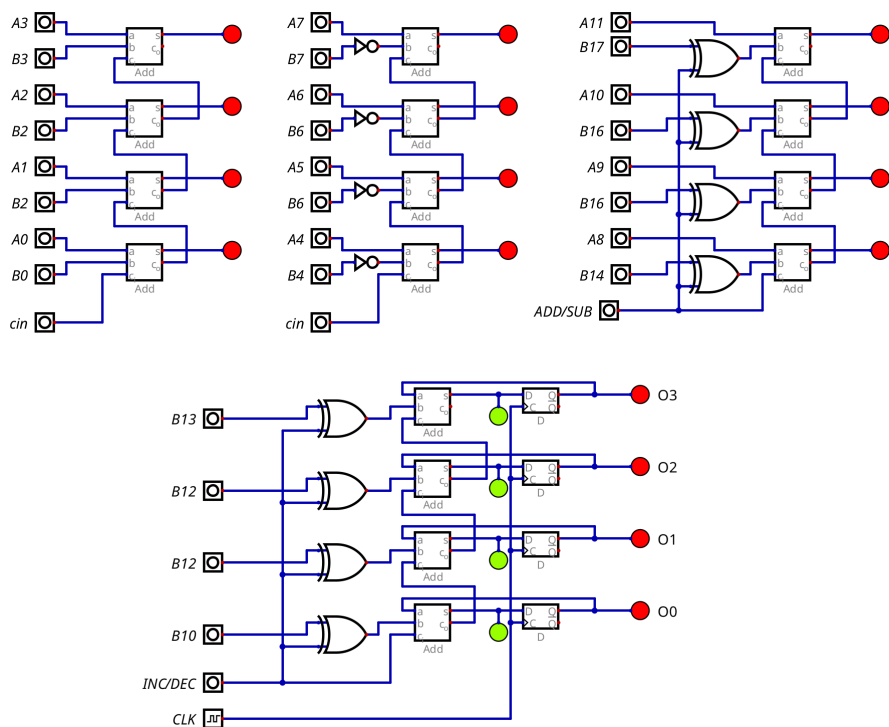


Figura 1.39 Construcción del circuito acumulador

Paso	b_3	b_2	b_1	b_0
+5	0	1	0	1
Ca1 (inv.)	1	0	1	0
+ 1				1
Ca2 (-5)	1	0	1	1

Verificación:

$$-1 \cdot 2^3 + 0 \cdot 2^2 + 1 \cdot 2^1 + 1 \cdot 2^0 = -8 + 3 = -5 \checkmark$$

Suma y resta

Restar $A - B$ equivale a sumar $A + \text{Ca2}(B)$, descartando el acarreo final. Ejemplo: $3 - 5$ en 4 bits:

$$0011_2 + 1011_2 = (1)1110_2 = -2 \checkmark$$

En la figura 1.40 se puede observar la representación gráfica de las operaciones de suma y resta en complemento a 2. Como vemos en el circuito central superior de la figura 1.39, cada bit del operando pasa por una compuerta *NOT*, con lo que obtenemos el complemento de B, al hacer $\text{cin} = 1$, estamos obteniendo el valor en complemento a dos de B, con lo que el circuito nos realiza la operación $A - B$.

El circuito de la parte superior derecha de la figura 1.39 implementa un sumador/restador, aquí, cada bit del operando B ingresa a una entrada de una compuerta XOR y el otro pin de la XOR a la señal ADD/SUB, la ecuación a la entrada b del sumador sería: $b_i = (\text{ADD/SUB}) \oplus (B_i)$; cuando $\text{ADD/SUB} = 0$ $b_i = B_i$ y cuando $\text{ADD/SUB} = 1$ $b_i = \overline{B_i}$ ¹², con lo que a la entrada del sumador tenemos $\overline{B} + 1$.

¹² $A \oplus B = A\overline{B} + \overline{A}B$

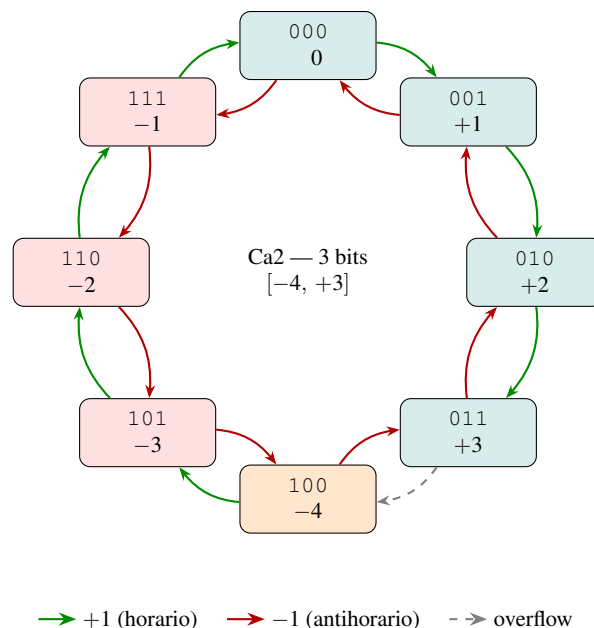


Figura 1.40 Representación circular del complemento a 2 en 3 bits. Las flechas verdes indican +1 (horario); las rojas indican -1 (antihorario). Las flechas exteriores muestran el sentido global de cada operación. La flecha gris discontinua marca la frontera de *overflow* entre +3 y -4.

En el circuito inferior de la figura 1.39 se agregan sendos Flip-Flops a las salidas del sumador, y las salidas de los Flip-Flops se conectan a la entrada del sumador con lo que la ecuación para la salida del circuito es: $O^* = O + / - B$. Existe abundante literatura sobre el uso de lenguajes de descripción de hardware para la implementación de sistemas digitales; por este motivo, en este libro no se realizará un estudio profundo de dichos lenguajes, sin embargo, en el repositorio ¹³ se proporciona una gran variedad de ejemplos con simulaciones para que puedan ser utilizadas como base en los cursos desarrollados alrededor de este texto.

Implementación de circuitos secuenciales básicos en Verilog

En la figura 1.41 podemos observar la implementación de tres circuitos secuenciales (parte superior) con sus respectivos testbench (parte inferior); Todo circuito secuencial debe contener un bloque *always* con *posedge* o *negedge* en su lista de sensibilidad, en la figura 1.41 los tres circuitos se activan con flanco de bajada. Todas las asignaciones que se realicen dentro de este bloque se realizarán a través de Flip-Flops. Las asignaciones en circuitos secuenciales se hacen con *!=* mientras que las combinatorias con *=* (ver table 1.4).

¹³ https://github.com/cicamargoba/digital_UN

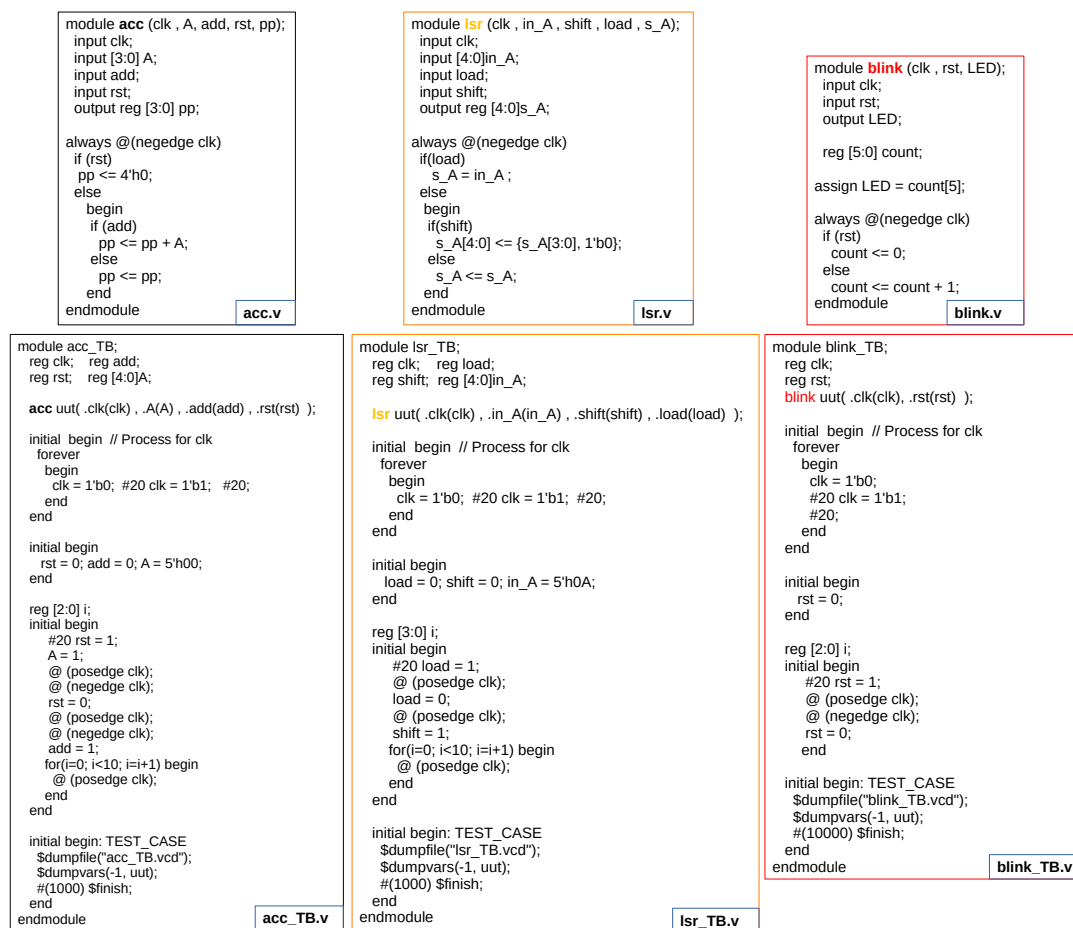


Figura 1.41 Implementación de circuitos básicos secuenciales en Verilog.

Cuadro 1.4 = vs <= en Verilog

Aspecto	= Bloqueante	<= No bloqueante
Simulación	Inmediata, en orden	Pospuesta a NBA
Síntesis combinacional	Correcto	Inferir FF no deseados
Síntesis secuencial	Divergencia sim/hw	Correcto
Intercambio de variables	Incorrecto	Correcto
Variable intermedia temp	Correcto	Problemático

1.4.7 Descripción del Verilog del multiplicador

Una vez estudiado la forma de implementar los bloques combinatorios y secuenciales, podemos regresar al ejemplo del multiplicador binario. En la figura 1.42 se muestra el diagrama de bloques de la máquina de estados algorítmica que implementa el multiplicador.

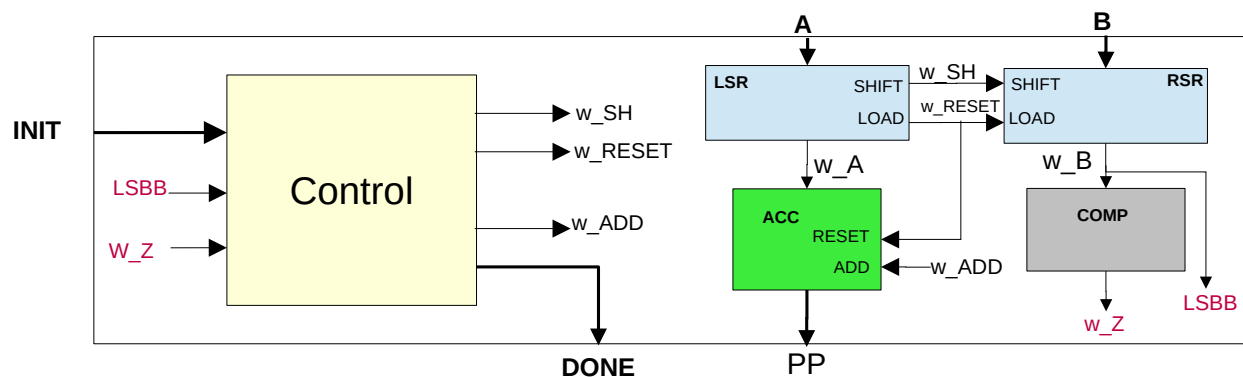


Figura 1.42 Camino de datos y control de la nASM multiplicador.

Es muy valioso anotar la importancia de la portabilidad del código, como es bien sabido, existen varias empresas que suministran entornos de desarrollo que permiten la entrada de diseño utilizando diferentes medios; las herramientas gráficas utilizados por ellos no son compatibles entre sí, lo que hace imposible el paso de un diseño implementado en una herramienta gráfica a otra de otro fabricante; sin embargo, todas las herramientas aceptan texto con el estándar del lenguaje; por esto, se recomienda utilizar únicamente entrada de texto en las descripciones.

En la figura 1.43 se muestra el código en verilog para el multiplicador, así como la forma de instanciar diferentes módulos y el código para probar el diseño (*testbench*). Esta descripción se encuentra en el repositorio del libro¹⁴.

¹⁴ https://github.com/cicamargoba/digital_UN/tree/main/basic_circuits/test_peripherals/rtl/cores/mult_ASM

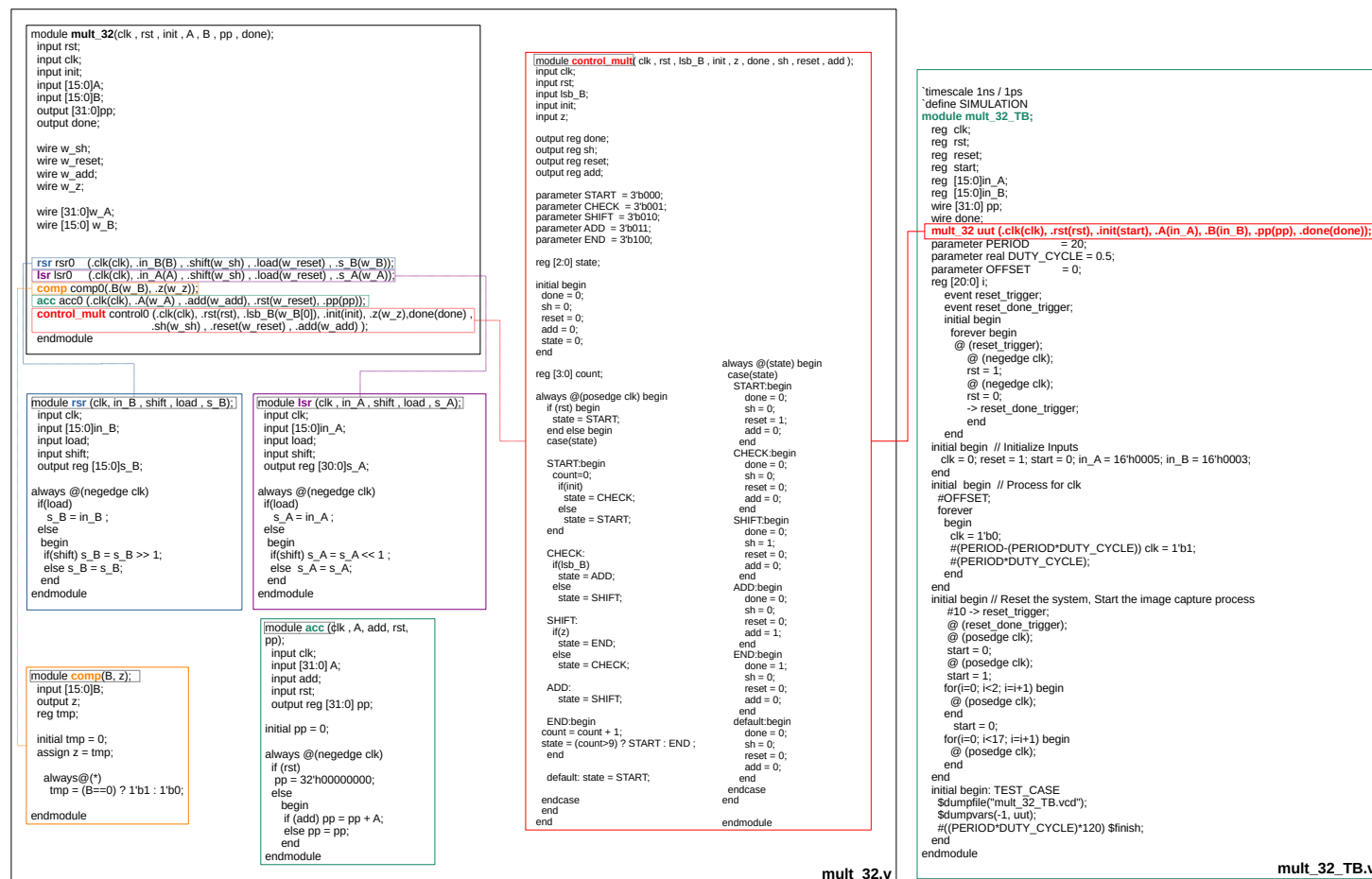


Figura 1.43 Código verilog de la ASM multiplicador.

En la figura 1.44 se muestra el diagrama de bloques sintetizado.

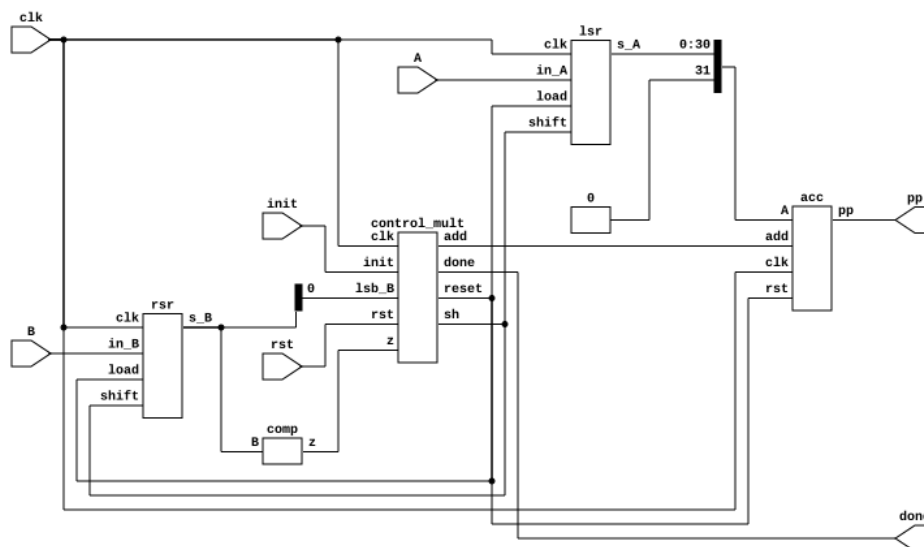


Figura 1.44 Diagrama RTL de la ASM multiplicador.

Simulación

Como se mencionó anteriormente, es posible realizar las simulaciones utilizando las herramientas gráficas de cada uno de los entornos de desarrollo que proporcionan los fabricantes de dispositivos lógicos programables, sin embargo, su uso dificulta la portabilidad del diseño. Por este motivo, se recomienda el uso de *testbench* escritos con el lenguaje estándar. Como parte del proceso de diseño, cada módulo debe ser simulado antes de ser integrado en la descripción de más alto nivel.

Los lenguajes de descripción de hardware permiten generar vectores de pruebas para ser aplicados a los módulos bajo prueba (**Unit Under Test**), para generar estos vectores se pueden utilizar instrucciones que no son sintetizables como el *for*, operaciones matemáticas, funciones trigonométricas, etc.

En la figura 1.45 se muestra el flujo de pruebas para la ASM multiplicador. En el archivo **mult_32_TB.v** se declaran las señales que generan los vectores de prueba (entradas de módulo: *clk*, *rst*, *start*, [15:0]*in_A*, [15:0]*in_B*), así como las que registrarán las salidas del módulo ante dichos vectores (salidas del módulo: [31:0] *pp*, *done*). Estas señales deben conectarse al módulo bajo prueba **mult_32** al que se le da un nombre arbitrario **uut**.

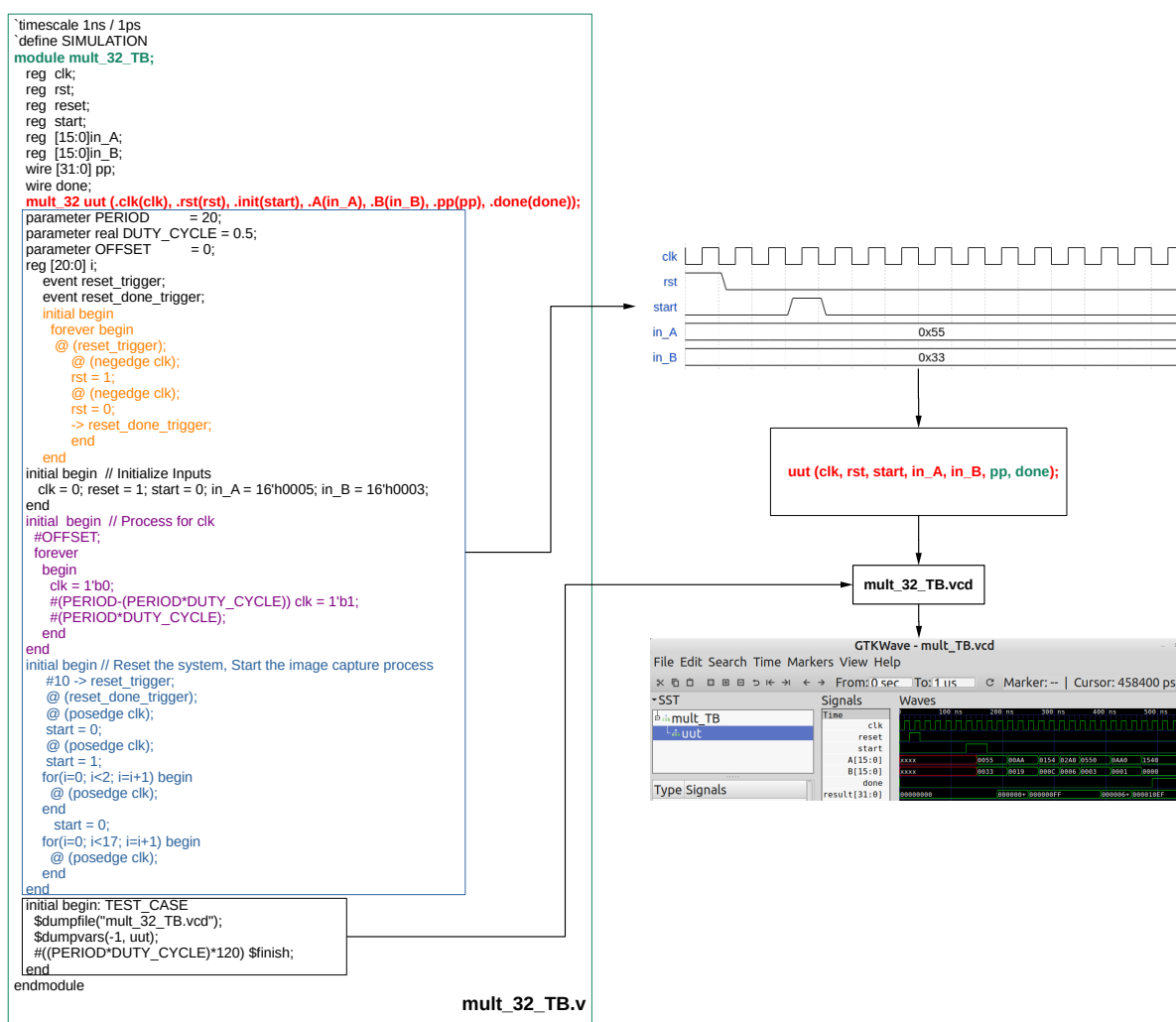


Figura 1.45 Simulación ASM multiplicador.

Las líneas resaltadas en color naranja, morado y azul general las formas de onda para las señales, *rst*, *clk* y *start* respectivamente.

El simulador aplica estas formas de onda a la unidad bajo prueba y almacena los resultados en el archivo *mult_32_TB.vcd*, este archivo puede ser visualizado con la herramienta *gtkwave*.

Niveles de simulación

Es bueno tener claro los diferentes niveles de simulación que se pueden realizar a un sistema digital; la simulación más rápida es la que tiene en cuenta únicamente el lenguaje de descripción de hardware utilizado, es decir, simula el lenguaje sin realizar ningún proceso de síntesis (convertir HDL a compuertas lógicas); sin embargo, no es posible garantizar que los resultados del circuito sintetizado (en el peor de los casos no es posible sintetizar algunas descripciones) sean los mismos que la simulación del lenguaje; por esto, existe la simulación post-síntesis, en la que se simula el RTL (lógica de transferencia de registros) o las compuertas lógicas básicas obtenidas del proceso de síntesis, esta simulación garantiza que el circuito obtenido del proceso de síntesis se comporta como lo deseamos; el tercer nivel de simulación se obtiene cuando se adiciona un modelo de tiempo al diagrama de compuertas del nivel anterior, en este nivel, se tienen en cuenta las capacitancias de carga y la capacitancia de los caminos de interconexión para obtener el retardo de cada elemento del circuito, esta simulación es la más precisa y permite conocer la velocidad máxima a la que puede operar el sistema, esta simulación en algunos entornos de desarrollo recibe el nombre de *simulación post place & route*.

En la figura 1.46 se muestra el resultado de una simulación funcional utilizando la herramienta *icarus verilog*, en ella se muestran todas las señales involucradas en la máquina de estados algorítmica del multiplicador. En la gráfica se muestra la simulación de multiplicar 5 (0101) * 9 (1001), la máquina de estados inicia en el estado (*state[2:0]*) *START* y continúa en este estado siempre que la señal *init* sea igual a 0, cuando la señal *init* = 1 se pasa al estado *check*, acá se revisa el LSB de B (1001), en la primera iteración el LSB = 1, por lo que se debe acumular el valor de A en el PP (0101) pasando al estado *ADD*, se realiza un corrimiento activando la señal *sh*, y se pasa al estado *check*, ahora el LSB de B (0100) es 0 por lo que no acumulamos y se realiza de nuevo un corrimiento y una revisión del LSB de B (0010), de nuevo se tiene un valor de 0 y no se realiza suma en el acumulador, al realizar el siguiente corrimiento el LSB de B (0001) es 1 lo que genera una acumulación de A (101000) en el PP; al realizar el siguiente corrimiento de B (0000) *z* = 1 indicando el fin del algoritmo, con lo que se pasa al estado *END* en el que la señal *DONE* = 1, el valor del PP es el resultado de la operación en este caso 0x2D (45).

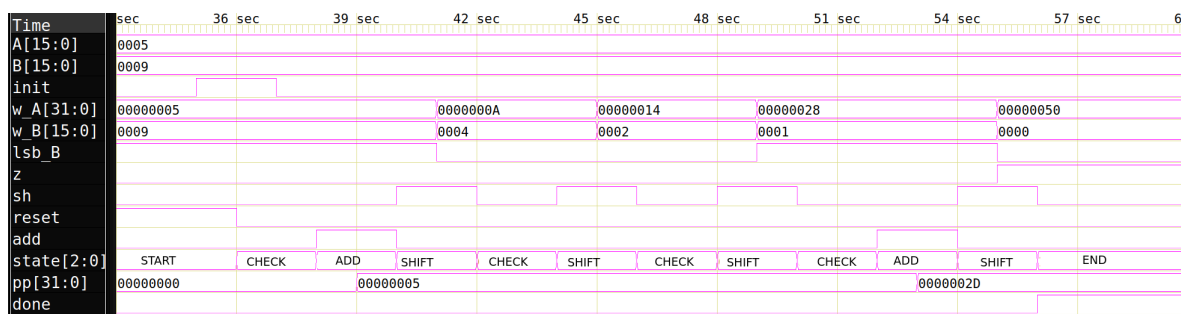


Figura 1.46 Simulación de la máquina de estados algorítmica del multiplicador binario.

En la figura 1.47 observamos los pasos que se deben seguir para realizar una simulación *post-synthesis* y *post-place and route* (proceso mediante el cual una herramienta implementa las funciones lógicas en el dispositivo lógico programable). Para la simulación post-síntesis se generan dos archivos con las interconexiones del circuito sintetizado en formatos JSON (.json) y Verilog (.v), estos archivos contienen la conexión de las compuertas lógicas (de la tecnología seleccionada) con las que la herramienta de síntesis implementa la funcionalidad deseada. La herramienta de simulación (*iverilog*) recibe como entrada el archivo en formato verilog junto con la descripción en verilog de las compuertas para la tecnología utilizada (*ICE40_SIM.CELLS*) y el archivo de pruebas (que es el mismo utilizado para la simulación funcional).

La simulación *post-place&route* se hace a partir de un archivo en verilog generado a partir de la salida de la herramienta que realiza el proceso de *place&route*, este archivo contiene la interconexión de los bloques internos del dispositivo, este archivo junto con el archivo que describe la tecnología es la entrada a la herramienta de simulación *iverilog*.

```

TARGET=mult_32
TOP=mult_32
ICE40_SIM_CELLS=$(shell yosys-config --datdir/ice40/cells_sim.v)
OBJJS = mult_32.v acc.v comp.v control_mult.v lsr.v rsr.v
PER_OBJJS = perip_mult.v $(OBJJS)

## synthesis
$(TOP).json:
    yosys -v3 -l synth.log -p 'synth_ice40 -top $(TOP) -json $(TOP).json;
                                write_verilog -attr2comment $(TOP)_synth.v' $(OBJJS)

## post-synth simulation
sim_post_syn: $(TOP).json $(OBJJS)
    [iverilog -o $@_PS.vvp -s $(TOP)_TB $(TARGET)_TB.v $(TOP)_synth.v $(ICE40_SIM_CELLS)
    vvp $@_PS.vvp
    gtkwave $(TOP)_TB.vcd &

## post-route simulation
$(TOP).asc: $(TOP).json
    nextpnr-ice40 --hx8k --package ct256 --json $(TOP).json --pcf $(TOP).pcf --asc $@
    sim_post_route: $(TOP).asc $(TOP).pcf
    icebox_vlog -L -n $(TOP) -sp $(TOP).pcf $< > $(TOP)_route.v
    [iverilog -o $@_vvp -s $(TOP)_TB $(TARGET)_TB.v $(TOP)_route.v $(ICE40_SIM_CELLS)
    vvp $@_route.vvp
    gtkwave $(TOP)_TB.vcd &

```

Makefile

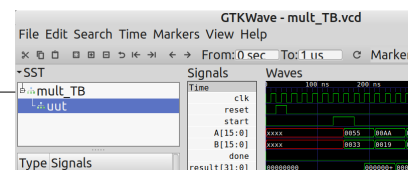
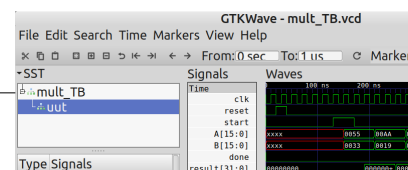
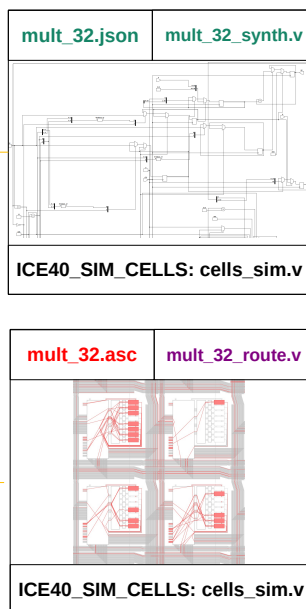


Figura 1.47 Simulación post-síntesis y post-place&route.

1.5 Implementación de un divisor de n bits sin signo

El proceso de división de números binarios es similar al de números decimales: Inicialmente se separan dígitos del Dividendo de izquierda a derecha hasta que la cifra así formada sea mayor o igual que el divisor. En la figura 1.48 separamos el primer dígito de la derecha (0) y le restamos el divisor (la operación de resta se realizó en complemento a dos), el resultado de esta operación es un número negativo (los números negativos en representación complemento a dos comienzan por 1). Esto indica que el número es menor que el divisor, por lo tanto, colocamos un cero en el resultado parcial de la división (este dígito será el más significativo) y separamos los dos primeros dígitos (00) y repetimos el proceso.

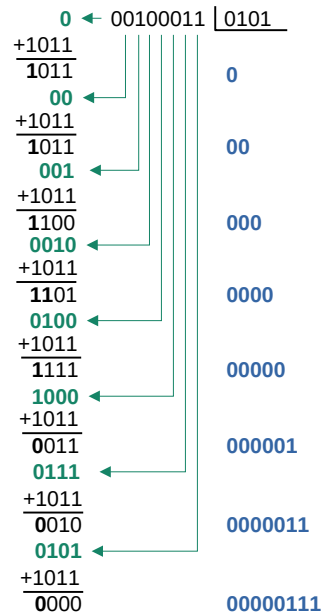


Figura 1.48 División de números binarios.

Sólo hasta el sexto resultado parcial obtenemos un cero en la primera cifra de la resta (recuerde que en complemento a dos los números tienen una longitud fija en nuestro caso 4 bits, si una operación provoca un bit adicional este se descarta, los bits descartados se encerraron en líneas punteadas en la Figura 1.48), lo que indica que el número es mayor o igual que el divisor, en este caso, se coloca un '1' en el resultado parcial y se conserva el valor de la operación de resta, el cual se convierte en el nuevo residuo parcial, este proceso se repite hasta haber "bajado" todos los dígitos del dividendo.

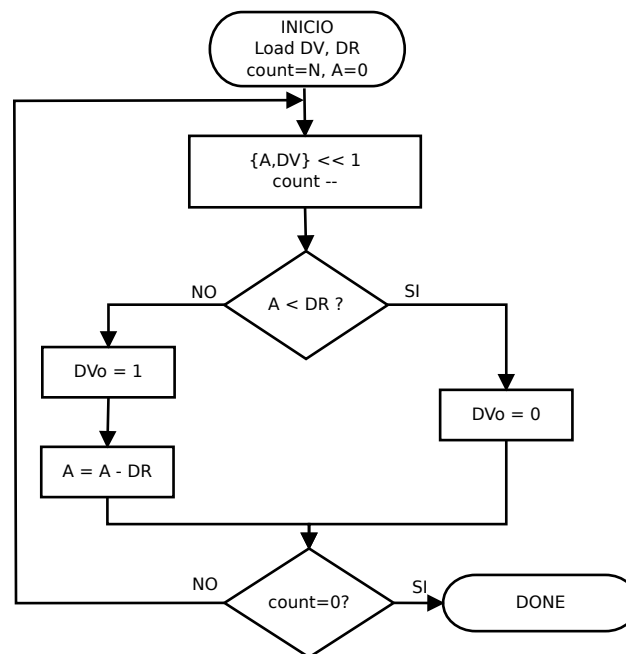


Figura 1.49 Algoritmo para la división de números binarios.

1.5.1 Identificación de componentes del camino de datos e interconexión

En la figura 1.50 podemos observar cómo se obtienen los componentes del camino de datos a partir del diagrama de flujo del divisor; se necesita un registro de corrimiento a la izquierda donde se almacena el Dividendo (DV) y las cifras que se van separando (A), un contador que cuente el número de bits que se han “bajado”, un restador (sumador en complemento a 2) para determinar si el número separado del dividendo “cabe” en el divisor (observando el bit más significativo MSB), y un comparador que indique que el valor del contador llegó a cero.

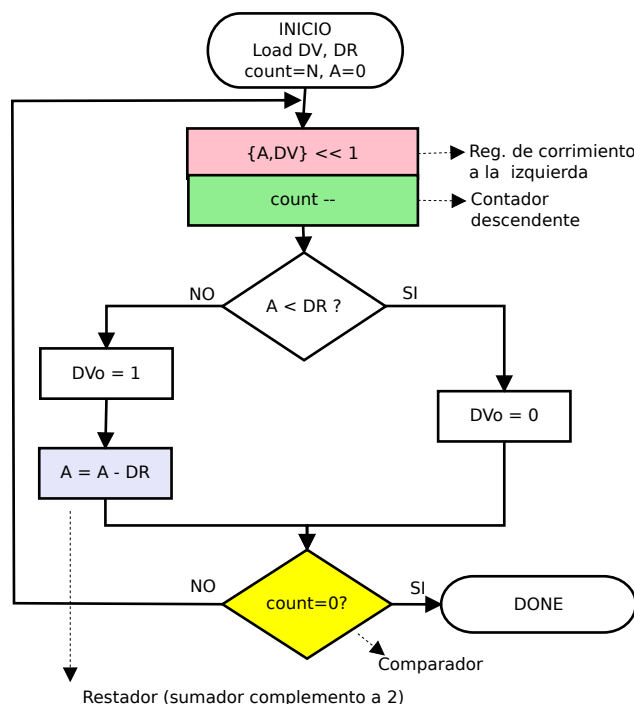


Figura 1.50 Identificación de componentes del camino de datos para la división de números binarios.

En la figura 1.51 se muestra la interconexión de los elementos del camino de datos y se muestran las señales de control. De nuevo, las señales que se activan en el mismo punto del diagrama de flujo pueden agruparse, por esto, la señal de inicialización del registro A, la carga de DV y la inicialización del contador se realizará con la señal *INIT*; el registro de desplazamiento a la izquierda va almacenando el resultado de la división a medida que se van utilizando los bits más significativos del dividendo, con esto se reduce el número de componentes, la señal *DV0* ayuda a formar el resultado; la señal *SH* realiza el corrimiento a la izquierda del registro $\{A, DV\}$ con lo que en *A* queda el número que se va separando del dividendo y en *DV* el resultado de la división; la señal *LDA* carga el resultado de la resta entre *A* y el divisor únicamente cuando el resultado de la resta es positivo, esto es cuando la señal *MSB* es igual a 1; la señal *DEC* hace que el valor del contador disminuya en 1, y la salida *Z* se hace 1 cuando el valor de este contador llega a cero indicando que el algoritmo terminó.

De lo anterior tenemos que la unidad de control tiene como entradas las señales: *Reset*, *Start*, *MSB* y *Z*; y como salidas: *INIT*, *DV0*, *SH*, *DEC* y *LDA*; de nuevo los bloques de decisión del diagrama de flujo del algoritmo hacen referencia a las entradas de la unidad de control.

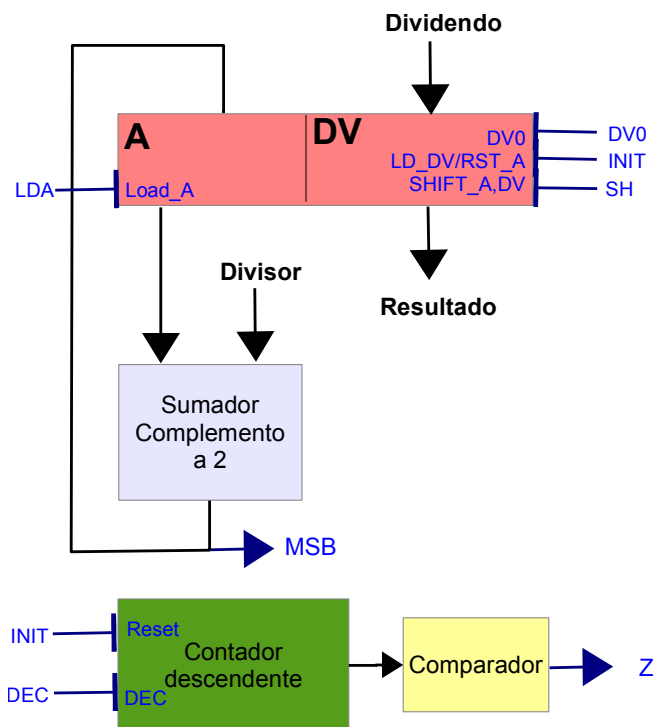


Figura 1.51 Interconexión del camino de datos para la división de números binarios.

1.5.2 Unidad de control

En la Figura 1.52 se muestra la relación existente entre el diagrama de flujo y el diagrama de estados de la unidad de control del divisor binario.

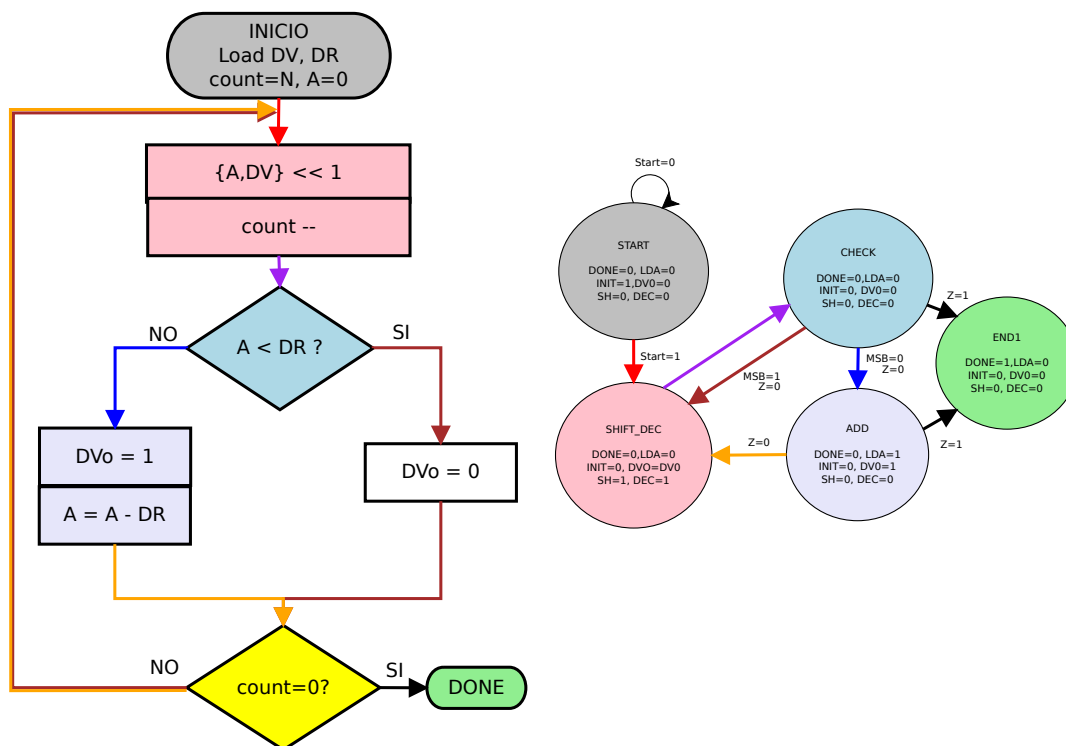


Figura 1.52 Diagrama de estados de la unidad de control para la división de números binarios.

1.5.3 Diagrama de bloques del divisor

En la figura 1.53 se muestra el diagrama de bloques del divisor; su arquitectura es muy similar a la del multiplicador ya que tiene prácticamente las mismas señales, y requieren la misma información para su operación.

Los periféricos en general poseen la misma arquitectura, las variables son almacenadas en registros, multiplexores en los buses de entrada y salida de datos controlan en donde se almacena o de donde proviene la información que llega o sale a la CPU. Esta arquitectura posee un decodificador de direcciones interno que controla los multiplexores de entrada y salida.

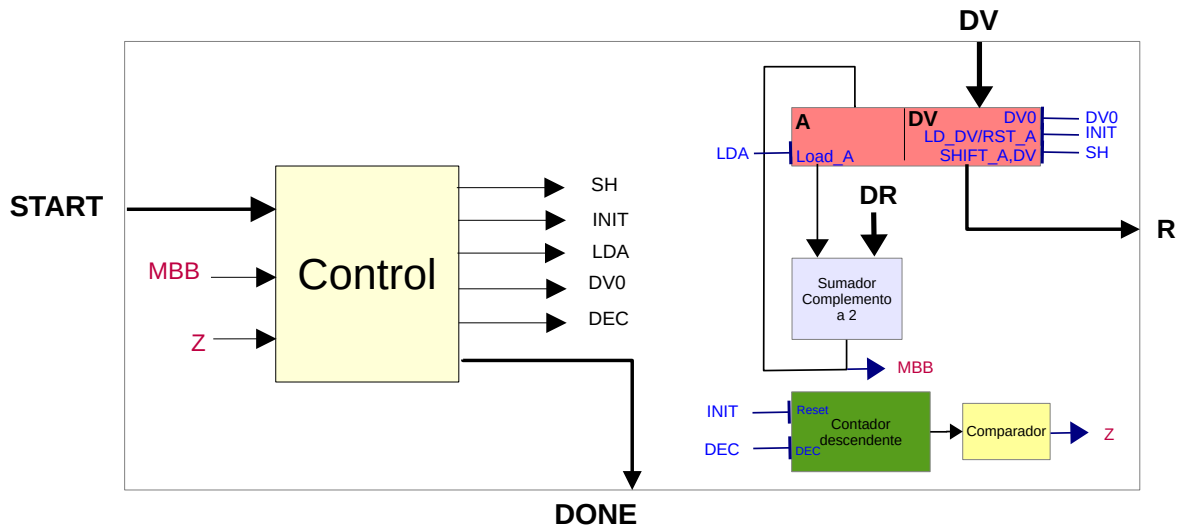


Figura 1.53 Diagrama de bloques del Divisor.

1.5.4 Simulación del divisor

En la figura 1.54 se muestra la simulación de la implementación en verilog del divisor.

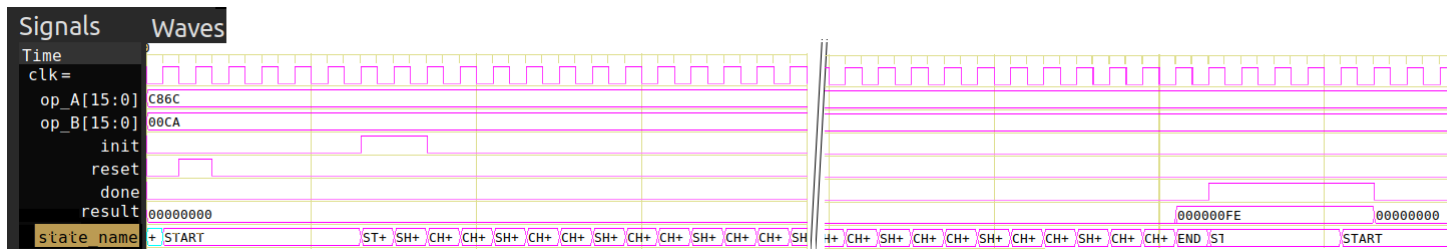


Figura 1.54 Simulación del divisor.

1.6 Implementación de un algoritmo para la raíz cuadrada

En la figura 1.55 se muestra la forma de obtener la raíz cuadrada de un número binario y en la figura 1.56 se muestra el diagrama de flujo de su algoritmo, así como el diagrama de bloques de la máquina de control y el camino de datos que lo implementa.

1.7 Conversión de Binario a Binario codificado en Decimal

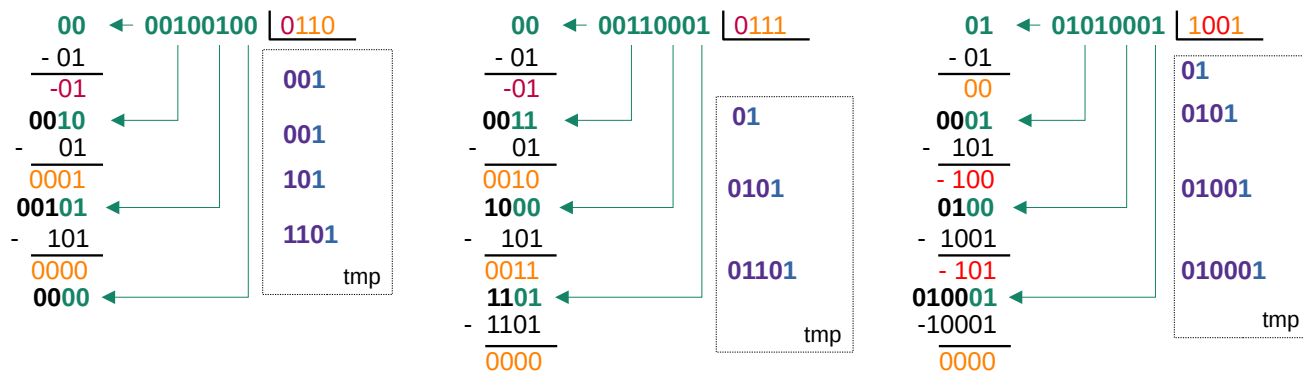


Figura 1.55 Raíz cuadrada de un número binario.

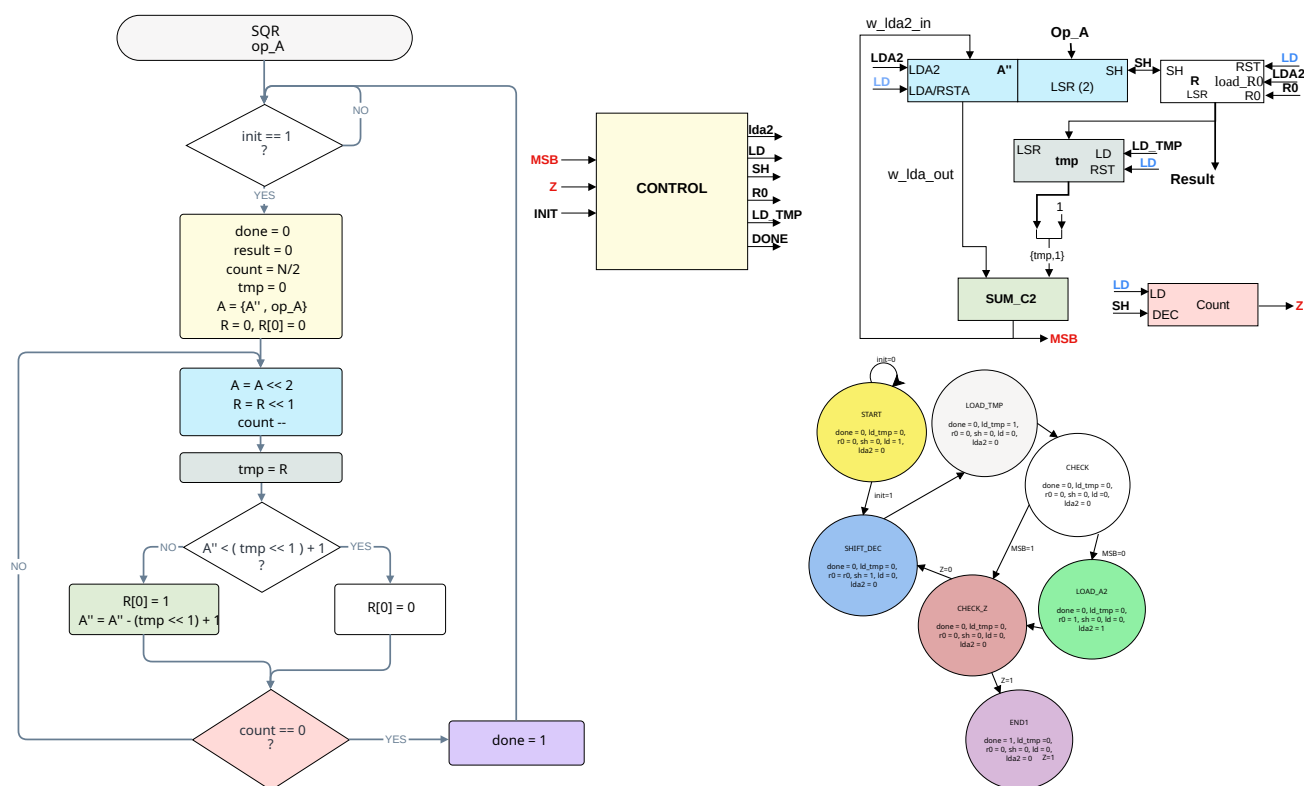


Figura 1.56 Algoritmo para obtener la raíz cuadrada de un número binario.

1.7 Conversión de Binario a Binario codificado en Decimal

En la figura 1.57 se muestra el algoritmo *double dabble* que convierte un número binario de 16 bits (solo el rango 0 - 9999) a su equivalente BCD.

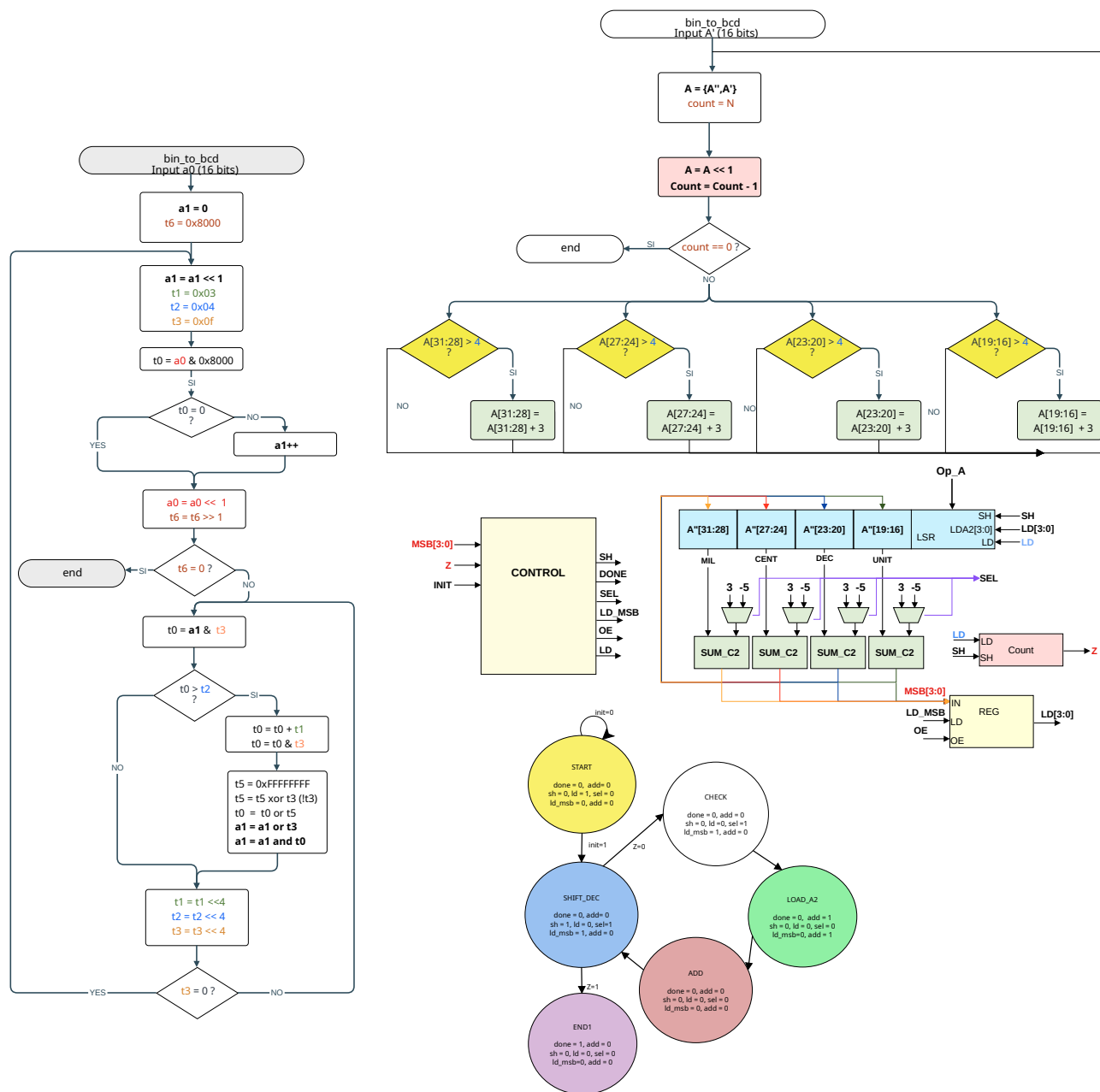


Figura 1.57 Raíz cuadrada de un número binario.

1.8 Controlador de Panel LED 8 colores

Las pantallas LED son muy utilizadas en la actualidad y están reemplazando a los televisores o pantallas similares en muchas áreas de aplicación como publicidad y entretenimiento, adicionalmente, su funcionamiento es muy interesante para adquirir habilidades en la elaboración de diagramas de flujo. En la figura 1.58 se muestra el principio de funcionamiento de estos paneles, los cuales, están formados por filas y columnas, solo una fila puede ser energizada a la vez, por lo que es necesario realizar una visualización dinámica (desplegar la información a una velocidad tal que la persistencia del ojo humano la vea continua) la información de color para las columnas deben ser fijadas utilizando un registro de corrimiento, (uno para cada color) cada pixel está compuesto por 3 leds Rojo, Verde y Azul, por lo

que contamos con señales de entrada para los registros de corrimiento (RGB). Para hacer más rápido el proceso los paneles se dividen en dos, cada mitad cuenta con señales para ingresar la información de las columnas. Si un panel tiene una dimensión de 64x64 pixles, cada mitad tendrá 32x64 pixles, las señales R1G1B1 controlarán las filas 0 a 31 y las señales R2G2B2 las filas 32 a 63.

El proceso de carga de información de colores de las columnas puede verse en la figura 1.58. Primero se selecciona la fila utilizando las señales **A-B-C-D-E** después se ingresa la información de la fila bit a bit en cada flanco de subida de la señal **CLK** en las señales RxGxBx, después de (para nuestro ejemplo) 64 pulsos de la señal **CLK** la información de la fila está lista. Después se activa la señal **LAT**, almacenando el contenido del registro de corrimiento al registro de salida, finalmente se activa la señal **OE**, permitiendo la visualización de la fila. Se selecciona la siguiente fila repitiendo el proceso hasta completar el panel.

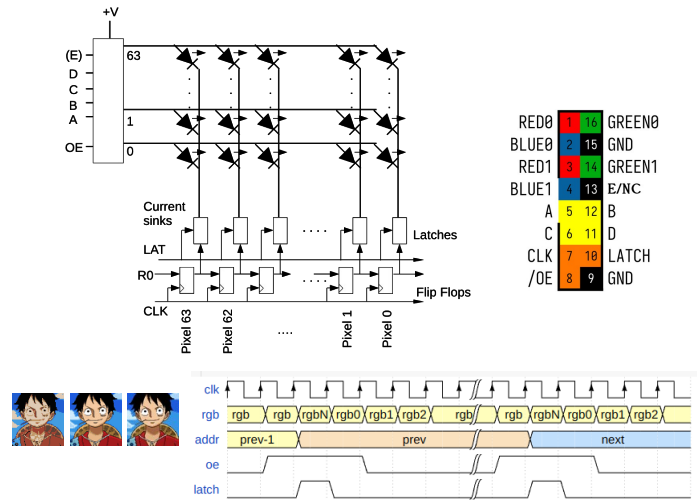


Figura 1.58 Funcionamiento de un Panel de LEDs 8 colores.

En la figura 1.59 se muestra el diagrama de flujo del controlador. Del cual se extrae el diagrama de bloques del camino de datos y el diagrama de estados de la máquina de control.

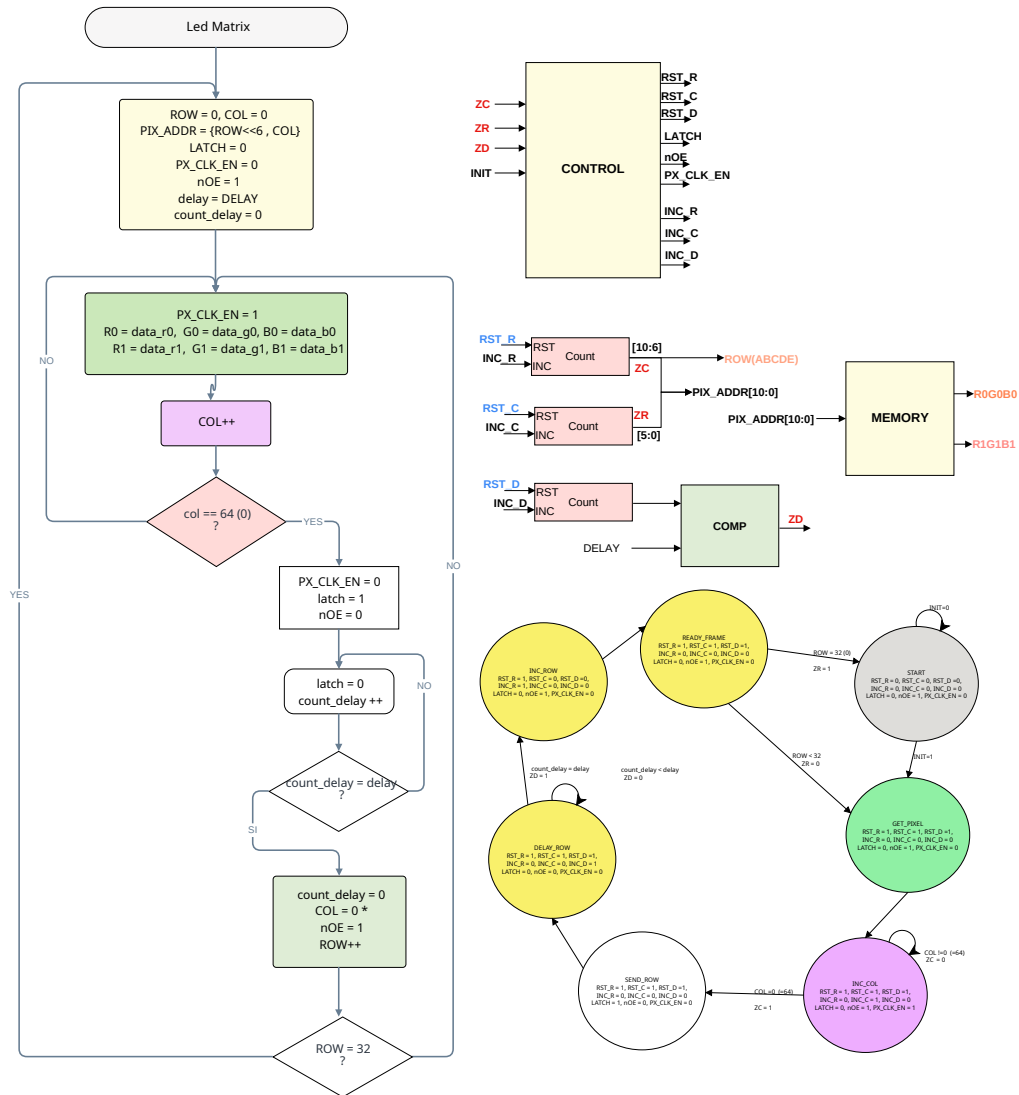


Figura 1.59 Diagrama de flujo y Camino de Datos del controlador de Panel de LEDs 8 colores.

En la figura 1.60 se muestra la simulación de este controlador.



Figura 1.60 Simulación del controlador de Panel de LEDs 8 colores.

1.8.1 Control de panel LED 12 bits por pixel

En el diseño anterior solo utilizabamos 3 bits por cada pixel los LEDs tenían el brillo máximo o se apagaban, por lo que solo podíamos obtener 8 colores diferentes en cada LED. Para aumentar el número de colores, es necesario que los LEDs brillen a diferentes intensidades, para lograr esto utilizaremos la técnica codificación binaria. Supongamos que tenemos 8 bits como se muestra en la figura 1.61, cada bit tiene un peso dado por su posición el bit menos significativo tiene el menor peso, el siguiente bit el doble, el siguiente bit cuatro veces más que el menos significativo y así sucesivamente, para representar este peso, se le asigna un tiempo de visualización equivalente al peso de cada bit, de esta forma podemos obtener un brillo equivalente al valor asignado a cada bit.



Figura 1.61 Codificación binaria.

Por lo tanto, es necesario modificar el algoritmo anterior adicionando el despliegue de la información de cada bit, en este ejemplo utilizaremos cuatro bits por pixel, con lo que debemos desplegar la información de cada fila cuatro veces (una para cada pixel) y el tiempo de visualización de cada muestra será equivalente al peso del bit.



Figura 1.62 Señales de control para la codificación binaria.

Como puede verse en la figura 1.62, la fila no cambia hasta que se despliegan los cuatro bits y el tiempo de despliegue de cada bit ($NOE = 0$) se duplica en cada ciclo para representar el peso de cada bit.

En la figura 1.63 se muestra el diagrama de flujo del algoritmo modificado para incorporar la codificación binaria, su correspondiente diagrama de bloques del camino de datos, y el diagrama de estados de la máquina de control.

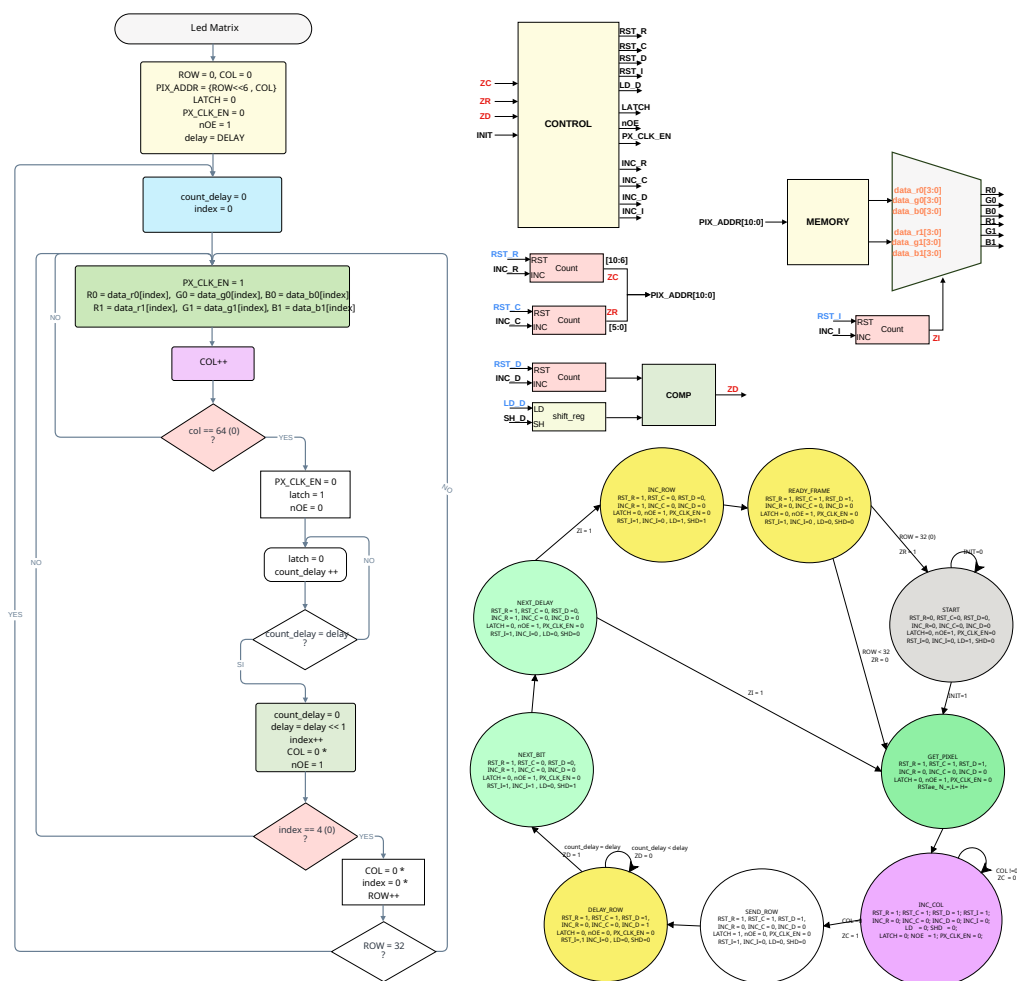


Figura 1.63 Diagrama de flujo, diagrama de estado, y diagrama de bloques del camino de datos del controlador de panel LD 12 bits por pixel.

1.8.2 Control de LED RGB WS2812

El led del tipo WS2812 es ampliamente utilizado en diferentes aplicaciones como iluminación, avisos publicitarios, juegos etc. Este dispositivo es un led RGB, donde cada color puede ser ajustado con 8 bits. Su interfaz es muy simple y está diseñado para ser utilizado en cadena, posee una señal de entrada y una señal de salida que va conectada al siguiente LED, todos los LEDs se alimentan a VCC y GND. En la figura 1.64 se muestra el diagrama de tiempos de un 1 y un 0 lógico; para configurar un LED, es necesario generar 24 bits 8 para el verde, 8 para el rojo y 8 para el azul. Si se tienen dos LEDS en cadena, los primeros 24 bits serán "tomados" por el primer LED y delará "pasar" los demás hacia ña cadena.

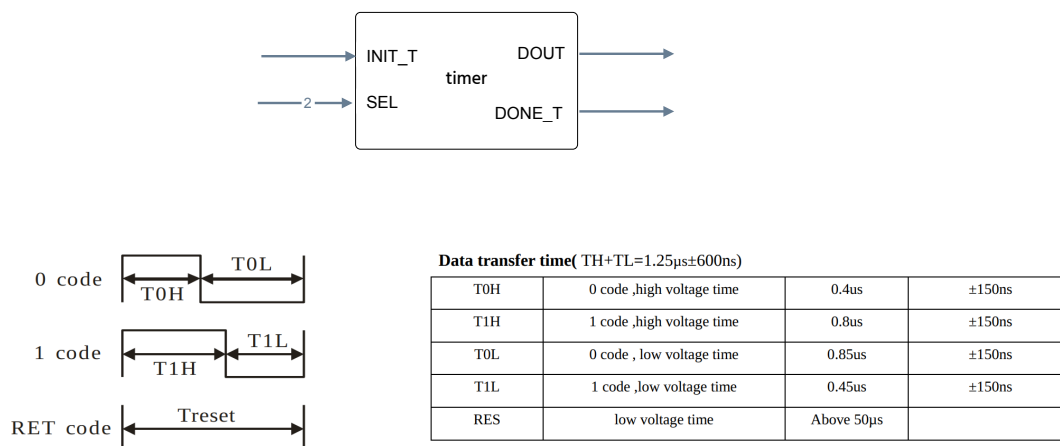


Figura 1.64 Operación del dispositivo WS2812.

Para controlar este dispositivo realizaremos un diseño incremental, primero se diseñará el circuito que genera las señales que el dispositivo reconoce como 1 y 0 lógicos, después, se diseñará el circuito que envía los 24 bits de color para 1 LED y finalmente se diseñará el circuito que envía las señales necesarias para configurar un arreglo de N LEDs.

Generador de bits de información para el WS2812

Cómo se mencionó anteriormente el ws2812 requiere unas formas de onda específicas para configurar los colores; En la figura 1.65 se muestra el diagrama de flujo, el diagrama de estados del control y el diagrama de bloques del camino de datos que implementa esta operación. Aquí, la señal SEL indica la señal que se va a generar SEL = 0 genera un 0, SEL = 1 genera un 1, SEL = 2 genera un Reset, una vez fijado SEL se debe generar un pulso en la señal INIT_T para dar inicio a la generación del bit, cuando el proceso termina, la señal DONE_T = 1¹⁵.

¹⁵ https://github.com/cicamargoba/digital_UN/tree/main/basic_circuits/test_peripherals/rtl/cores/test/ws2812

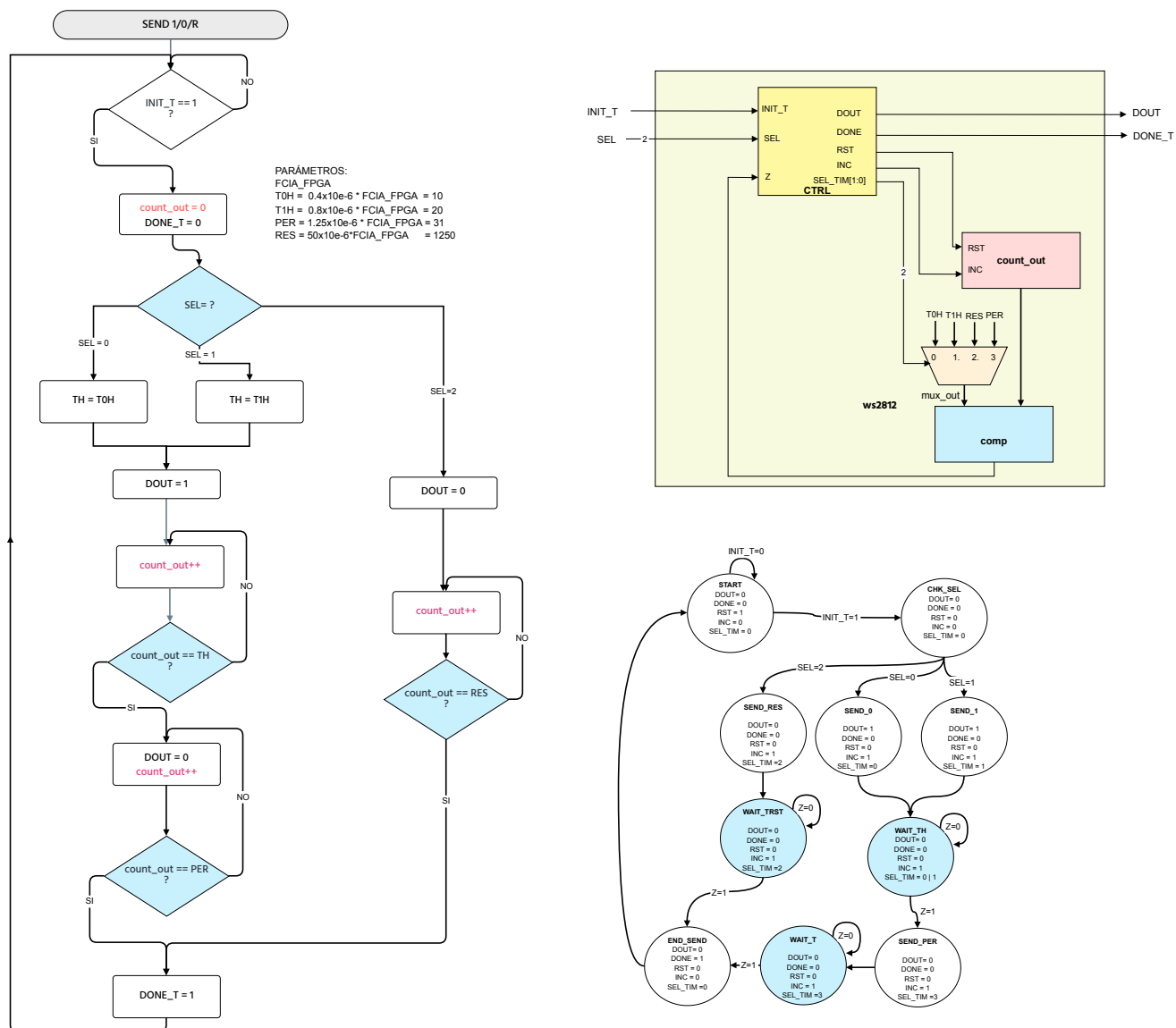


Figura 1.65 Generación de bits para el dispositivo WS2812.

Configuración de un LED WS2812

Como se dijo anteriormente, es necesario enviar 24 bits con la información de color para que el WS2812 despliegue el color deseado. En la figura 1.66 se muestra el diagrama de caja negra, el diagrama de flujo, el diagrama de bloques del camino de datos y el diagrama de estados del control. Aquí se utiliza el módulo diseñado anteriormente, el cual recibe la información a transmitir directamente de los registros y señales de entrada, la máquina de estados da la orden de inicio de la generación del bit, espera a que se finalice la transmisión y genera un corrimiento para realizar estas operaciones con el siguiente bit.

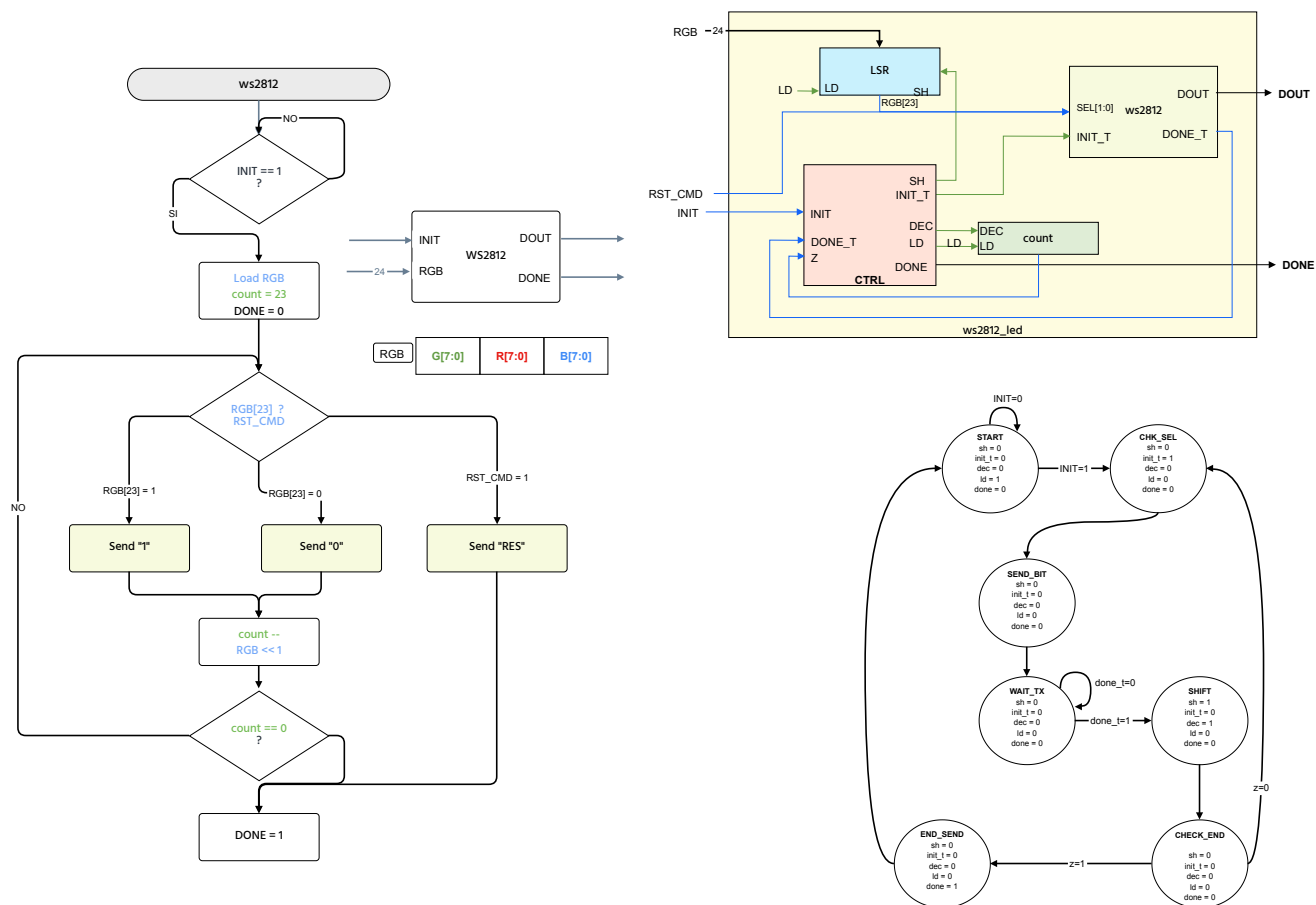


Figura 1.66 Configuración de un WS2812.

Configuración de N LEDs WS2812

Finalmente, el circuito de la figura 1.67 realiza la configuración de N LEDs ws2812, para esto utilizamos el circuito anterior junto con una memoria que almacena la información que se desea desplegar. La memoria puede verse como un arreglo direccionable donde cada elemento (en este caso) es de 24 bits, la entrada de dirección *ADDR*, indica el elemento al que se desea acceder y la señal de salida *DATA*, entrega el valor almacenado en esa posición. En verilog es posible inicializar estas memorias como puede verse en el archivo¹⁶ el código `$readmemh("./display.hex", MEM)`; inicializa el arreglo *MEM* con el contenido del archivo *display.hex*, el cual debe ser una lista de valores en hexadecimal de la siguiente forma:

```
FFFF80
FFFF80
FFFF80
FFFF80
FFFF80
....
```

¹⁶ https://github.com/cicamargoba/digital_UN/blob/main/basic_circuits/test_peripherals/rtl/cores/test/ws2812/led_mem.v

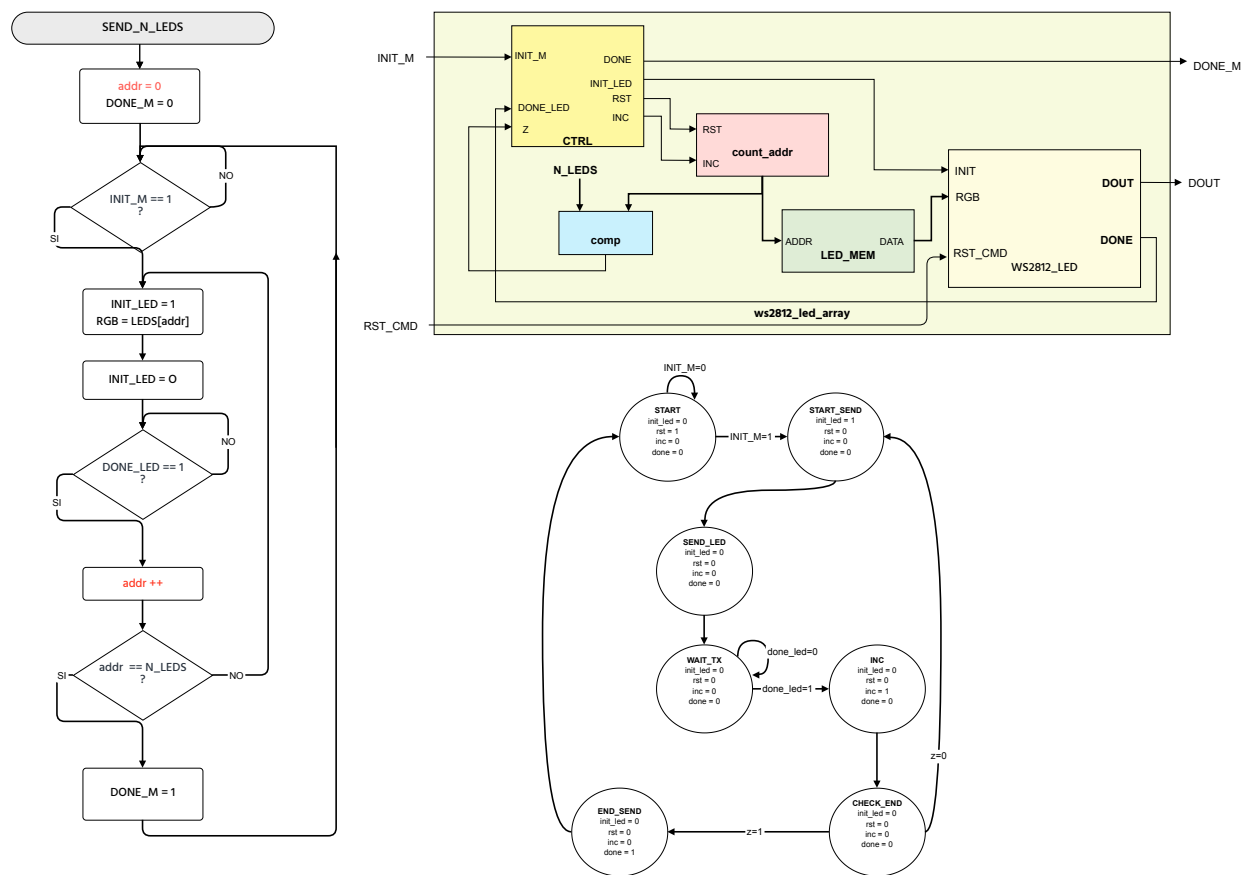


Figura 1.67 Configuración de un arreglo de N WS2812.

1.9 Flujo de diseño Hardware - Software

Como se dijo anteriormente todo algoritmo puede ser implementado en Software (código secuencial que se ejecuta en un procesador) o en Hardware (unidad de control + camino de datos), la forma más eficiente de implementación dependerá de restricciones de velocidad, consumo de potencia, tamaño y costos.

Hasta el momento hemos descrito dos periféricos que implementan las operaciones binarias multiplicación y división, ahora vamos a mostrar como controlar estos periféricos desde una unidad de procesamiento o CPU. Por ahora la CPU se utilizará como una caja negra que podemos programar, en el siguiente capítulo se estudiará internamente.

En la figura 1.68 se muestra el flujo de diseño que se utiliza con procesadores *softcore*, es decir, procesadores implementados en FPGAs. Como puede verse el procesador *softcore* que utilizamos cuenta con una memoria de programa (ram.v) mapeada en las direcciones $0x0 - 0x3FFFFFF$, cuando se activa la señal de reset, el contador de programa de la CPU se hace igual a 0 y comienza a leer, decodificar y ejecutar las instrucciones en dicha memoria.

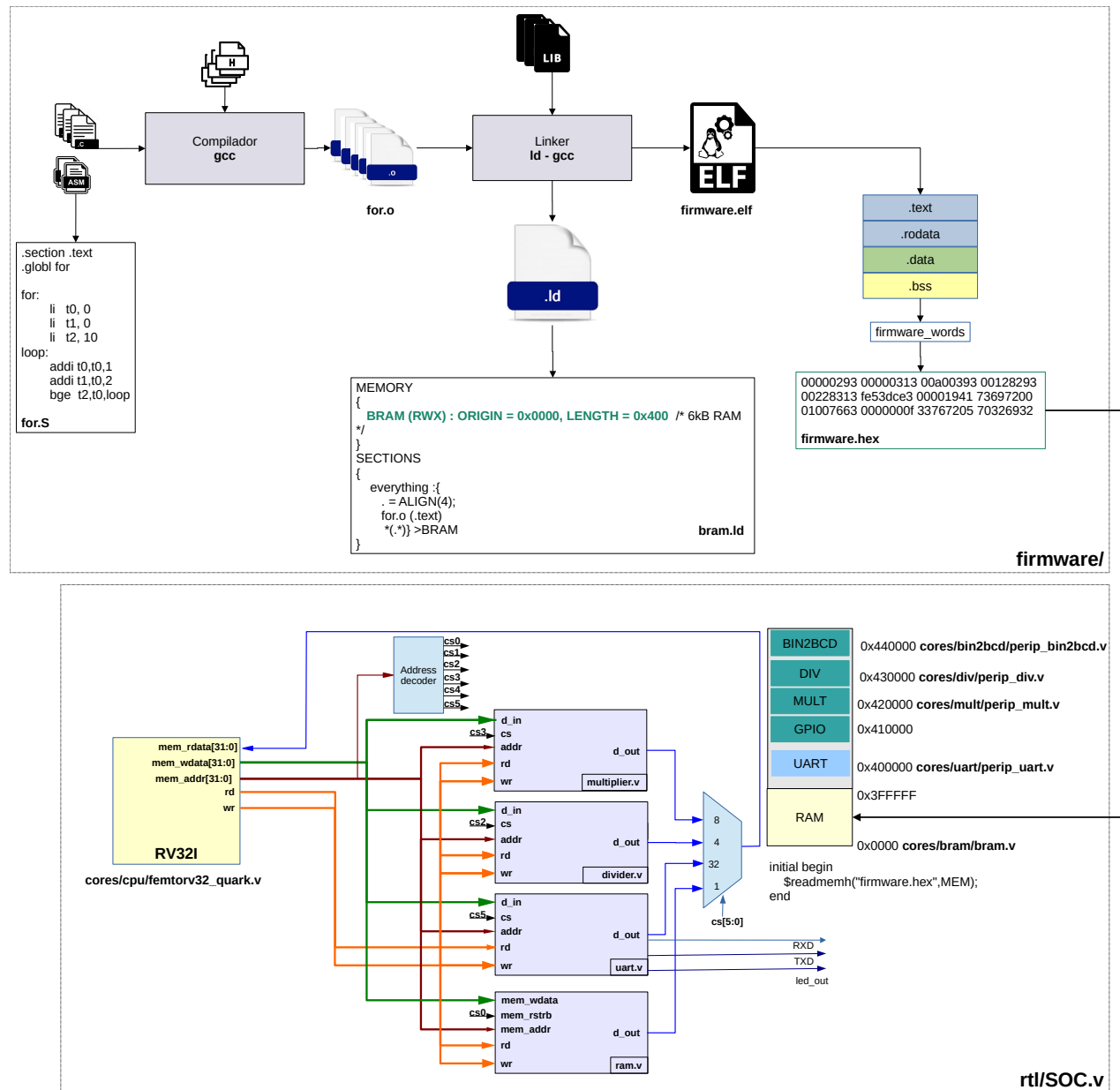


Figura 1.68 Flujo de diseño Hardware - Software.

Las herramientas de síntesis modernas permiten inicializar el contenido de las memorias con archivos de texto donde se almacena el código en formato hexadecimal tal como se muestra a continuación.

```
00000293 00000313 00a00393 00128293
00228313 fe53dce3 00001941 73697200
01007663 0000000f 33767205 70326932
```

Para generar este código es necesario ejecutar el flujo de diseño software, el cual convierte los archivos fuente en las instrucciones que se almacenaran en la memoria de programa. En la figura 1.69 se muestran los archivos necesarios para compilar una aplicación escrita en lenguaje ensamblador.

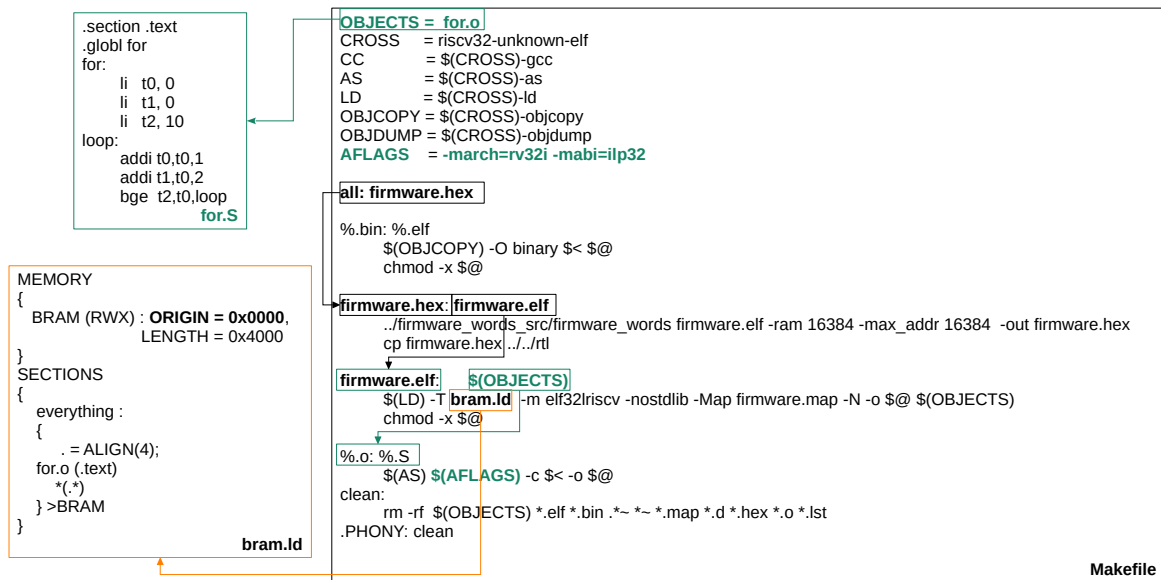


Figura 1.69 Flujo de diseño Software para la implementación del ciclo *for*.

El archivo *Makefile* ejecuta las instrucciones de forma ordenada y ayuda a realizar el proceso sin necesidad de ejecutar comandos largos. La herramienta *make* ejecuta las opciones disponibles en el archivo *Makefile*, si no se le especifica un parámetro a *make*, esta ejecutará la opción **all**, que en este caso tiene como requisito la opción **firmware.hex** quien a su vez ejecutará la opción **firmware.elf**, quien ejecutará la opción **OBJECTS**, **OBJECTS** es una variable que contiene los objetos necesarios para generar el ejecutable, en este caso el objeto *for.o*, la opción *%o: %S* le dice a *make* como generar un archivo *.o* a partir de un archivo *.S*. Por lo que el primer comando en ejecutarse será:

```
riscv32-unknown-elf-as -march=rv32i -mabi=ilp32 -c for.S -o for.o
```

Una vez cumplido el requerimiento de la opción **firmware.elf**, se ejecutan sus tareas:

```
riscv32-unknown-elf-ld -T bram.ld -m elf32lriscv -nostdlib -Map \
firmware.map -N -o firmware.elf for.o
chmod -x firmware.elf
```

El archivo **bram.ld** le indica al enlazador (*riscv32-unknown-elf-ld*) en donde se encuentra el origen de la memoria de programa (0x0 en este caso) y el objeto que debe estar en esa posición, **for.o** para este ejemplo.

Con el requisito de **firmware.hex** cumplido se ejecutan sus tareas:

```
../firmware_words_src/firmware_words firmware.elf -ram 16384 -max_addr \
16384 -out firmware.hex
```

Acá la herramienta *firmware_words* genera el archivo de inicialización *firmware.hex* el cual debe tener el tamaño de la memoria declarada en el código en verilog (*rtl/cores/cpu/femtorv32-quark.v*) $16384 = 0x4000$ bytes:

```
module Memory
...

    reg [31:0] MEM [0:4095];
    initial begin
        $readmemh("../firmware.hex", MEM);
    end
...
endmodule
```

Finalmente el archivo *firmware.hex* es copiado al directorio donde se encuentra el código en verilog del procesador para ser incluido en la inicialización de la memoria.

```
cp firmware.hex ../rtl
```

1.9.1 simulación

Una vez generado el archivo de inicialización de la memoria de programa se procede a la simulación, al simular el SoC podemos observar el contenido del banco de registros, del contador de programa y los registros de la ALU y de esta forma verificar el correcto funcionamiento del program implementado.

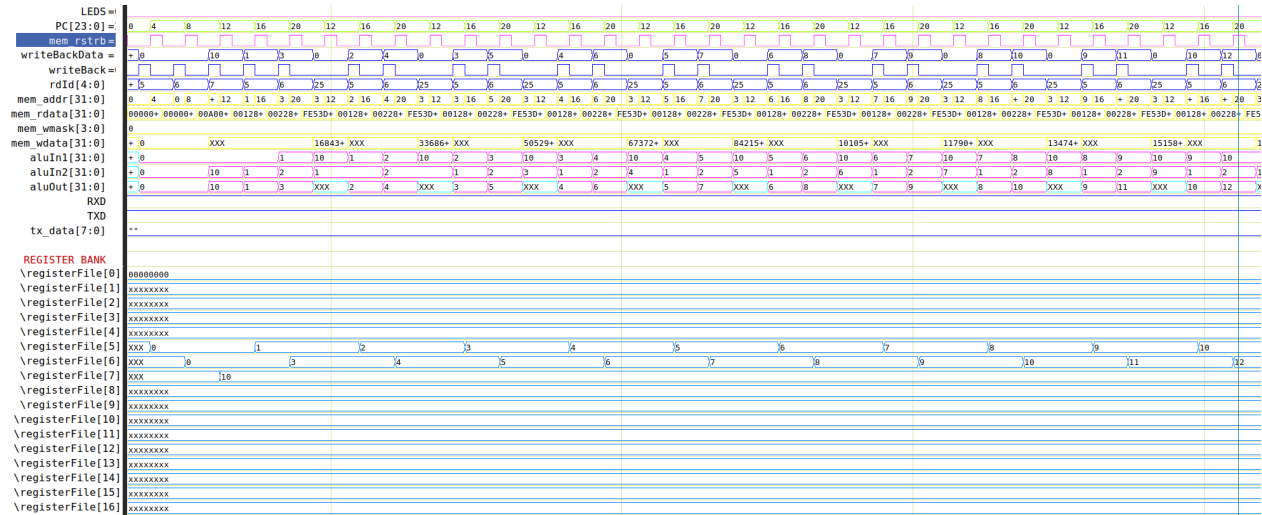


Figura 1.70 Simulación de la implementación del ciclo *for*.

1.9.2 Estructura de los ejemplos

Como parte del material de apoyo se cuenta con un repositorio git donde se aloja código fuente que puede ser utilizado para reproducir los ejemplos aquí mostrados. El directorio que contiene los ejemplos relacionados con la arquitectura HW-SW acá presentada es el siguiente:

```
-- firmware
|   |-- asm
|   |   |-- bram.ld
|   |   |-- for.S
|   |   |-- Makefile
|   |-- c
|   |   |-- hello.c
|   |   |-- libfemtorv
|   |   |-- linker.ld
|   |   |-- Makefile
|   |-- firmware_words_src
|   |   |-- firmware_words.cpp
|   |   |-- Makefile
|-- Makefile
-- rtl
|   |-- bench_quark.v
|   |-- cores
```

```

| | | -- bram
| | | -- cpu
| | | -- div
| | | -- mult
| | `-- uart
| | | | -- perip_uart.v
| | | | -- uart.v
|-- Makefile
|-- SOC_TB.v
`-- SOC.v

```

En el directorio firmware aparecen dos carpetas **asm** y **c** para código fuente en lenguaje **ensamblador** y en lenguaje **c** respectivamente.

En el directorio firmware se aloja todo el código relacionado con la descripción a alto nivel de los periféricos y del procesador. En la raíz de este directorio se encuentra el archivo SOC.v (ver figura 1.71) que describe la arquitectura mostrada en la figura 1.68. El subdirectorio **cores** contiene los periféricos del SOC una carpeta por cada periférico con un nombre representativo. Dentro de cada carpeta de periférico se encuentran por lo menos dos archivos uno que implementa la funcionalidad y el que implementa la conexión al procesador este último tendrá el prefijo **perip_**.

Se dispondrá de archivos **Makefile** en cada directorio para facilitar el proceso de síntesis, simulación, compilación y configuración, se recomienda familiarizarse con estos archivos.

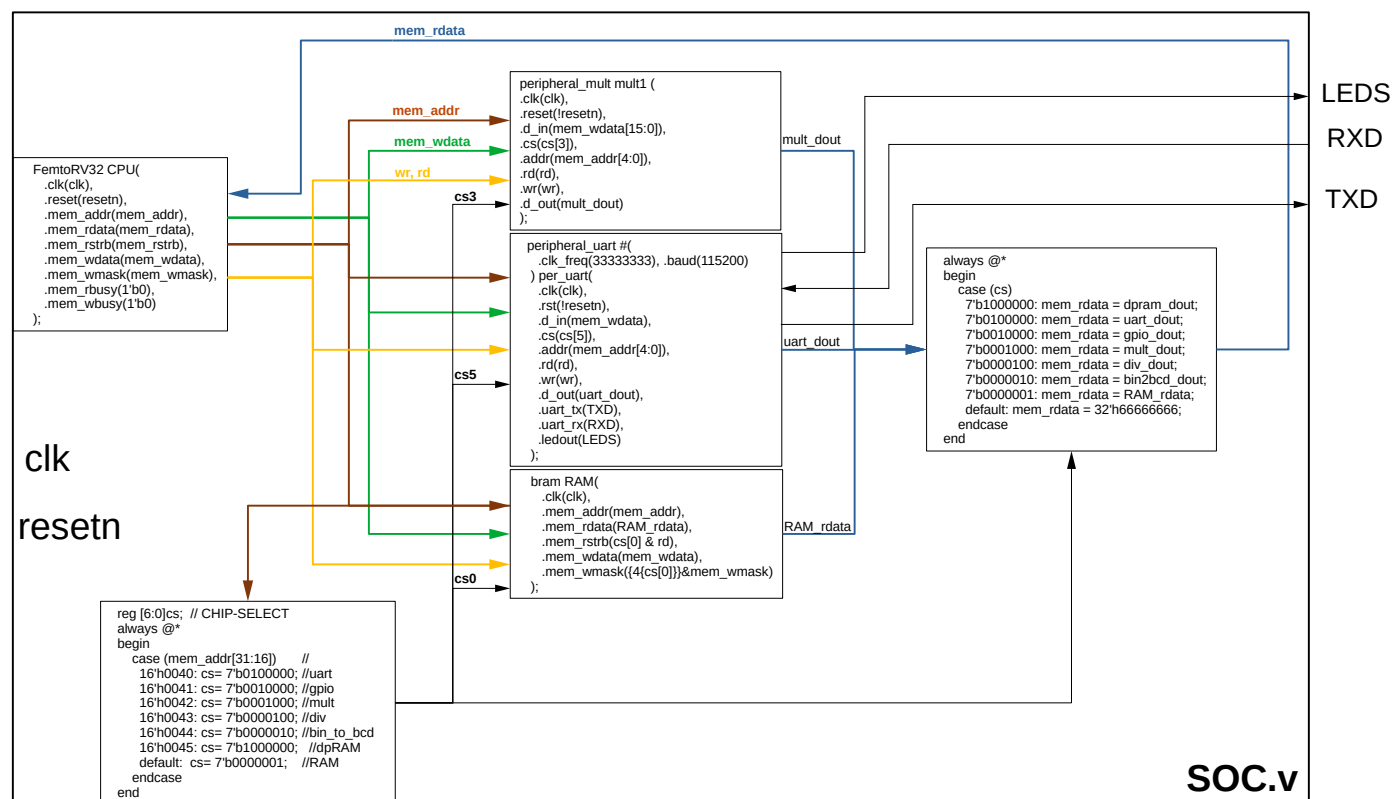


Figura 1.71 Sistem On Chip femtoRV

1.9.3 Aplicación software para utilizar el multiplicador

En la figura 1.72 se muestra la implementación del multiplicador en el procesador RISC-V-I utilizando las instrucciones (función *mult32*) y usando un periférico que implementa la multiplicación (función *mult_hw*).

```
.equ MULT_BASE, 0x420000
.equ MULT_OP_A, 0x04
.equ MULT_OP_B, 0x08
.equ MULT_INIT, 0x0C
.equ MULT_RESULT, 0x10
.equ MULT_DONE, 0x14

.section .text
.globl mult
.globl mult_hw

mult:
    addi sp, sp, -12    # store a3, a4, a5 on stack
    sw a3, 0(sp)
    sw a4, 4(sp)
    sw a5, 8(sp)

    mv a4, a0
    mv a5, a1    # a4 * a5
    li a0, 0    # PP = 0
.L2:
    andi a3, a5, 1    #
    beqz a3, .L1    # LSB of a5 = 0 ?
    add a0, a0, a4    # PP = PP + a4
.L1:
    srli a5, a5, 1    # a5 = a5/2 right shift
    slli a4, a4, 1    # a4 = a4*2 left shift
    bnez a5, .L2    # loop until a5 = 0

    lw a3, 0(sp)
    lw a4, 4(sp)
    lw a5, 8(sp)
    addi sp, sp, 12    # restore sp
    ret

mult_hw:
    li gp, MULT_BASE
    sw a0, MULT_OP_A(gp)
    sw a1, MULT_OP_B(gp)
    li a0, 1
    sw a0, MULT_INIT(gp)
    sw zero, MULT_DONE(gp)
.L0:
    li t0, 1
    lw t1, MULT_DONE(gp)
    and t1, t1, t0
    beqz t1, .L0
    lw a0, MULT_RESULT(gp)
    ret
```

Figura 1.72 Implementación Software (RISC-V-I) del multiplicador.

Se presentan estas dos implementaciones con fines de comparación, para analizar la velocidad de ejecución de una tarea software y una tarea Hardware.

En la figura 1.73 se muestra la relación entre las variables del código y el rtl del periférico

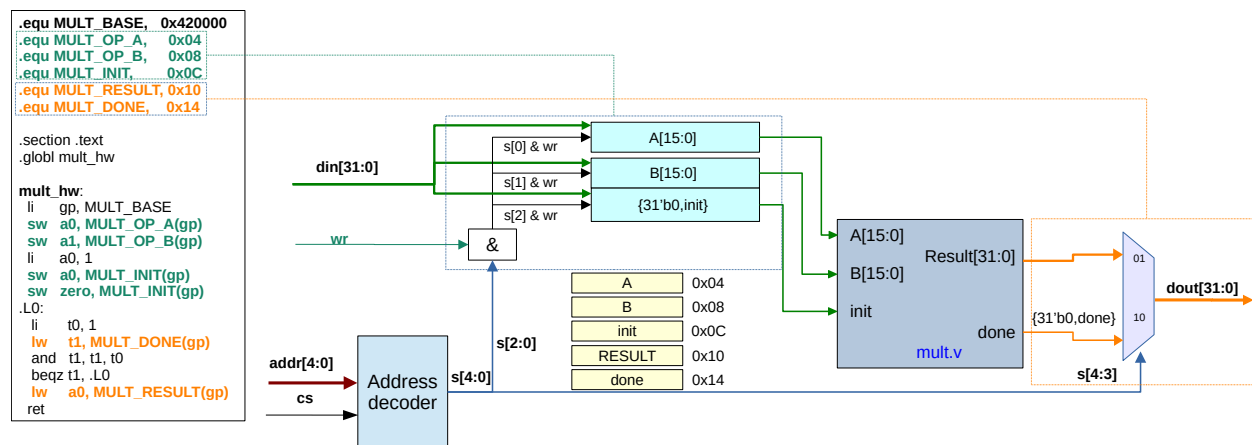


Figura 1.73 Implementación Software (RISC-V-I) del multiplicador.

1.10 Programación en lenguaje C

El lenguaje ensamblador es muy útil para entender los conceptos básicos de programación y para entender las operaciones internas que realiza el procesador, sin embargo, es poco práctico para la implementación de aplicaciones con mediana complejidad, además que impide la portabilidad a otras arquitecturas. El lenguaje C facilita la escritura de algoritmos y permite su ejecución en las plataformas que soporte el compilador (siempre que no se utilicen asignaciones a recursos propios del procesador).

Capítulo 2

Procesador RISC-V

2.1 Introducción

En el capítulo anterior se estudió la forma de implementar tareas hardware utilizando máquinas de estado algorítmicas. La implementación de tareas hardware es un proceso un poco tedioso ya que involucra la realización de una máquina de estados por cada tarea; la implementación del camino de datos se simplifica de forma considerable ya que existe un conjunto de bloques constructores que pueden ser tomados de una librería creada por el diseñador. El uso de tareas hardware se debe realizar únicamente cuando las restricciones temporales del diseño lo requieran, ya que como veremos en este capítulo, la implementación de tareas software es más sencilla y rápida.

La estructura de una máquina de estados algorítmica permite entender de forma fácil la estructura de un procesador ya que tienen los mismos componentes principales (unidad de control y camino de datos), la diferencia entre ellos es la posibilidad de programación y la configuración fija del camino de datos del procesador.

En este capítulo se estudiará la arquitectura del procesador MICO32 creado por la empresa Lattice semiconductor y gracias a que fué publicado bajo la licencia GNU, es posible su estudio, uso y modificación. En la primera sección se hace la presentación de la arquitectura; a continuación se realiza el análisis de la forma en que el procesador implementa las diferentes instrucciones, iniciando con las operaciones aritméticas y lógicas siguiendo con las de control de flujo de programa (saltos, llamado a función); después se analizarán la comunicación con la memoria de datos; y finalmente el manejo de interrupciones.

En la segunda sección se abordará la arquitectura de un SoC (System on a Chip) basado en el procesador LM32, se analizará la forma de conexión entre los periféricos y la CPU utilizando el bus wishbone; se realizará una descripción detallada de la programación de esta arquitectura utilizando herramientas GNU.

Este proceso se repetirá para el procesador RISC-V; RISC-V es una arquitectura de conjunto de instrucciones de hardware libre basado en un diseño de conjunto de instrucciones reducido (RISC), su carácter libre le permite ser utilizado sin tener que pagar licencias de ningún tipo. En este capítulo se utilizará la descripción del conjunto de instrucciones RV32I implementada por Bruno Levy

2.2 Arquitectura del procesador RV32I

El RISC-V posee una estructura modular que permite adaptar la arquitectura a requerimientos funcionales y económicos, en la figura ?? se muestra el concepto de modularidad, el módulo base es un procesador de 32 bits de base entera (RV32I) al que se le pueden integrar módulos para realizar multiplicaciones y divisiones enteras (RV32M), instrucciones atómicas (instrucciones que automáticamente modifican la lectura-escritura a memoria para permitir sincronización entre múltiples procesadores RISC-V) (RV32A) e instrucciones comprimidas (Instrucciones que engloban una serie de instrucciones, se usa para reducir el tamaño de la memoria de programa) (RV32C). Adicionalmente existen otras 24 extensiones entre las que se encuentran las de operaciones aritméticas de punto flotante de precisión simple (RV32F) y las de doble precisión (RV32D).

RV32IMAC

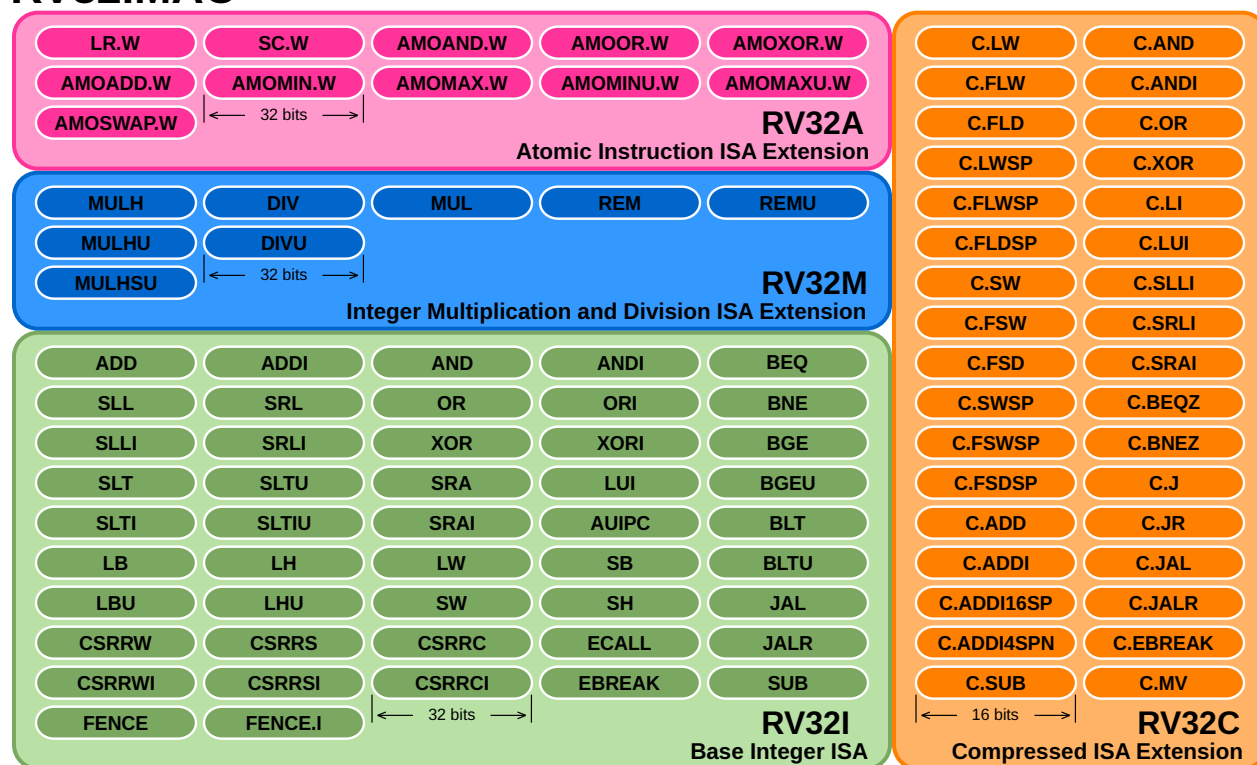


Figura 2.1 Set de instrucciones modular de la variante RV32IMAC. CPU de 32 bits de Base entera (RV32I), con extensión ISA para multiplicación y división entera (RV32M), instrucciones Atómicas (RV32A) e instrucciones Comprimadas (RV32C); fuente: Eduardo Corpeño (<https://github.com/kuashio>)

Registros

El RISC-V posee 32 registros (o 16 en la variante embebida). En la tabla ?? se muestran sus nombres.

Register	ABI Name	Description	Saver
x0	zero	Zero constant	—
x1	ra	Return address	Caller
x2	sp	Stack pointer	Callee
x3	gp	Global pointer	—
x4	tp	Thread pointer	—
x5-x7	t0-t2	Temporaries	Caller
x8	s0 / fp	Saved / frame pointer	Callee
x9	s1	Saved register	Callee
x10-x11	a0-a1	Fn args/return values	Caller
x12-x17	a2-a7	Fn args	Caller
x18-x27	s2-s11	Saved registers	Callee
x28-x31	t3-t6	Temporaries	Caller
f0-7	ft0-7	FP temporaries	Caller
f8-9	fs0-1	FP saved registers	Callee
f10-11	fa0-1	FP args/return values	Caller
f12-17	fa2-7	FP args	Caller
f18-27	fs2-11	FP saved registers	Callee
f28-31	ft8-11	FP temporaries	Caller

Cuadro 2.1 Registros del procesador RV32I.

Set de Instrucciones

En la tabla 1.1 se puede apreciar la lista de las instrucciones del RV32I con su respectivo código y función.

Inst	Name	FMT	Opcode	funct3	funct7	Description (C)
add	ADD	R	0110011	0x0	0x00	rd = rs1 + rs2
sub	SUB	R	0110011	0x0	0x20	rd = rs1 - rs2
xor	XOR	R	0110011	0x4	0x00	rd = rs1 ^ rs2
or	OR	R	0110011	0x6	0x00	rd = rs1 — rs2
and	AND	R	0110011	0x7	0x00	rd = rs1 & rs2
sll	Shift Left Logical	R	0110011	0x1	0x00	rd = rs1 << rs2
srl	Shift Right Logical	R	0110011	0x5	0x00	rd = rs1 >> rs2
sra	Shift Right Arith*	R	0110011	0x5	0x20	rd = rs1 >> rs2
slt	Set Less Than	R	0110011	0x2	0x00	rd = (rs1 < rs2)?1:0
sltu	Set Less Than (U)	R	0110011	0x3	0x00	rd = (rs1 < rs2)?1:0
addi	ADD Immediate	I	0010011	0x0		rd = rs1 + imm
xori	XOR Immediate	I	0010011	0x4		rd = rs1 ^ imm
ori	OR Immediate	I	0010011	0x6		rd = rs1 — imm
andi	AND Immediate	I	0010011	0x7		rd = rs1 & imm
slli	Shift Left Logical Imm	I	0010011	0x1	imm[5:11]=0x00	rd = rs1 << imm[0:4]
srlr	Shift Right Logical Imm	I	0010011	0x5	imm[5:11]=0x00	rd = rs1 >> imm[0:4]
srai	Shift Right Arith Imm	I	0010011	0x5	imm[5:11]=0x20	rd = rs1 >> imm[0:4]
slti	Set Less Than Imm	I	0010011	0x2		rd = (rs1 < imm)?1:0
sltiu	Set Less Than Imm (U)	I	0010011	0x3		rd = (rs1 < imm)?1:0
lb	Load Byte	I	0000011	0x0		rd = M[rs1+imm][0:7]
lh	Load Half	I	0000011	0x1		rd = M[rs1+imm][0:15]
lw	Load Word	I	0000011	0x2		rd = M[rs1+imm][0:31]
lbu	Load Byte (U)	I	0000011	0x4		rd = M[rs1+imm][0:7]
lhu	Load Half (U)	I	0000011	0x5		rd = M[rs1+imm][0:15]
sb	Store Byte	S	0100011	0x0		M[rs1+imm][0:7] = rs2[0:7]
sh	Store Half	S	0100011	0x1		M[rs1+imm][0:15] = rs2[0:15]
sw	Store Word	S	0100011	0x2		M[rs1+imm][0:31] = rs2[0:31]
beq	Branch ==	B	1100011	0x0		if(rs1 == rs2) PC += imm
bne	Branch !=	B	1100011	0x1		if(rs1 != rs2) PC += imm
blt	Branch <	B	1100011	0x4		if(rs1 < rs2) PC += imm
bge	Branch ≥	B	1100011	0x5		if(rs1 ≥ rs2) PC += imm
bltu	Branch < (U)	B	1100011	0x6		if(rs1 < rs2) PC += imm
bgeu	Branch ≥ (U)	B	1100011	0x7		if(rs1 ≥ rs2) PC += imm
jal	Jump And Link	J	1101111			rd = PC+4; PC += imm
jalr	Jump And Link Reg	I	1100111	0x0		rd = PC+4; PC = rs1 + imm
lui	Load Upper Imm	U	0110111			rd = imm << 12
auipc	Add Upper Imm to PC	U	0010111			rd = PC + (imm << 12)
ecall	Environment Call	I	1110011	0x0	imm=0x0	Transfer control to OS
ebreak	Environment Break	I	1110011	0x0	imm=0x1	Transfer control to debugger

Cuadro 2.2 Set de Instrucciones del RV32I. fuente: <https://github.com/jameslzh/riscv-card/tree/master>

2.2.1 Diagrama de Bloques del FemtoRV

Como se mencionó anteriormente, en este capítulo se utilizará el procesador **femtoRV** desarrollado por Bruno Levy¹. En la figura ?? se muestra el diagrama de bloques del procesador.

¹ <https://github.com/BrunoLevy>

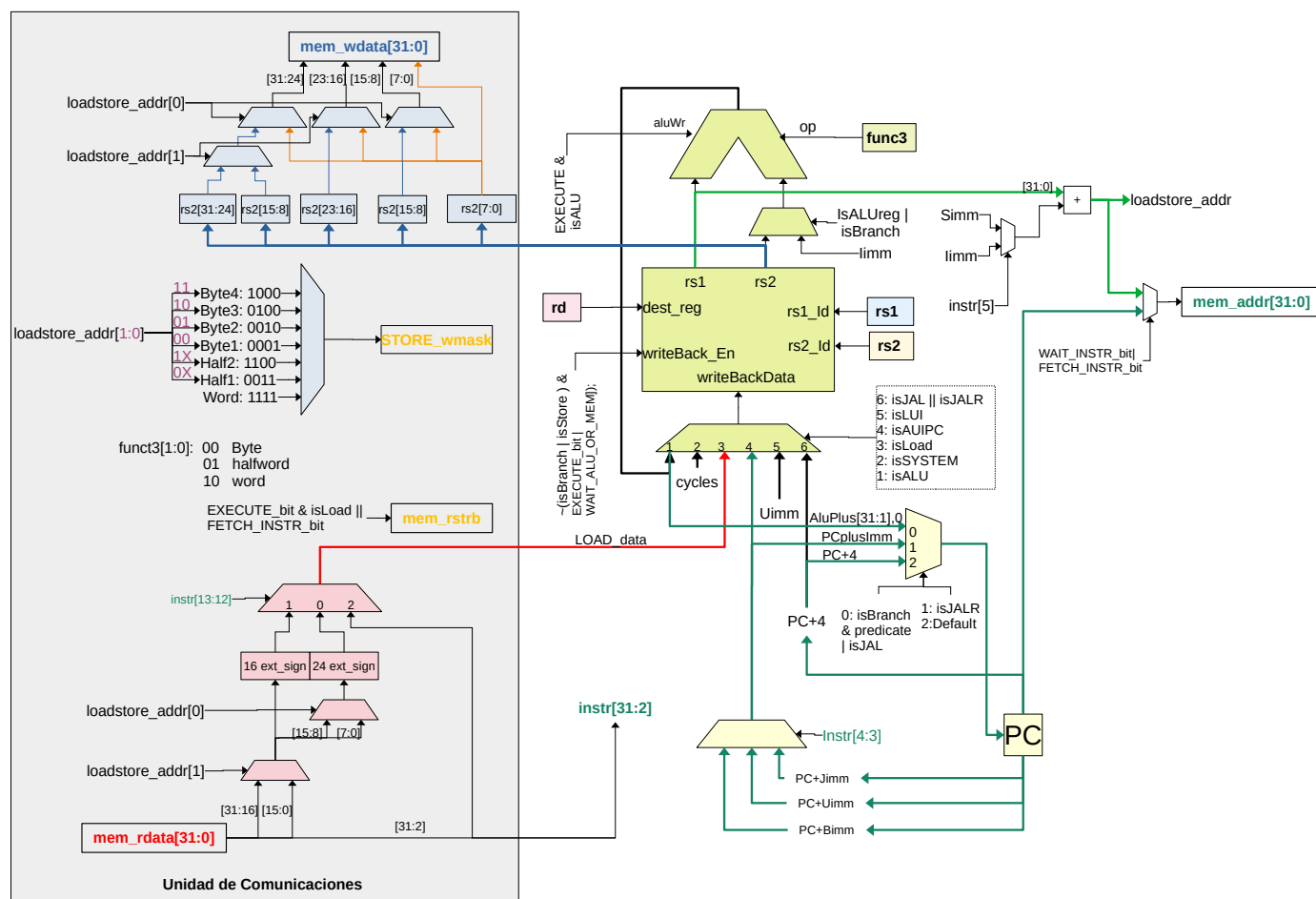


Figura 2.2 Diagrama de bloques del RV32I

En esta figura podemos observar el camino de datos típico del procesador sin etapas de *pipeline*; es un camino compuesto por la Unidad Aritmética y Lógica que a su vez se encarga de calcular los valores del contador de programa (PC) ante instrucciones de salto, y se encarga de realizar las operaciones para los saltos condicionales. La interfaz del procesador con el mundo exterior se realiza a través de los buses:

- Dirección: **mem_addr**: Encargado de direccionar la memoria externa donde se almacenan las instrucciones que componen el programa y de realizar operaciones de lectura y escritura a los periféricos.
- Salida de Datos: **mem_wdata**: Por este bus se envían los datos hacia los periféricos.
- Entrada de Datos: **mem_rdata**: Por este bus se reciben los datos provenientes de los periféricos y las instrucciones almacenadas en la memoria de programa.
- Control **mem_rstrb**, **store_wmask**: Señales que indican a los periféricos y a la memoria de programa cuando se pueden realizar las operaciones de lectura y escritura.

2.2.2 Set de Instrucciones

Como se mencionó anteriormente, el RISC-V posee un set de instrucciones reducido que permite implementar las diferentes aplicaciones, en esta sub-sección mostraremos cómo se implementan las diferentes instrucciones en el camino de datos mostrado en la figura ??.

Instrucciones aritméticas y lógicas

Las instrucciones aritméticas y lógicas que puede implementar la variante RV32I pueden verse en la figura ???. Existen dos formatos para este tipo de operaciones, en el primero los operandos y el resultado se almacenan en el banco de registros, mientras que en el segundo uno de los operandos proviene de la misma instrucción, en las figuras ?? y ?? se muestran los componentes del camino de datos que intervienen en estas instrucciones y el formato de la instrucción.

Operaciones Aritméticas y lógicas entre registros Como puede verse en la figura ??? en la instrucción existen campos donde se indica el ID de los registros fuente y destino **rs1**, **rs2** y **rd**, cada uno de estos campos tiene 5 bits, lo que permite colocar cualquiera de los 32 registros del RISC-V. El camino de datos se simplifica al conectar la salida del banco de registros a la entrada de la ALU y la salida de esta a la entrada de datos del banco de registros.

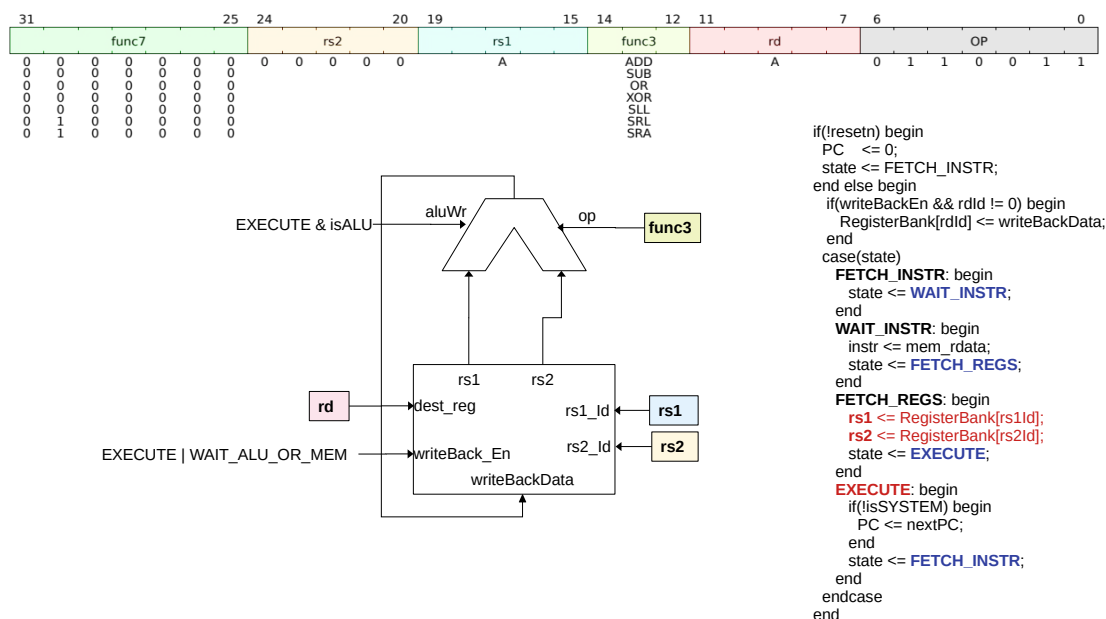


Figura 2.3 Instrucciones aritméticas entre registros del RV32I

Operaciones Aritméticas y Lógicas con un operador inmediato Las operaciones aritméticas pueden realizarse con operandos que están inmersos en la instrucción a este tipo de operaciones se les conoce con el nombre de inmediatas; como puede verse en la figura ??, la instrucción posee 12 bits reservados para este operando, 5 bits para el ID del registro donde se almacena el segundo operando **rs1** y 5 bits para el ID del registro donde se almacenará el resultado **rd**. El camino de datos se modifica para que la ALU reciba directamente el operando desde la instrucción.

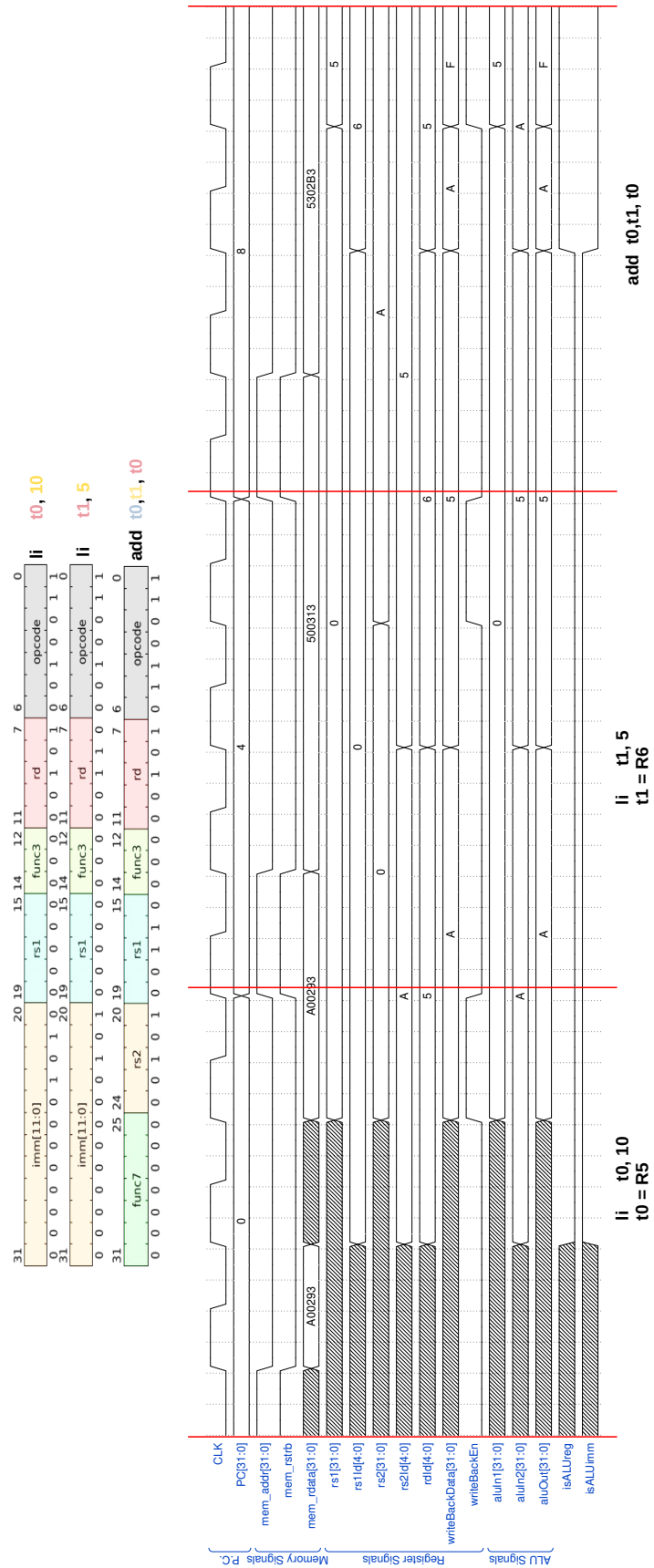


Figura 2.5 Formas de onda de la simulación de las instrucciones aritméticas inmediatas y entre registros del RV32I

2.2.3 Saltos

No es posible realizar algoritmos sin que se realicen saltos (cambios del flujo de ejecución del procesador), estos permiten realizar decisiones, ciclos, etc. Existen dos tipos de saltos:

- Incondicionales: El salto se realiza sin ninguna condición.
- Condicionales: Se debe cumplir una condición para que se pueda realizar el salto.

Saltos Incondicionales

Estas instrucciones permiten modificar el flujo de ejecución del programa y existen dos tipos:

- JAL: Valor a donde se salta almacenado en la Instrucción.
- JALR: Valor a donde se salta almacenado en la Instrucción y en un Registro.

En ambos casos la acción de la instrucción es modificar el valor del contador de programa (PC) para que la siguiente instrucción a ejecutar se encuentre en un lugar diferente al actual. En ambos casos se almacena el valor de la siguiente instrucción que se debería ejecutar si no existiera el salto (PC+4) lo que permite volver al sitio del llamado del salto (así operan las funciones).

En la figura ?? se muestra la instrucción **JAL** en ella podemos ver que el contador de programa se modifica a $PC + Jimm$.

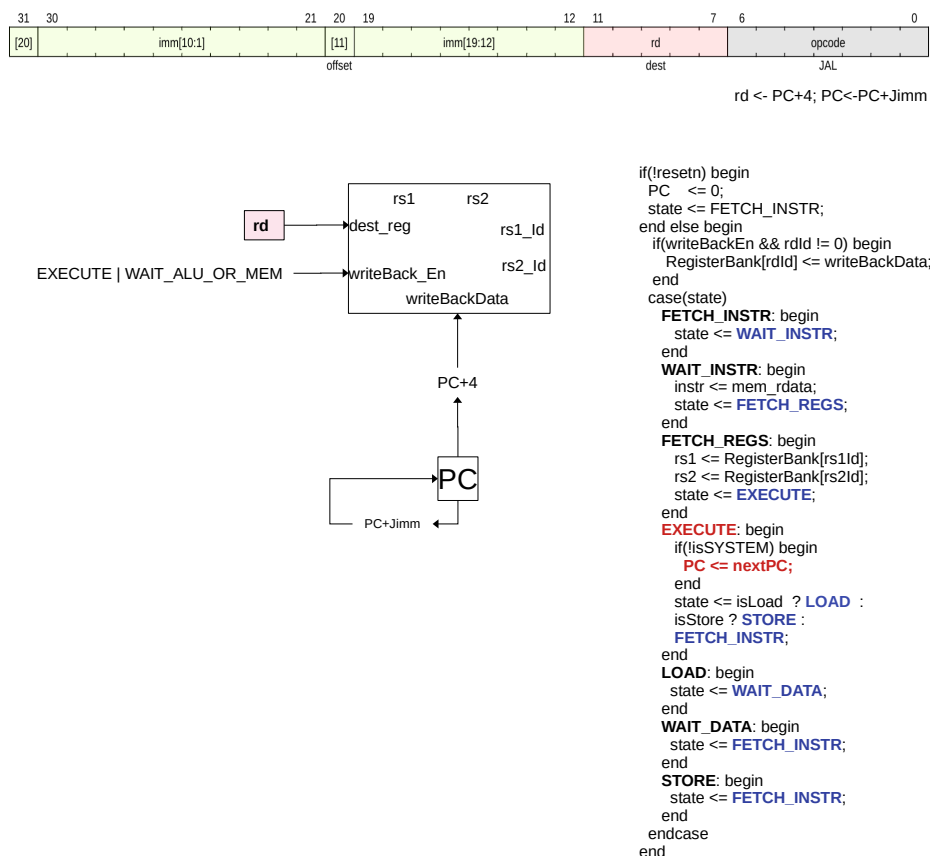
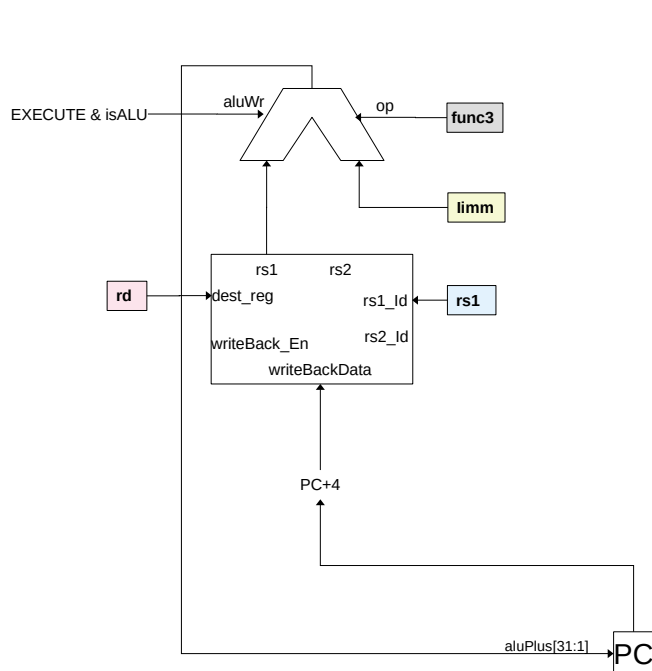
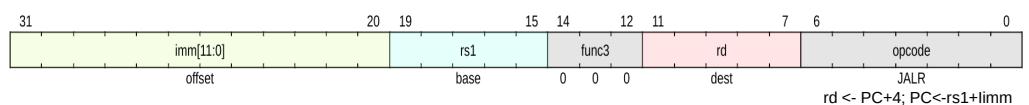


Figura 2.6 Instrucción JAL en el RV32I

En la figura ?? se muestra la instrucción **JALR**, aquí el valor actual del PC se almacena en el banco de registros y la suma del valor del registro **rs1** se suma al valor imm proveniente de la instrucción se almacena en el contador de programa (PC).



```

if(!resetn) begin
    PC <= 0;
    state <= FETCH_INSTR;
end else begin
    if(writeBackEn && rdd != 0) begin
        RegisterBank[rdd] <= writeBackData;
    end
    case(state)
        FETCH_INSTR: begin
            state <= WAIT_INSTR;
        end
        WAIT_INSTR: begin
            instr <= mem_rdata;
            state <= FETCH_REGS;
        end
        FETCH_REGS: begin
            rs1 <= RegisterBank[rs1ld];
            rs2 <= RegisterBank[rs2ld];
            state <= EXECUTE;
        end
        EXECUTE: begin
            if(!isSYSTEM) begin
                PC <= nextPC;
            end
            state <= isLoad ? LOAD :
                isStore ? STORE :
                FETCH_INSTR;
        end
        LOAD: begin
            state <= WAIT_DATA;
        end
        WAIT_DATA: begin
            state <= FETCH_INSTR;
        end
        STORE: begin
            state <= FETCH_INSTR;
        end
    endcase
end
end

```

Figura 2.7 Instrucción JALR del RV32I

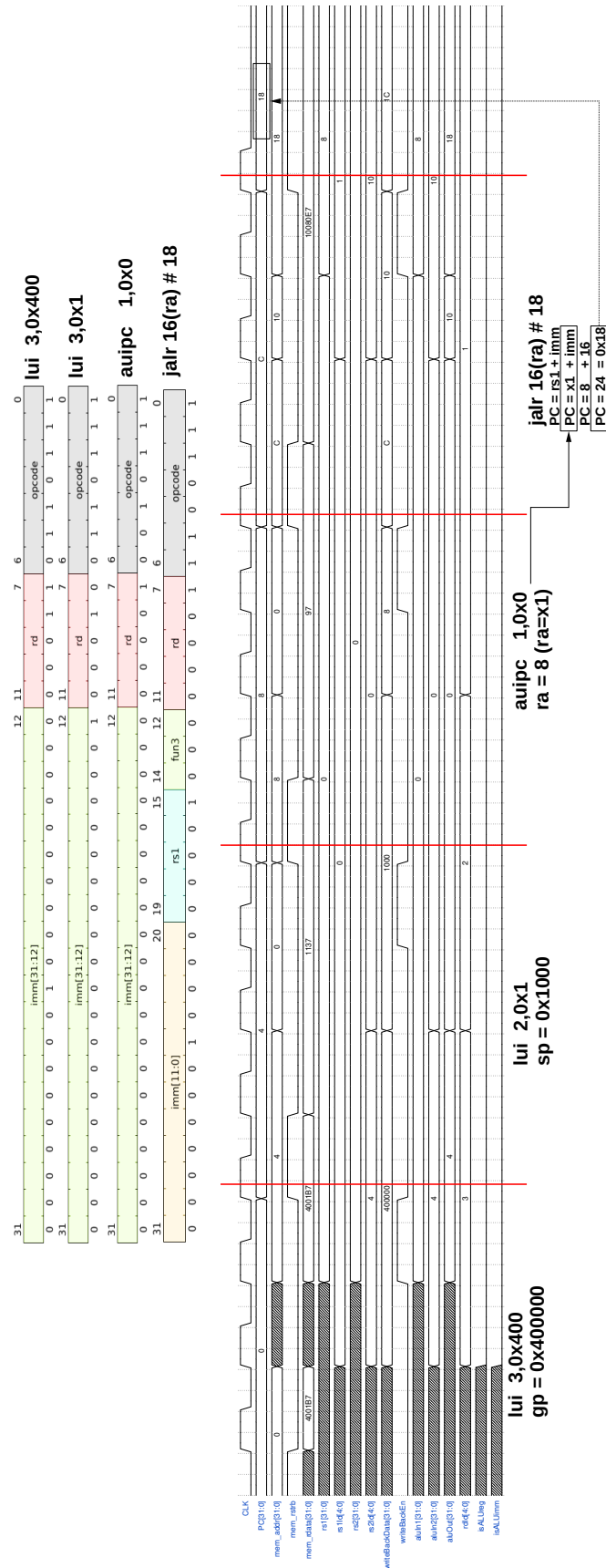


Figura 2.8 Formas de onda de la simulación de la instrucción JALR del RV32I

2.2 Arquitectura del procesador RV32I

77

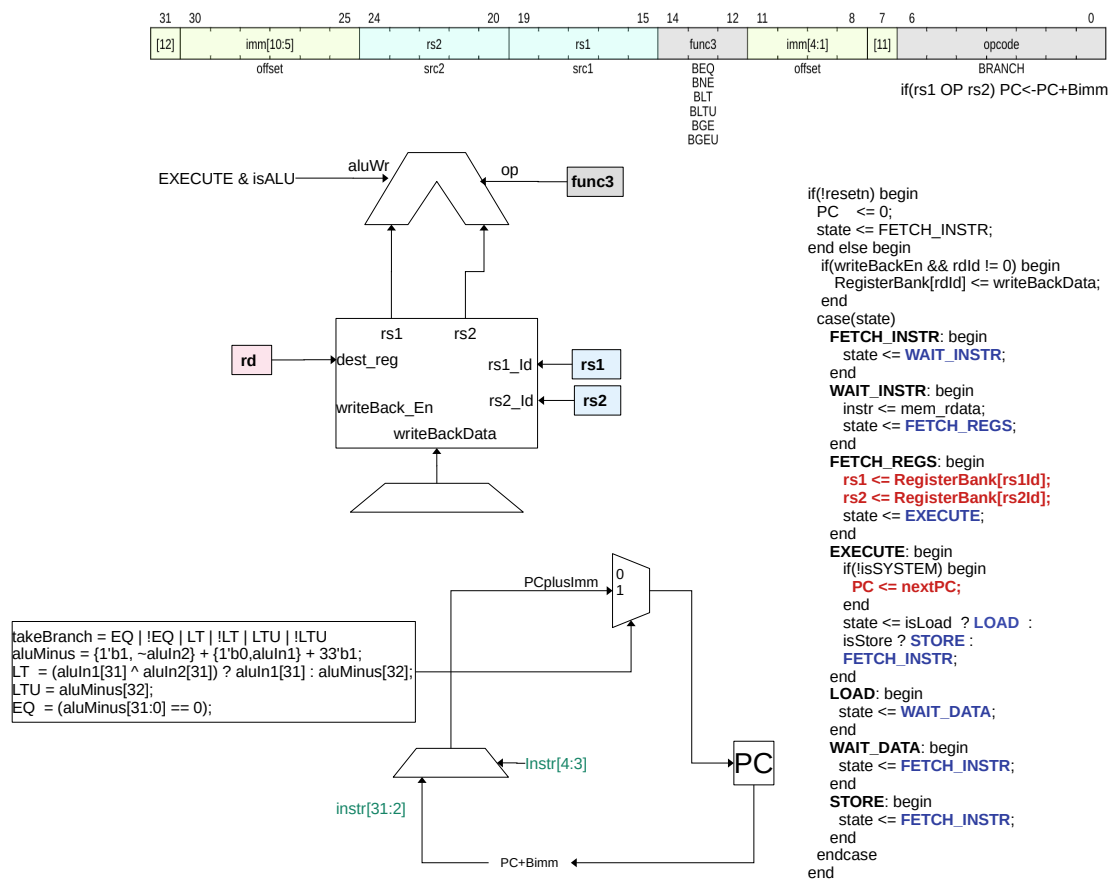


Figura 2.9 Saltos condicionales en el RV32I

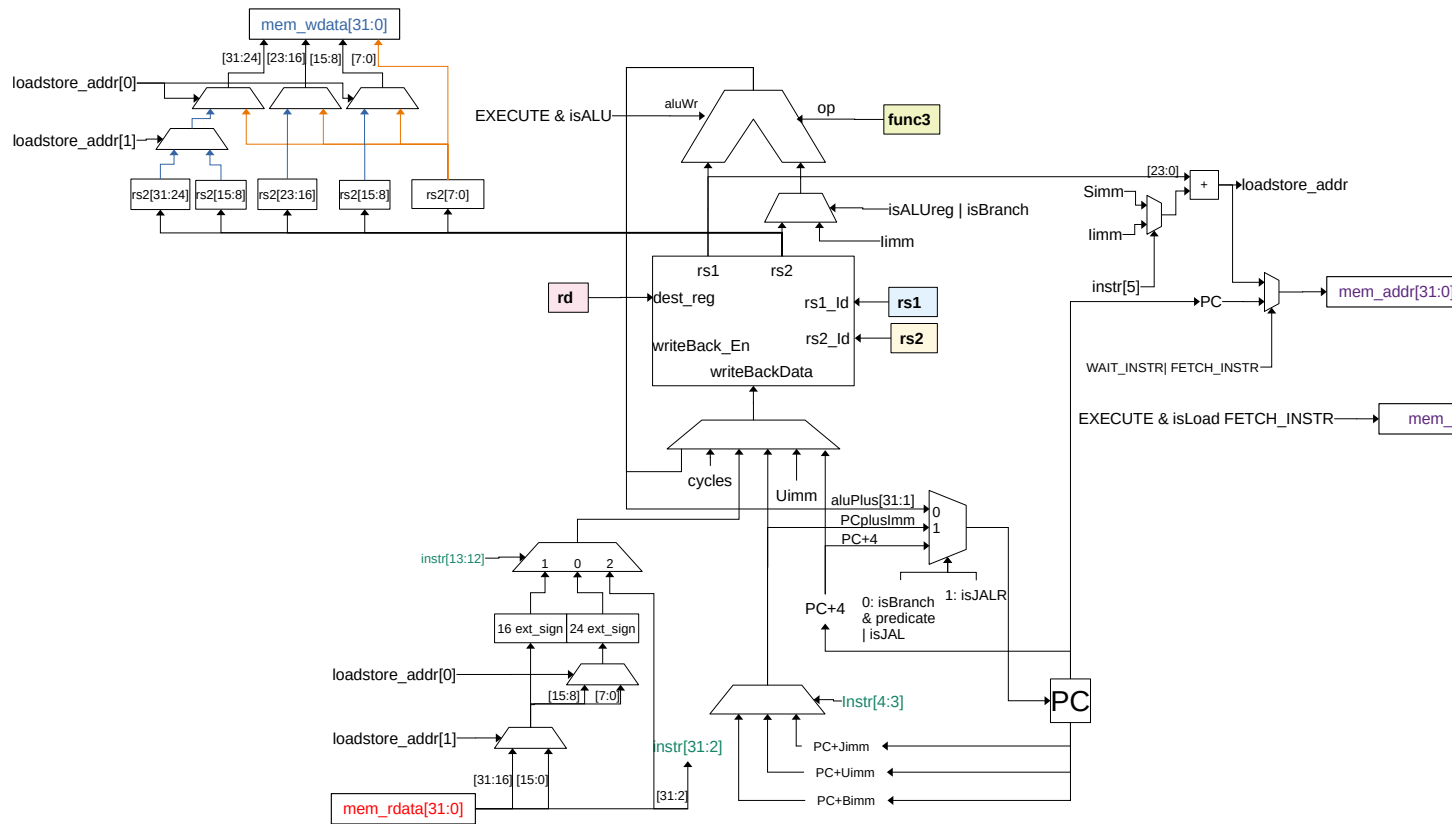


Figura 2.10 Lectura desde memoria externa en el RV32I

2.2 Arquitectura del procesador RV32I

79

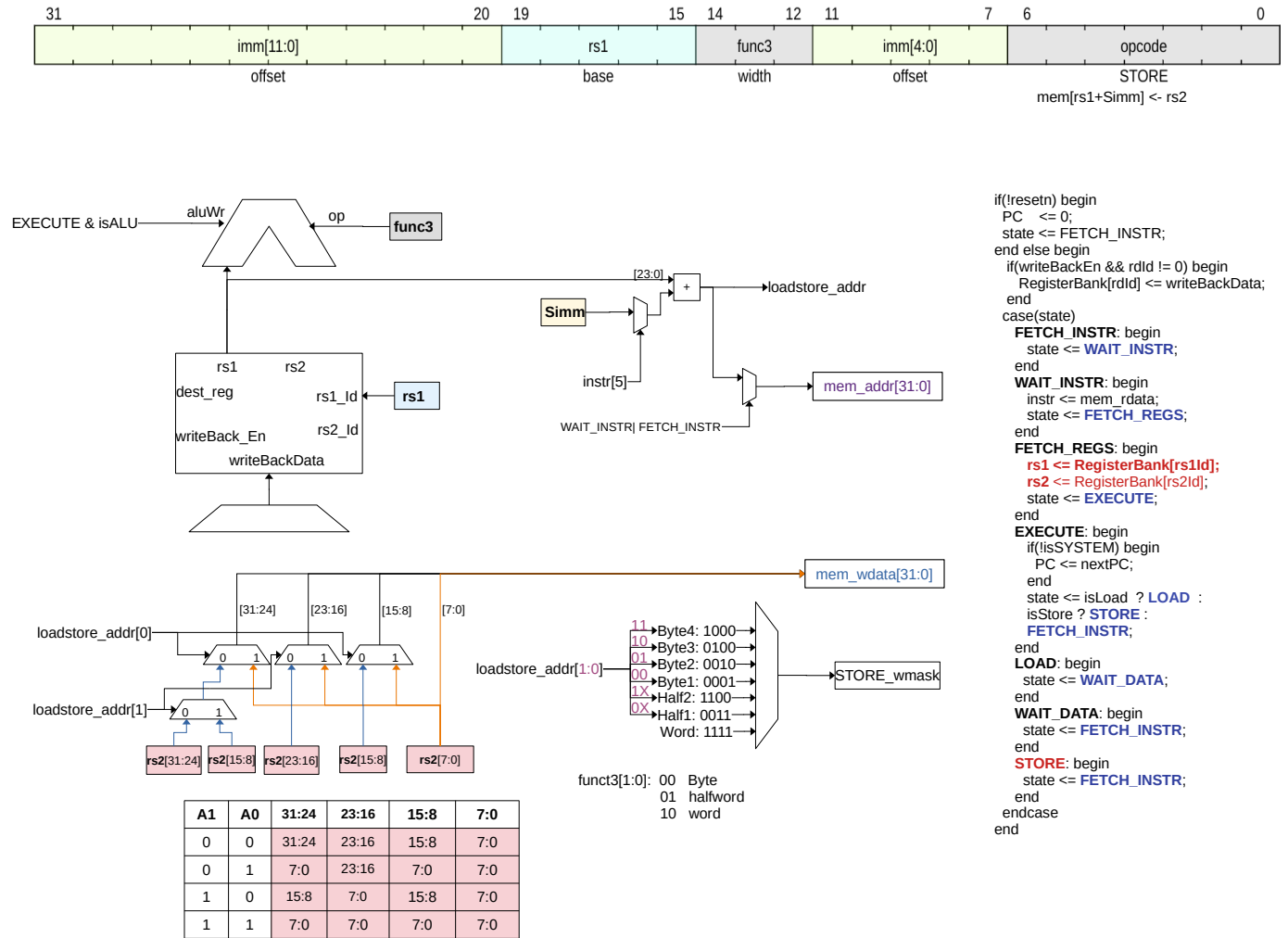


Figura 2.11 Escritura a memoria externa en el RV32I

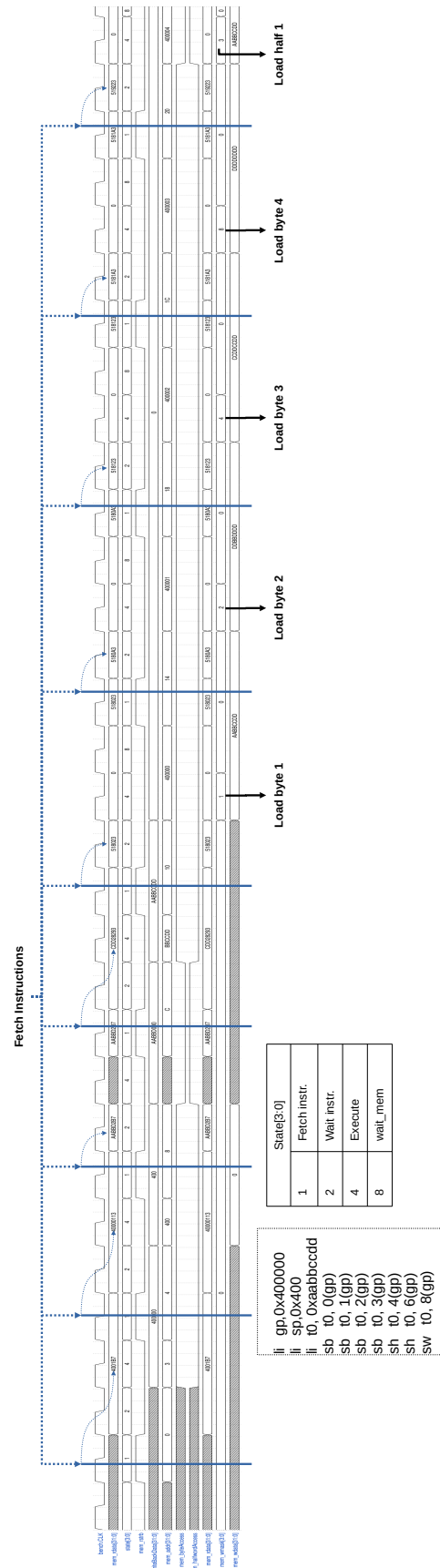


Figura 2.12 Simulación de una escritura a memoria externa en el RV32I

2.2.4 Arquitectura del SOC basado en RV32I

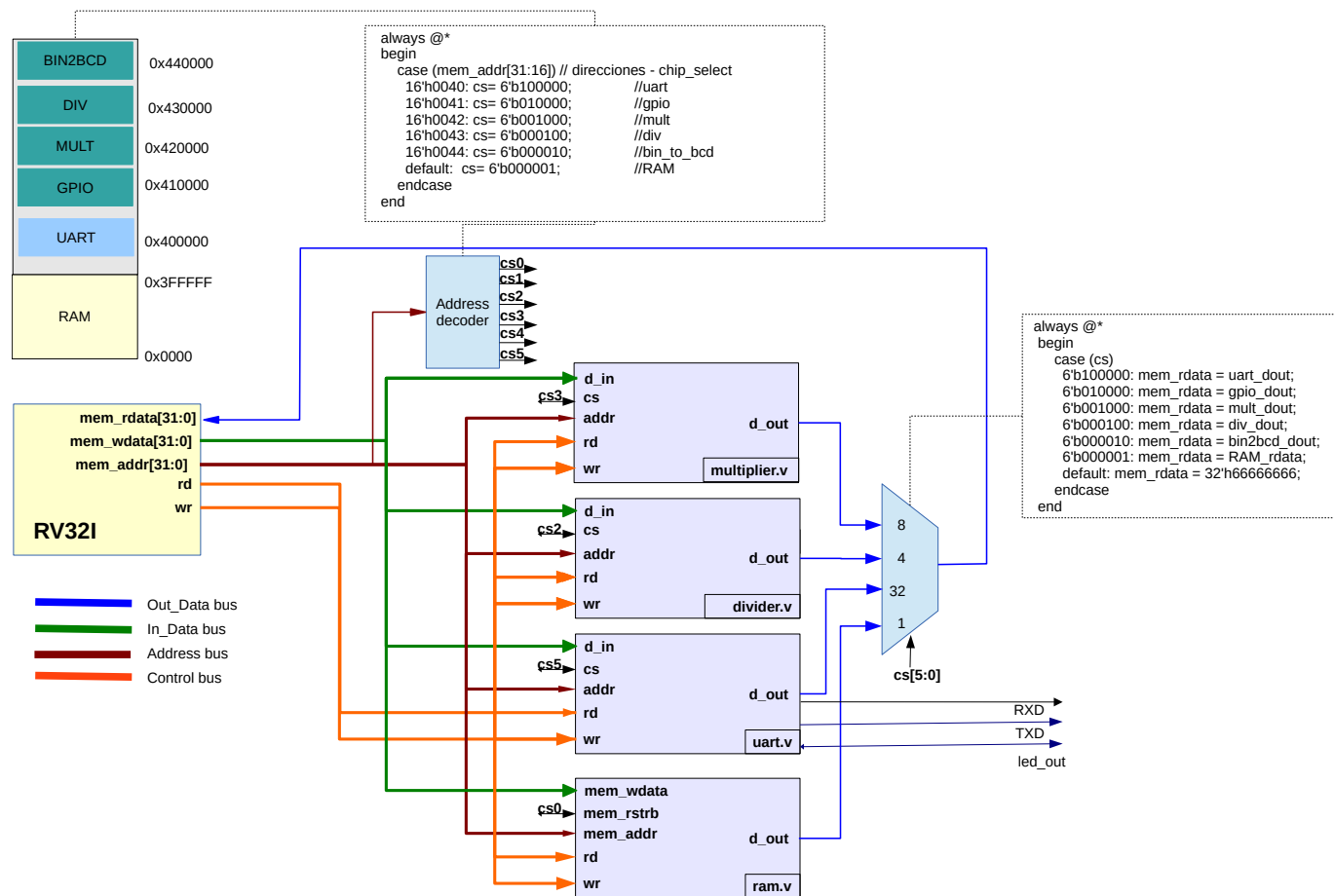


Figura 2.13 Diagrama de bloques del SoC basado en el RV32I

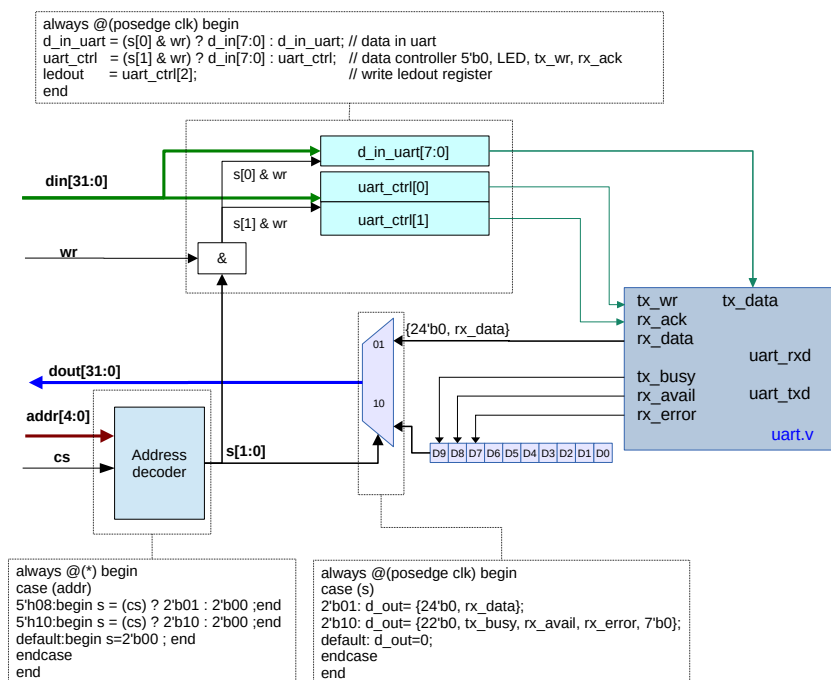


Figura 2.14 Diagrama de bloques del periférico uart

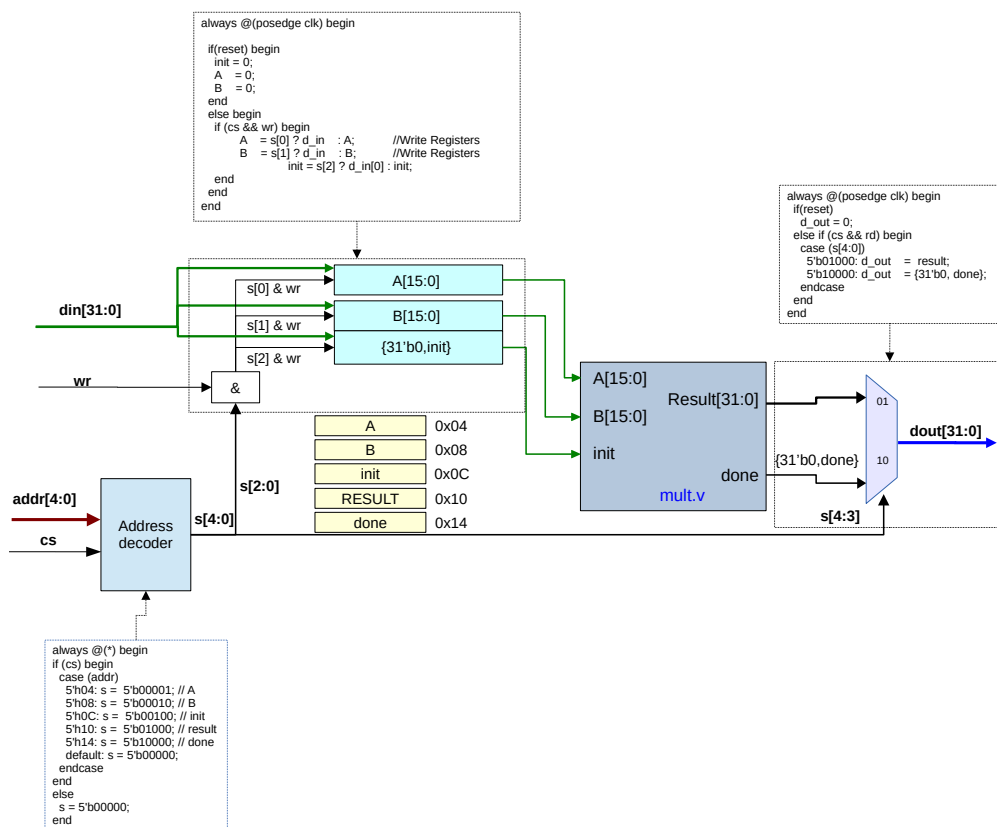


Figura 2.15 Diagrama de bloques del periférico mult

2.2.5 Extensión RV32M

El set de instrucciones RV32I no posee un multiplicador ni un divisor implementado en hardware, por esta razón es necesario incluir la librería matemática al momento de realizar el enlazado.

Si se observa el archivo firmware.lst generado en los ejemplos que se simularon para el RV32I se observan las funciones `_divsi3`, `_umodsi3`, y `_modsi3`. El problema de realizar estas operaciones por software es que toman muchos ciclos de reloj, lo que reduce el tiempo de respuesta del procesador y puede poner en riesgo el cumplimiento de las restricciones temporales. La extensión RV32M (ver tabla ??) añade estas operaciones al incluir módulos hardware que las implementan. En la tabla

Cuadro 2.3 Extensión RM32M fuente: <https://github.com/jameslzhuriscv-card/>

Inst	Name	FMT	Opcode	funct3	funct7	Description (C)
mul	MUL	R	0110011	0x0	0x01	$rd = (rs1 * rs2)[31:0]$
mulh	MUL High	R	0110011	0x1	0x01	$rd = (rs1 * rs2)[63:32]$
mulhsu	MUL High (S) (U)	R	0110011	0x2	0x01	$rd = (rs1 * rs2)[63:32]$
mulhu	MUL High (U)	R	0110011	0x3	0x01	$rd = (rs1 * rs2)[63:32]$
div	DIV	R	0110011	0x4	0x01	$rd = rs1 / rs2$
divu	DIV (U)	R	0110011	0x5	0x01	$rd = rs1 / rs2$
rem	Remainder	R	0110011	0x6	0x01	$rd = rs1 \% rs2$
remu	Remainder (U)	R	0110011	0x7	0x01	$rd = rs1 \% rs2$

Bruno Levy modifica la ALU para incluir estas instrucciones adicionando los bloques que se muestran en la figura ??

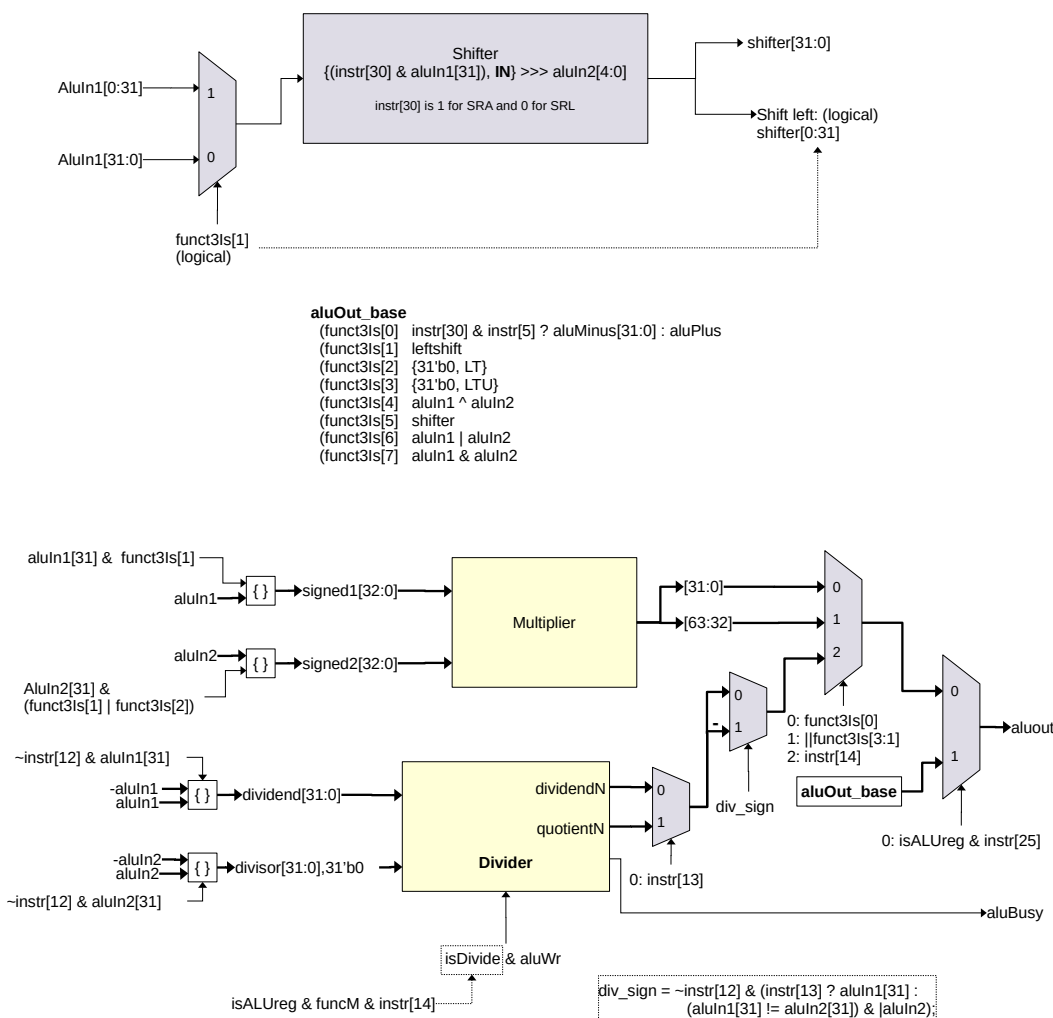


Figura 2.16 Diagrama de bloques del módulo aritmético del RV32M

En la figura ?? se muestra el diagrama de bloques del RV32M la ALU contiene los bloques representados en la figura ??, en este diagrama se elimina la entrada directa del contador de ciclos del RV32I (*cycles*) y se pasa a una estructura CSR (Registro de Status y Control), el funcionamiento es el mismo y es necesario ya que con este módulo se puede implementar la función *delay*.

La máquina de control es casi igual a la del RV32I, se introduce una nueva condición para *needToWait*, para el RV32I dependía de *isLoad* e *isStore*, para el RV32M se adiciona *isDivide* ya que el algoritmo de división se ejecuta en varios ciclos de reloj.

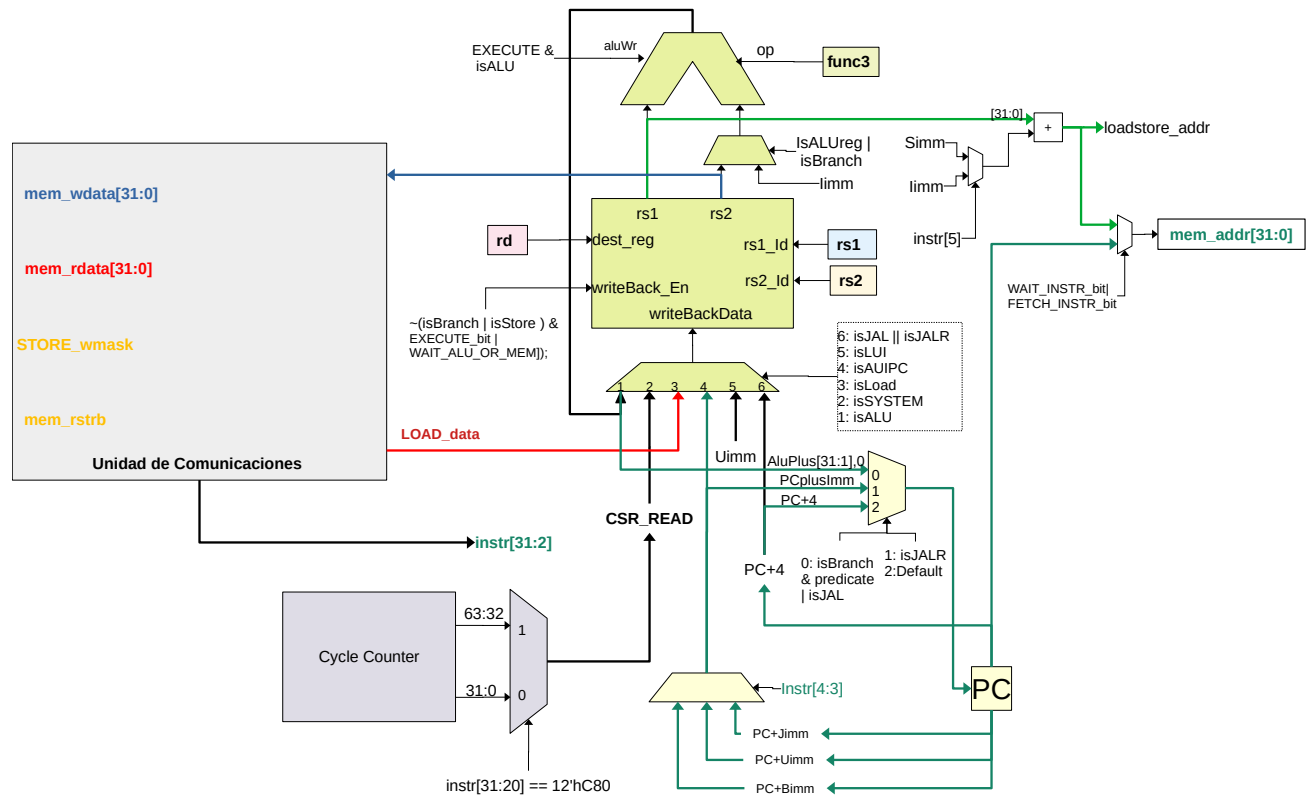


Figura 2.17 Diagrama de bloques del RV32M

2.2.6 Flujo Hardware y Software para programar el RV32I

En la Figura ?? se observa el flujo requerido para una aplicación sencilla en el SoC *femtorv32*; en esta figura mostramos la implementación de un ciclo *for*. El código fuente es compilado (gcc, as) para generar un objeto que representa las instrucciones necesarias para implementar la funcionalidad requerida. El enlazador (ld) genera el ejecutable en formato ELF, con las secciones y direcciones para: la memoria de programa (.text), las variables inicializadas (.rodata), las constantes (.data) y las variables sin inicializar(.bss).

La aplicación *firmware_words* genera el archivo de inicialización de la memoria RAM del SoC a partir de la sección .data.

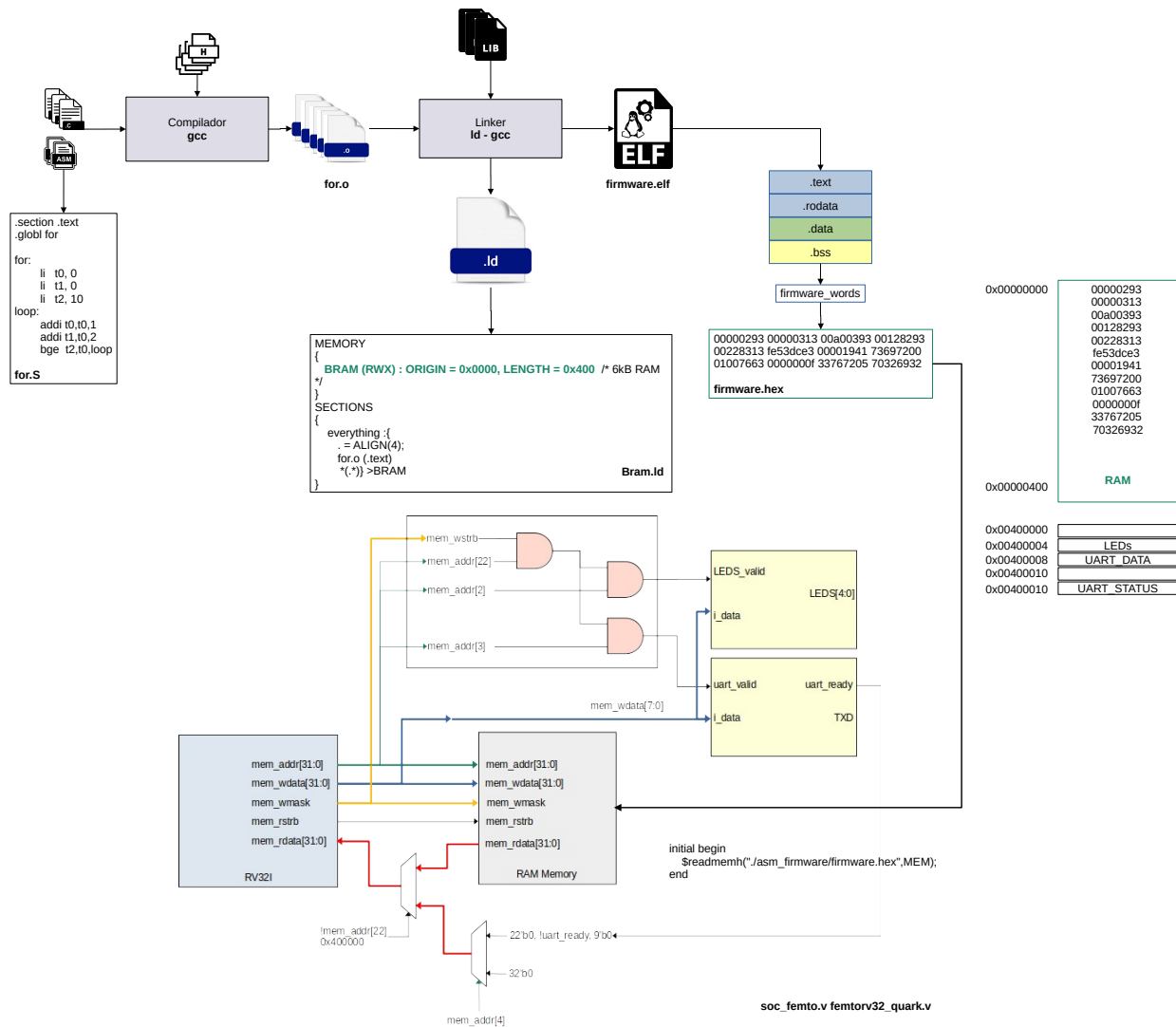


Figura 2.18 Flujo software (assembler) para el procesador femtoRV

En la página <https://luplab.gitlab.io/rvcodecs/> se encuentra una utilidad que permite decodificar las instrucciones y facilita el entendimiento de los programas

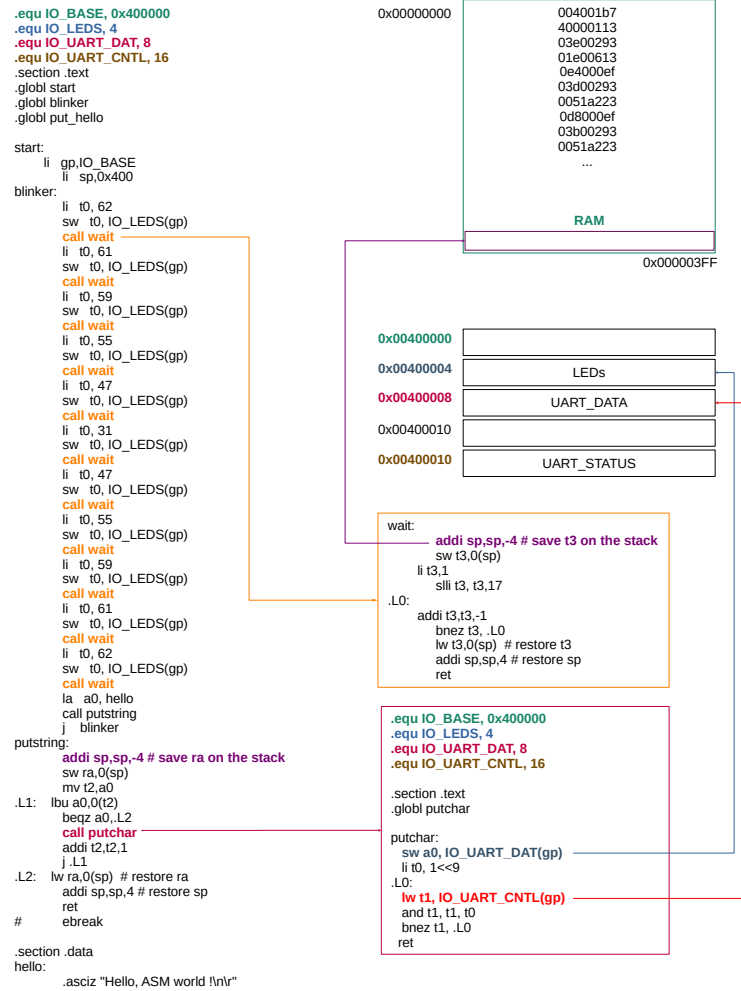


Figura 2.19 Manejo de periféricos LEDs y UART en el RV32I

2.2.7 Ejemplos

Conversor Binario a BCD

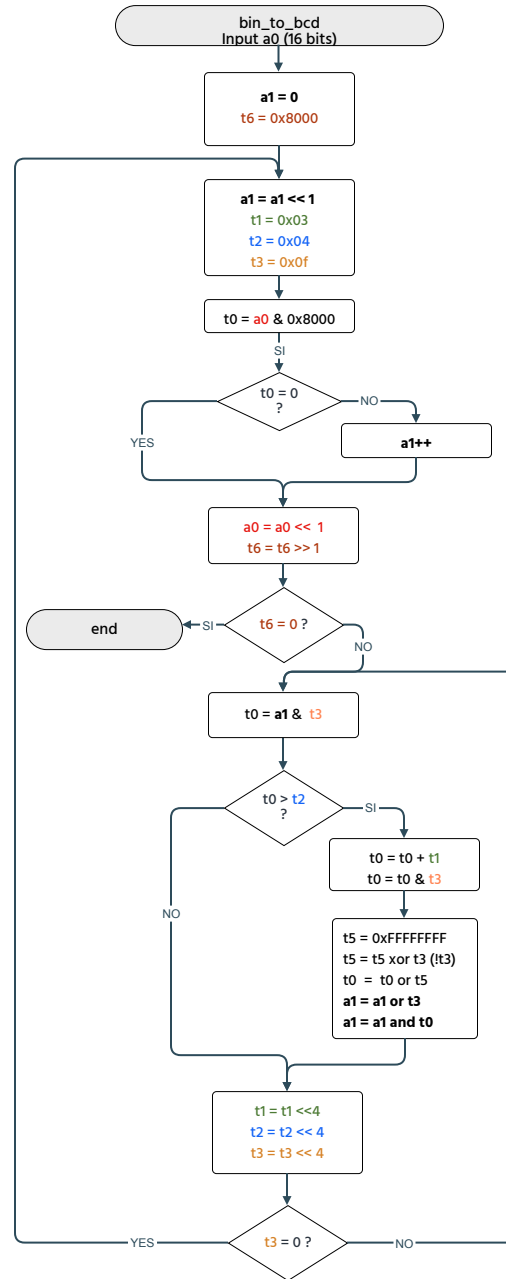


Figura 2.20 Diagrama del algoritmo double dabble

2.3 Memoria SPI_FLASH como memoria de programa

EL objetivo final de este estudio es la implementación en un ASIC del procesador *femtorv*, por esto, es necesario reducir el área del circuito y el número de pines del SoC; debido a esto, es necesario que la memoria de programa se encuentre en el exterior, pero sacar la memoria actual implica disponer de más de 64 pines para su conexión, por este motivo optamos por utilizar una memoria SPI_Flash, lo que reduce el número de pines a 4 (CS, CLK, MISO, MOSI). En la figura ?? se muestra el Diagrama de Bloques de esta modificación.

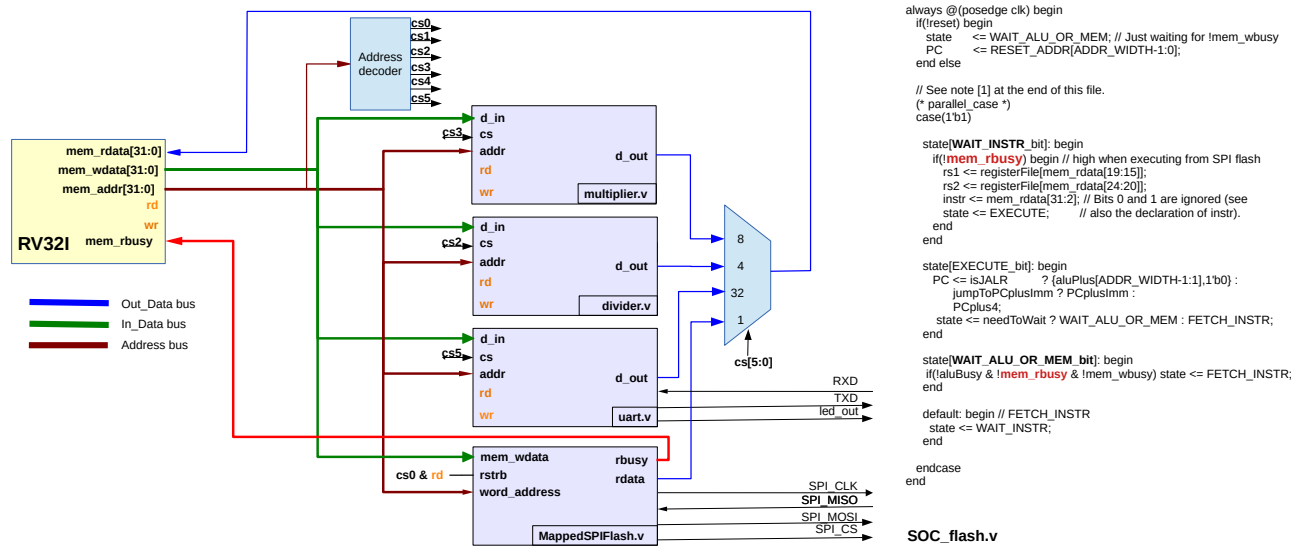


Figura 2.21 Diagrama de bloques del SoC utilizando memoria SPI Flash

Como puede verse en la figura ?? el procesador posee la señal *mem_rbusy* que le indica que la memoria no se encuentra lista para enviar el dato leído. En la figura ??

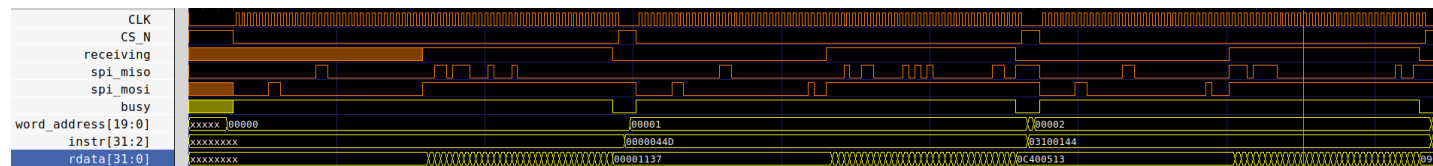
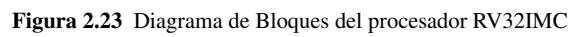


Figura 2.22 Formas de onda del acceso a la memoria SPI_Flash

2.4 Procesador RV32IMC

En la figura ?? se muestra el diagrama de bloques del procesador RV32IMC, este procesador soporta operaciones de multiplicación y división, instrucciones comprimidas e Interrupciones.



Capítulo 3

Ejemplos de Aplicaciones del VHDL.

3.1 INTRODUCCION

En los capítulos anteriores se describió el proceso de diseño para sistemas digitales y las herramientas disponibles (tanto software como hardware) para su implementación. En este capítulo se tratarán ejemplos prácticos de diseño de sistemas digitales, iniciando con su descripción hasta llegar a su implementación final utilizando en algunos casos componentes discretos y VHDL.

3.2 ¿QUE ES UNA FSM?

Una máquina de estados es un sistema secuencial sincrónico que posee un número fijo de posibles estados. El valor de sus salidas y la transición entre los estados depende del valor de sus entradas y del estado en que se encuentra actualmente. **Todos los cambios de estado ocurren ante un determinado flanco de la señal de reloj** (ya sea de subida o bajada).

Para entender mejor este concepto imaginemos que nuestra máquina de estados es un televisor, que sólo puede sintonizar cuatro canales y que se puede controlar por un control remoto que tiene solo dos teclas para aumentar o disminuir el canal. Los estados de nuestro sistema están representados por el número de canales que se pueden sintonizar, así pues, solo tendremos cuatro posibles estados (Número fijo de estados). En la Figura 1 se muestra un diagrama de nuestro sistema:



Figura 1. Ejemplo de FSM.

Ahora hagámonos una pregunta: ¿Qué nos produce un cambio en el estado de nuestro sistema?, Lo único que nos puede producir un cambio de estado (Canal sintonizado) es un cambio en las teclas de nuestro control remoto (Entradas del sistema). Observemos como actúa el sistema ante cambios de las entradas:

Si oprimimos la tecla de aumentar el canal, el televisor aumentará en uno la cadena sintonizada, por ejemplo, si estaba en el canal 1 pasará al 2, si estaba en el 2 al 3, del 3 al 4 y del 4 al 1 nuevamente.

Si oprimimos la tecla menos es canal disminuirá pasando del 4 al 3, del 3 al 2, del 2 al 1 y del 1 al 4 nuevamente.

Si no oprimimos una tecla, el televisor debe permanecer en la cadena sintonizada actualmente y quedarse ahí hasta que se le ordene un cambio por medio del control remoto.

Note que el estado (canal sintonizado) al que pasará el sistema depende del estado actual (canal actual) y de las entradas (tecla del control remoto oprimida). Si las entradas no cambian el sistema no cambia su posición (esto no es necesariamente cierto para todas las máquinas de estado).

3.3 TABLAS Y DIAGRAMAS DE ESTADO

Existen varias formas de representar el comportamiento de las máquinas de estado, siendo los más utilizados las tablas y los diagramas de estado. Tomemos nuevamente el ejemplo del televisor y representemos en una tabla los estados actual y siguiente (Estado al que pasa el sistema ante un cambio de las entradas).

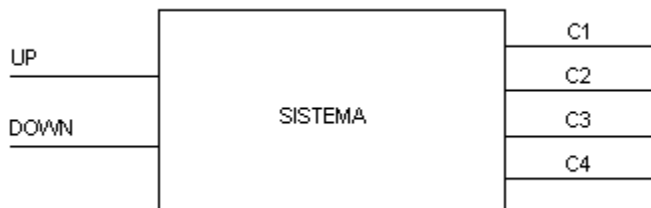


Figura 2. Entradas y salidas del Sistema.

La Figura 2 muestra el sistema como una caja negra en la que sólo se indican las entradas y las salidas. Supongamos que nuestro sistema tiene dos entradas que son las correspondientes a Adelantar (UP) y disminuir (DN) canal, y que tiene cuatro salidas C1, C2, C3, C4 que corresponden a los cuatro canales y que me indican cual canal se está sintonizando actualmente. En este punto debemos tomar varias decisiones:

¿Cuando se considera que una entrada esta activa?, Es decir, con que valor lógico de la entrada se produce un cambio y con cual no. En nuestro caso un valor lógico alto en las entradas producirá un cambio de estado, es decir, si UP = '1' el canal sintonizado aumentará o si DN = '1' el canal disminuirá. Otra decisión que debemos tomar y que se deriva de esta es: que sucede si por error las dos entradas son '1' simultáneamente, lo más conveniente es que el sistema conserve su estado ante esta posible situación.

El valor de las salidas para cada estado, en este ejemplo, un uno lógico en una salida indica que el canal ha sido seleccionado (por ejemplo, un uno en C1 indicara que actualmente el canal seleccionado es el 1), por lo tanto sólo una salida puede tener un valor lógico alto.

Una vez definido completamente el funcionamiento de nuestro sistema se procede a representarlo mediante una tabla de estados:

3.3.1 Tabla de Estados

Todas las condiciones posibles de las entradas

Estado Actual (S)	Estado Siguiente (S*)			
	UP = 0, DN=0	UP=0,DN=1	UP=1,DN=0	UP=1,DN=1
Canal 1	Canal 1	Canal 4	Canal 2	Canal 1
Canal 2	Canal 2	Canal 1	Canal 3	Canal 2
Canal 3	Canal 3	Canal 2	Canal 4	Canal 3
Canal 4	Canal 4	Canal 3	Canal 1	Canal 4

Figura 3. Tabla de estados del sistema Televisor.

La Figura 3. Muestra una tabla de estados típica en la cual se resume el comportamiento del sistema. La tabla tiene tres secciones:

El estado actual: Lista de todos lo posibles estados.

Posibles combinaciones de las entradas: El número de entradas del sistema determina el número de columnas de la tabla de estados. Así, si la máquina tiene n entradas, la tabla tendrá $2^n + 1$ Columnas.

El estado siguiente: Indica a que estado pasará la FSM cuando se presente una determinada entrada, Por ejemplo, Si UP=0 y DOWN = 1 y el estado actual es el canal 4 la máquina de estados irá al estado Canal 3.

Otra forma de representar el estado de las entradas es utilizando una convención en la que si la variable aparece negada entonces toma un valor de cero lógico, pero si no lo esta tiene un valor de uno lógico. Esto se hace para simplificar las tablas. Por ejemplo:

A: A = '1'

!A: A = 0;

Con lo que la tabla de estados se convierte en :

Estado Actual	Estado Siguiente				Salidas
	!UP,!DN	!UP,DN	UP,!DN	UP,DN	
Canal 1	Canal 1	Canal 4	Canal 2	Canal 1	C1,C2,C3,C4
Canal 2	Canal 2	Canal 1	Canal 3	Canal 2	
Canal 3	Canal 3	Canal 2	Canal 4	Canal 3	
Canal 4	Canal 4	Canal 3	Canal 1	Canal 4	

Tabla 1. Tabla de Estados del sistema Televisor.

3.3.2 Diagrama de Estados

Otra forma de representar el comportamiento de una máquina de estados es el diagrama de estados, este diagrama es una representación gráfica del funcionamiento del sistema.

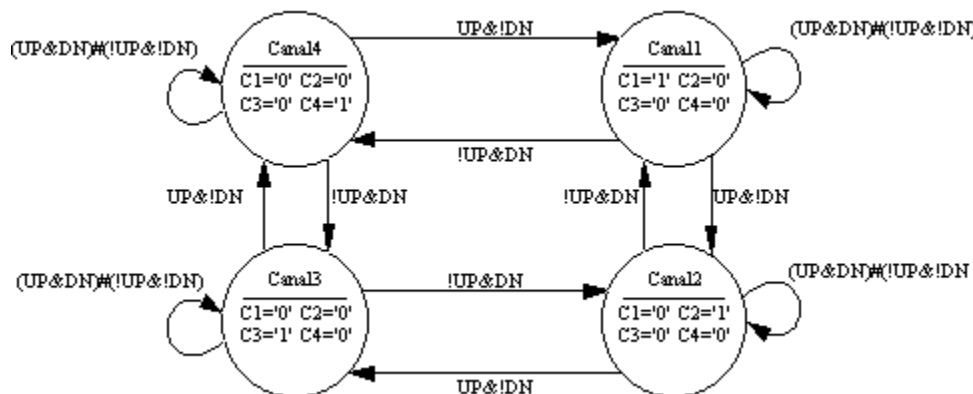


Figura 4. Diagrama de estados del sistema Televisor.

La Figura 4 muestra el diagrama de bloques del sistema televisor. Observamos que cada estado está representado por un círculo en el que se indica el nombre del mismo y el valor de las salidas. Las líneas que unen los estados representan el cambio que sufre la máquina cuando se cumple una determinada regla de transición. (La regla de transición normalmente se indican en las líneas que unen los estados). En esta figura se introduce una nueva nomenclatura para representar las funciones lógicas, el operador *not* se representa con el signo **!**, la operación **AND** con el signo **&** y la **OR** con **#**.

Analizando el estado Canal1, observamos que de este estado salen dos líneas:

Hacia el estado Canal2, esta línea indica que la máquina pasará de Canal1 a Canal2 si UP = '1' y DN = '0' y se presenta un flanco adecuado en la señal de reloj.

Hacia el estado Canal4: La máquina de estados pasará de Canal1 a Canal4 si UP = '0', DN = '1' y se presenta un flanco adecuado en la señal de reloj.

Entrad dos líneas:

Desde el estado Canal2: El sistema pasará de Canal2 a Canal 1 si UP = '0', DN = '1' y se presenta un flanco adecuado en la señal de reloj.

Desde el estado Canal4: El sistema pasará de Canal4 a Canal 1 si UP = '1' y DN = '0' y se presenta un flanco adecuado en la señal de reloj.

Además observamos que existe una línea que sale del Canal1 y vuelve a entrar a Canal1, esto indica que la máquina mantendrá su estado si: (UP = '0' Y DN = '0') O (UP = '1' Y DN = '1').

3.4 SINTESIS DE FSM

En esta sección analizaremos la arquitectura de la FSM y el proceso de síntesis. Como vimos en el capítulo anterior la síntesis parte de una descripción funcional y llega a una descripción a nivel de compuertas.

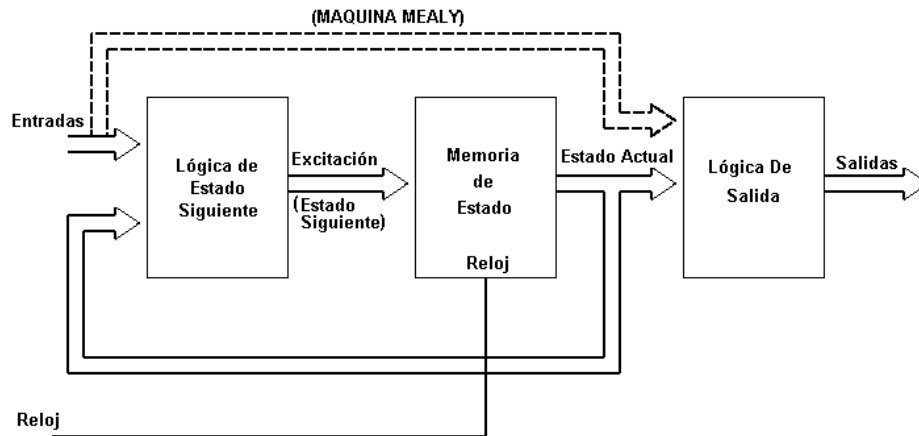


Figura 5. Estructura de una FSM.

3.4.1 Estructura de una FSM

La estructura general de una máquina de estados se muestra en la Figura 5. Como puede observarse existen tres bloques bien definidos:

Lógica de estado siguiente: Está encargada de generar las señales necesarias para producir un cambio de estado en la FSM (Estado Siguiente). Puede observarse que el estado siguiente depende del estado actual y de las entradas.

Memoria de Estado: Normalmente esta formada por Flip-Flops tipo D o JK, los cuales tienen la misma señal de reloj. Las salidas de los Flip-Flops determinan el estado actual de la FSM, cada salida del Flip-Flop puede tomar dos valores distintos '1' o '0', por lo tanto se pueden tener dos estados con un Flip-Flop. Si tenemos N Flip-Flops tendremos 2^N estados.

Lógica de Salida: La lógica de salida esta encargada de producir los valores necesarios a la salida de la FSM, su arquitectura esta basada en decodificadores.

Existen dos tipos de máquinas de estados:

Moore: El estado siguiente depende del valor de las entradas y del estado actual; Y la salida depende únicamente del estado actual.

Mealy: Al igual que la máquina de Moore el estado siguiente depende del valor de las entradas y del estado actual, pero se diferencian en que la salida depende del estado actual y del valor de las entradas.

3.4.2 Proceso de Síntesis

El primer paso en el proceso de síntesis de una FSM y en general de cualquier sistema digital es la descripción del sistema ya sea mediante un algoritmo o de un diagrama de tiempos. El siguiente paso consiste en pasar la descripción funcional a un diagrama de estados para su posterior representación en tablas de estado y de salida. A continuación debemos reducir el número de estados (si es posible) utilizando algoritmos de minimización. Después debemos realizar la codificación de estados, es decir, asignarle a los estados un grupo único de valores que corresponden a los valores que tomarán los Flip-Flops. A continuación se debemos obtener las ecuaciones de estado siguiente y de salidas. El siguiente paso consiste en la elección del tipo de Flip-Flop que se va a utilizar para la implementación, recuerde que todos los Flip-Flops tienen diferentes ecuaciones de estado siguiente:

Tipo de F-F	Estado Siguiente (Q^*)
D	$Q^* = D$
JK	$Q^* = J \cdot !Q + !K \cdot Q$
T	$Q^* = !Q$
SR	$Q^* = S + !R \cdot Q$

Tabla 2. Ecuaciones de Estado siguiente de los Flip-Flop.

Una vez elegido el Flip-Flop se procede a obtener las ecuaciones de las señales de excitación de los FFs. Después se debe realizar un proceso de minimización con las ecuaciones obtenidas anteriormente para realizar un diagrama de la implementación lógica de las ecuaciones.

Finalmente debemos verificar el correcto funcionamiento del circuito, esto se logra simulando el circuito obtenido y si cumple con lo requerimientos se llevará al plano físico para realizar pruebas de tiempos. La figura 6 resume los pasos a seguir en el proceso de síntesis.

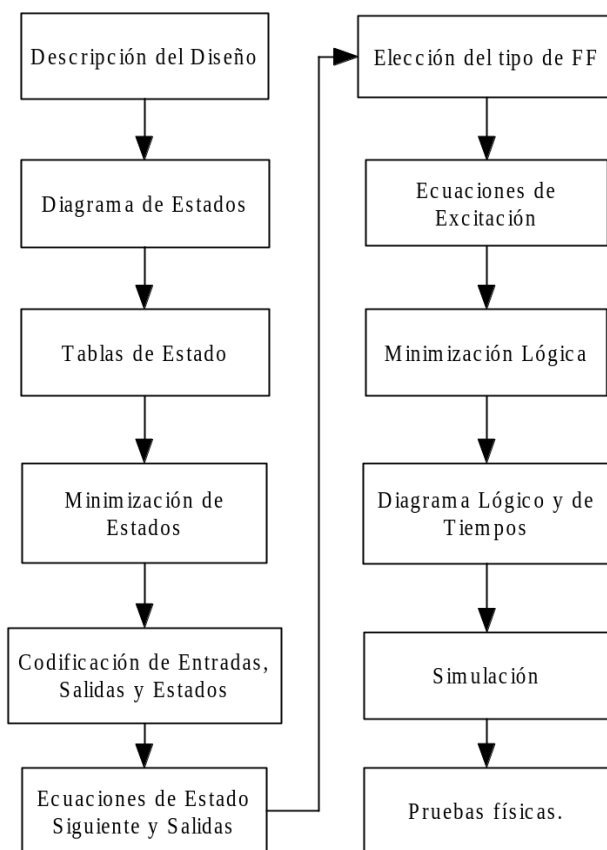


Figura 6. Diagrama de flujo del proceso de Síntesis para FSM

Para entender mejor el proceso de síntesis realizaremos paso a paso un sencillo ejemplo. Desarrollando todas las posibilidades de implementación para buscar la más óptima.

3.4.3 Control de Motor de Paso.

Un motor de paso a diferencia de los motores de Corriente Continua necesita una secuencia determinada en sus cuatro terminales para originar el giro de su rotor. La secuencia necesaria para controlar el motor es la siguiente:

	A	B	C	D
S1	V+	GND	V+	GND
S2	V+	GND	GND	V+
S3	GND	V+	GND	V+
S4	GND	V+	V+	GND

Para que el motor gire un paso (normalmente 1.8 grados) es necesario que se aplique S1 y luego S2, o S2 y S3 o S3 y S4 o S4 y S1. Si se desea que el motor gire cinco pasos se debe seguir la secuencia S1, S2, S3, S4, S5. La inversión del sentido de giro se logra invirtiendo la secuencia anterior, es decir, S1, S4, S3, S2, S1.

Diagrama de Caja Negra

El primer paso consiste en realizar un diagrama de caja negra en el que se indiquen las entradas y salidas (Figura 7).



Figura 7. Diagrama de Caja Negra del Controlador de Motor de Paso.

A continuación realizaremos una breve descripción del funcionamiento del circuito. Como se observa en la Figura 7. Se tienen tres entradas:

DIR: Encargada de indicar la dirección de giro del motor. DIR = 0 Secuencia Directa (S1, S2, S3, S4), DIR = 1 Secuencia Invertida (S4, S3, S2, S1)

EN: ENABLE, encargada de habilitar nuestro control Si EN = 1 el circuito realizará su función si EN = 0 el control conservará el último estado de las salidas.

CLOCK: Es el reloj del sistema y gobierna todas las transiciones entre estados.

Y las cuatro salidas A, B, C, D son las encargadas de manejar los terminales del motor de paso.

Diagrama de Estado

Una vez se conoce el funcionamiento del circuito, o las funciones que debe cumplir se procede a realizar el diagrama de estados del mismo. La Figura 8 muestra este diagrama para el controlador de motor de paso.

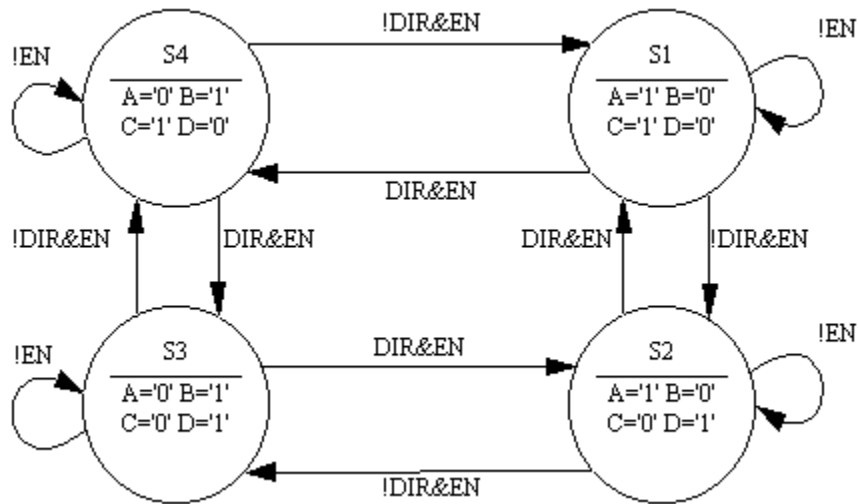


Figura 8 Diagrama de Estados del Controlador de Motor de Paso.

Tabla de estados

Una vez realizado el diagrama de estados se procede a la realización de la tabla de estados y salidas. Dicha tabla para el controlador de motor de paso se presente a continuación:

Estado Actual	Estado Siguiente				Salidas
	!EN.!DIR	!EN.DIR	EN.!DIR	EN.DIR	
S1	S1	S1	S2	S4	1, 0, 1, 0
S2	S2	S2	S3	S1	1, 0, 0, 1
S3	S3	S3	S4	S2	0, 1, 0, 1
S4	S4	S4	S1	S3	0, 1, 1, 0

Minimización de Estados

El objetivo de la minimización de estados es la reducción del número de Flip-Flops necesarios para la implementación, reduciendo de este modo el costo de la misma. Sin embargo para reducir el número de Flip-Flops de un circuito es necesario reducir el número de estados en múltiplos de 2. Por ejemplo, supongamos que tenemos 7 estados, para lo cual necesitamos 3 FFs, para utilizar sólo 2 FFs necesitamos reducir el número de estados de 7 a 4 o menos.

La minimización de estados se basa en la equivalencia funcional, por ejemplo, se dice que dos circuitos son equivalentes cuando proporcionan la misma salida ante los mismos cambios de entrada. Dos o más estados son equivalentes sí:

Ambos estados producen las mismas salidas ante igual cambio en las señales de entrada.

Ambos estados tienen los mismos estados siguientes ante los mismos cambios de las señales de entrada.

En nuestro caso no tenemos equivalentes ya que todos tienen diferentes valores de salida y diferentes estados siguientes ante variaciones de las entradas.

Codificación de estados

La codificación de estados consiste en la asignación de valores a las salidas de los FFs para cada estado, estos valores deben ser únicos para cada estado, es decir, no se deben repetir. Debido a que nuestra máquina tiene cuatro estados, tenemos 2 FFs y por lo tanto debemos asignar a cada estado un código formado por dos bits. Existen muchas posibles

codificaciones para los cuatro estados, unas más eficientes que otras. En este libro no se tratarán las técnicas existentes para hallar la codificación óptima ya que esta función las realizan las herramientas CAD y además, se sale del objetivo de este libro.

Para nuestro ejemplo utilizaremos la siguiente codificación:

	Q1	Q0
S1	0	0
S2	0	1
S3	1	0
S4	1	1

Donde Q1 y Q0 son las salidas del primer y segundo FF correspondientemente. Con esta asignación de estados la tabla de estados queda de la siguiente forma:

		Estado Siguiente (S*)				SALIDAS		
Estado Actual (S)		!EN		EN		DIR		
				!DIR		DIR		
Q1	Q0	Q1*	Q0*	Q1*	Q0*	Q1*	Q0*	A, B, C, D
0	0	0	0	0	1	1	1	1, 0, 1, 0
0	1	0	1	1	0	0	0	1, 0, 0, 1
1	0	1	0	1	1	0	1	0, 1, 0, 1
1	1	1	1	0	0	1	0	0, 1, 1, 0

Donde Q1* y Q0* representan los valores siguientes de las señales Q1 y Q0 respectivamente. Note que no se representaron los casos !EN.!DIR y !EN.DIR, esto se debe a que cuando la señal En tiene un valor lógico bajo la FSM conserva su estado sin importar el valor de DIR.

Ecuaciones de estado siguiente

Una vez realizada la codificación de estados se procede a la obtención de las ecuaciones de estado siguiente Q1* y Q0*. Para obtener estas ecuaciones debemos sumar todos los unos de la tabla de estados (suma de minterminos). Para Q1* debemos observar todas las columnas correspondientes a Q1* y sumar los minterminos asociados. Por ejemplo, la primera columna de Q1* correspondiente a la asociada con !EN, el primer mintermino asociado es: !EN.Q1.!Q0 ya que está en la fila correspondiente a Q1 = 1 y Q0 = 0. Y el segundo !EN.Q1.Q0.

$$Q1^* = !EN.Q1.!Q0 + !EN.Q1.Q0 + EN.!DIR.!Q1.Q0 + EN.!DIR.Q1.!Q0 + EN.DIR.!Q1.!Q0 + EN.DIR.Q1.Q0.$$

$$Q0^* = !EN.!Q1.Q0 + !EN.Q1.Q0 + EN.!DIR.!Q1.!Q0 + EN.!DIR.Q1.!Q0 + EN.DIR.!Q1.!Q0 + EN.DIR.Q1.Q0.$$

Estas ecuaciones deben pasar a través de un proceso de minimización para encontrar una implementación óptima.

$$Q1^* = !EN.Q1(!Q0+Q1) + EN.(!DIR.(!Q1.Q0 + Q1.!Q0) + DIR (!Q1.!Q0 + Q1.Q0))$$

$$Q1^* = !EN.Q1 + EN.(!DIR.Q1.Q0 + DIR.(Q1.Q0))$$

$$Q1^* = !EN.Q1 + EN.DIR (Q1 \text{ XOR } Q0)$$

$$Q0^* = !EN.Q0.(!Q1 + Q1) + EN.(!Q1.Q0.(!DIR + DIR) + Q1.Q0.(!DIR + DIR))$$

$$Q0^* = !EN.Q0 + EN.(!Q1.Q0 + Q1.Q0)$$

$$Q0^* = !EN.Q0 + EN.(Q0.(Q1+ Q1))$$

$$Q0^* = !EN.Q0 + EN.Q0$$

$$Q0^* = EN \text{ XOR } Q0$$

Las ecuaciones para las salidas son:

$$A = !Q1.!Q0 + !Q1.Q0 = !Q1$$

$$B = Q1.!Q0 + Q1.Q0 = Q1$$

$$C = !Q1.Q0 + Q1.Q0 = (Q1 \text{ XOR } Q0)$$

$$D = !Q1.Q0 + Q1.Q0 = Q1 \text{ XOR } Q0$$

$$A = !B = !Q1$$

$$C = !D = Q1 \text{ XOR } Q0$$

Elección del tipo de Flip-Flop, ecuaciones de excitación y minimización

Utilizando las ecuaciones obtenidas anteriormente para $Q1^*$ y $Q0^*$, debemos escoger el tipo de Flip-Flop a utilizar. La siguiente tabla resume los valores necesarios en las entradas de los FFs para obtener los cambios indicados en sus salidas. Por ejemplo en un FF JK si Q tiene un valor de 0 lógico y se quiere que pase a tener un valor de 1 lógico es necesario que $J = 1$ y el valor de K no importa.

Q Q*	S R	J K	T	D
0 0	0 X	0 X	0	0
0 1	1 0	1 X	1	1
1 0	0 1	X 1	1	0
1 1	X 0	X 0	0	1

El diagrama de Karnough para la máquina de estados es el siguiente

	00	01	11	10
00	0 0	0 0	1 1	0 1
01	0 1	0 1	0 0	1 0
11	1 1	1 1	1 0	0 0
10	1 0	1 0	0 1	1 1

La región sombreada corresponde a los valores de las señales $Q1^*$ (en negrilla) y $Q0^*$.

Para el FF D tenemos: $Q^* = D$. Por lo tanto:

$$D1 = !EN.Q1 + EN.DIR (Q1 \text{ XOR } Q0)$$

$$D0 = EN \text{ XOR } Q0$$

Para el FF JK debemos ordenar las ecuaciones obtenidas de la forma: $Q^* = J.!Q + !K.Q$

Para $Q1$:

	00	01	11	10
00	0 X	0 X	1 X	0 X
01	0 X	0 X	0 X	1 X
11	X 0	X 0	X 0	X 1
10	X 0	X 0	X 1	X 0

La región sombreada indican los valores que deben tener las señales $J1$ (en negrilla) y $K1$.

$$J1 = !Q1.!Q0.EN.DIR + !Q1.Q0.EN.!DIR = EN.!Q1(Q0 \text{ XOR } DIR)$$

$$K1 = Q1.!Q0.EN.DIR + Q1.Q0.EN.!DIR = EN.Q1.(Q0 \text{ XOR } DIR)$$

Para $Q0$

	00	01	11	10
00	0 X	0 X	1 X	1 X
01	X 0	X 0	X 1	X 1
11	X 0	X 0	X 1	X 1
10	0 X	0 X	1 X	1 X

$$J0 = EN$$

$$K0 = EN$$

Flip-Flop tipo T:

Para $Q1$:

	00	01	11	10
00	0	0	1	0

01	0	0	0	1
11	0	0	0	1
10	0	0	1	0

$$T1 = !Q0.EN.DIR + Q0.EN.!DIR = EN.(Q0 \text{ XOR } DIR)$$

Para Q0:

	00	01	11	10
00	0	0	1	1
01	0	0	1	1
11	0	0	1	1
10	0	0	1	1

$$T0 = EN$$

Flip-Flop tipo SR.

Para Q1:

	00	01	11	10
00	0 X	0 X	1 0	0 X
01	0 X	0 X	0 X	1 0
11	X 0	X 0	X 0	0 1
10	X 0	X 0	0 1	X 0

$$S1 = !Q1.!Q0.EN.DIR + !Q1.Q0.EN.!DIR = EN.!Q1(Q0 \text{ XOR } DIR)$$

$$R1 = Q1.!Q0.EN.DIR + Q1.Q0.EN.!DIR = EN.Q1.(Q0 \text{ XOR } DIR)$$

Para Q0:

	00	01	11	10
00	0 X	0 X	1 0	1 0
01	X 0	X 0	0 1	0 1
11	X 0	X 0	0 1	0 1
10	0 X	0 X	1 0	1 0

$$S0 = EN.!Q0$$

$$R0 = EN.Q0$$

Del análisis anterior observamos que la implementación que ocupa el menor número de compuertas es la correspondiente a los Flip-Flops tipo T.

SIMULACION

En la Figura 9 se muestra la implementación final de la máquina de Estados (Sin la Unidad de Salida) y en la Figura 10 la simulación correspondiente.

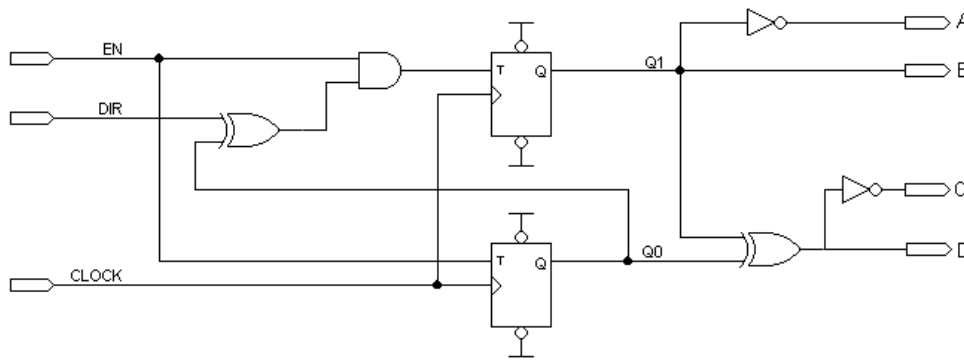


Figura 9. Diagrama a nivel de compuertas del controlador de Motor de Paso.

Como puede observarse en la Figura 9, la máquina de estados está formada por la lógica de estado siguiente (Compuertas XOR y AND), la memoria de estado (FFs tipo T con la misma señal de Reloj) y la lógica de salida, (Compuertas NOT y XOR) que en este caso corresponde al de una máquina de estados tipo MOORE.

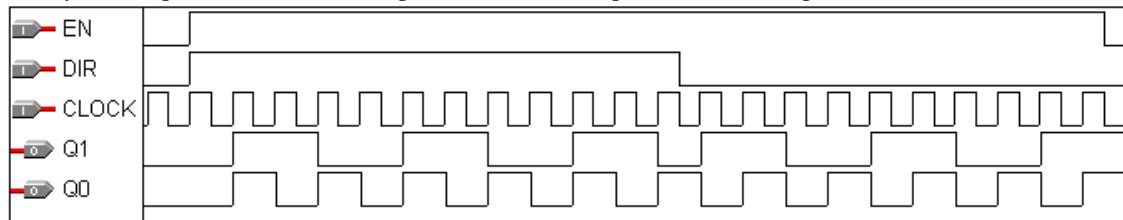


Figura 10. Simulación del Controlador de Motor de Paso.

3.5 DESCRIPCION VHDL DE UNA FSM

Con la descripción funcional no es necesario preocuparse por el tipo de Flip-Flop ni por los procesos de minimización. Normalmente las herramientas CAD proporcionan información sobre resultado de la síntesis del diseño, las ecuaciones implementadas en un CPLD por una de estas herramientas es:

```
clk : INPUT;
DIR : INPUT;
EN : INPUT;
A = DFFE( state~2 $ VCC, GLOBAL( clk), VCC, VCC, VCC);
B = DFFE(!state~2 $ VCC, GLOBAL( clk), VCC, VCC, VCC);
C = DFFE(!state~1 $ state~2, GLOBAL( clk), VCC, VCC, VCC);
D = DFFE( state~1 $ state~2, GLOBAL( clk), VCC, VCC, VCC);
state~1 = TFFE( EN, GLOBAL( clk), VCC, VCC, VCC);
state~2 = TFFE( _EQ001, GLOBAL( clk), VCC, VCC, VCC);
_EQ001 = !DIR & EN & state~1 # DIR & EN & !state~1;
```

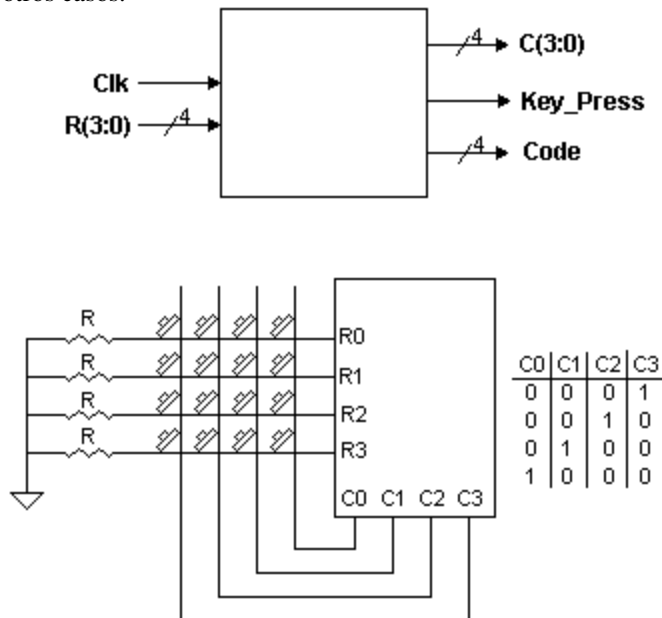
En donde se puede observar que se llegó a las mismas ecuaciones obtenidas anteriormente. Observe que se utilizaron 2 Flip-Flops tipo T uno de los cuales tiene a la señal EN como entrada (primera señal que aparece dentro de los paréntesis después de TFFE.). Mientras que el otro tiene a la señal _EQ001 conectada a su entrada T. La ecuación para _EQ001 es:

$$!DIR \& EN \& state\sim1 \# DIR \& EN \& !state\sim1 = EN, ((!DIR.sate\sim1) + (DIR.!state\sim1))$$

Como vimos en capítulos anteriores la arquitectura de un CPLD no posee compuertas XOR, por lo tanto las ecuaciones son de la forma de suma de productos. Lo interesante es observar el ahorro de tiempo asociado a la utilización del VHDL como herramienta de diseño.

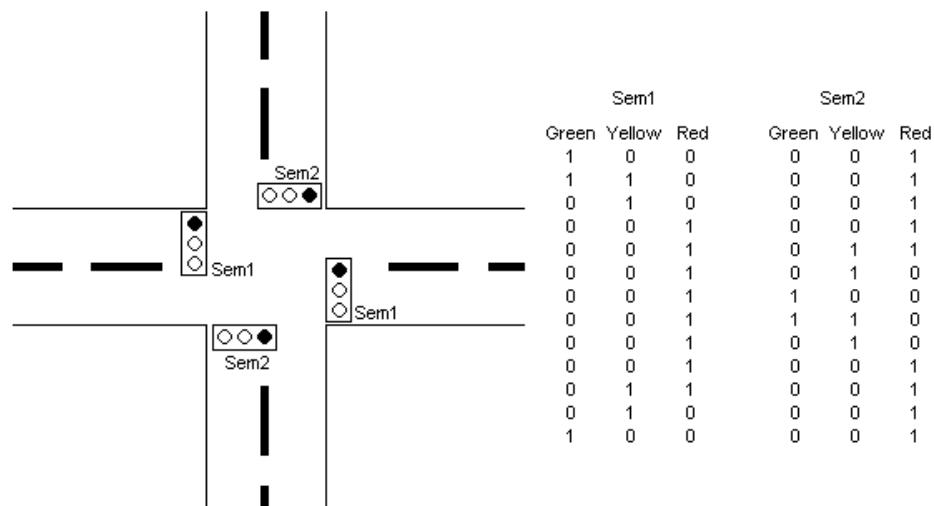
3. Diseñar el decodificador del teclado matricial que se muestra en la siguiente figura. Cuando no se tiene ninguna tecla presionada, en las entradas R0 a R3 se tiene una lectura de "0000". Si por ejemplo se presiona la tecla superior

izquierda, se recibirá un '1' lógico en R0 cuando la secuencia de las columnas C0 a C3 sea "1000". Por lo tanto un valor lógico alto en las entradas R0 a R3 indica la fila de la tecla oprimida, pero no sabemos cual de las cuatro teclas de la fila está presionada. Para averiguarlo debemos colocar un '1' lógico en solo una de las columnas y rotarlo por todas ellas. R0 será igual a '1' sólo cuando se coloque el '1' lógico en la columna de la tecla oprimida y '0' en los otros casos.

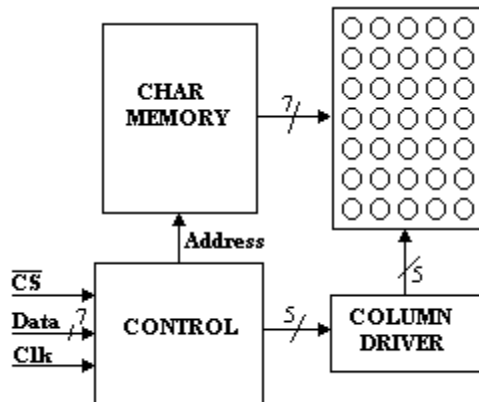


Una vez detectada la tecla oprimida el decodificador debe generar un código único para cada una de ellas y debe indicar el evento haciendo *key_press* = '1'.

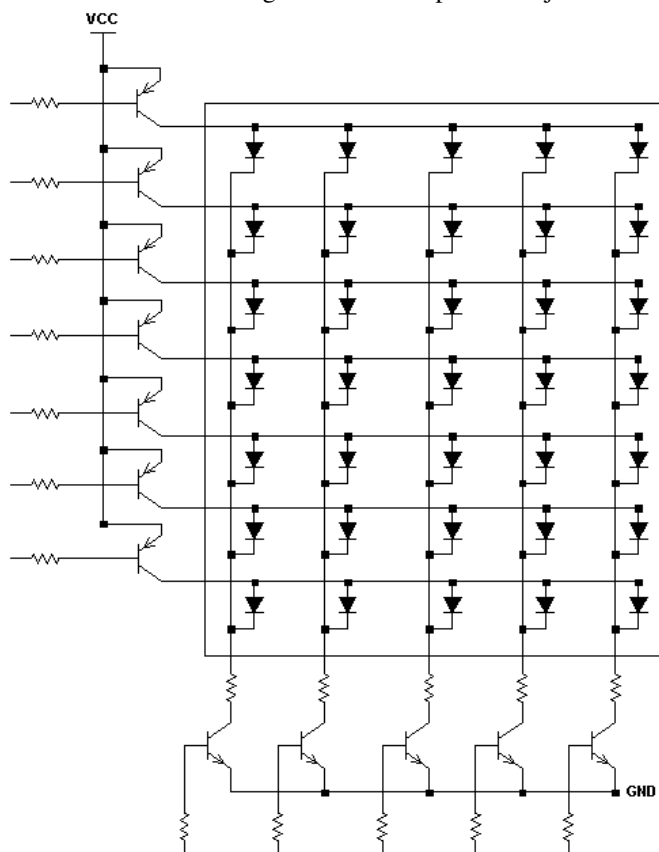
4. Diseñar el sistema de control de los semáforos de un cruce de vehículos.



6. Diseñar el control de un display inteligente. Este display tiene como elemento de visualización una matriz de leds de 7x5. Se desea visualizar los caracteres ASCII desde el 20H ' ' hasta el 7EH '~'. La siguiente figura muestra el diagrama de bloques propuesto.



Debido a que los dispositivos lógicos programables no pueden suministrar la corriente necesaria para manejar los leds, es necesario utilizar el siguiente circuito para manejar la matriz:



El contenido de la memoria debe ser el siguiente:

```
' '000h,000h,000h,000h,000h
'!'000h,000h,0FAh,000h,000h
'''000h,0E0h,000h,0E0h,000h
'#'044h,0FEh,044h,0FEh,044h
'$'048h,054h,0FEh,054h,024h
'%'046h,026h,010h,0C8h,0C4h
'&'00Ah,044h,0AAh,092h,06Ch
'''000h,0C0h,0A0h,000h,000h
'('000h,082h,044h,038h,000h
')'000h,038h,044h,082h,000h
```

```

'*'028h,010h,07Ch,010h,028h
+'010h,010h,07Ch,010h,010h
','000h,00Ch,00Ah,000h,000h
-'010h,010h,010h,010h,010h
.'000h,006h,006h,000h,000h
/'040h,020h,010h,008h,004h
'0'07Ch,0A2h,092h,08Ah,07Ch
'1'000h,002h,0FEh,042h,000h
'2'062h,092h,08Ah,086h,042h
'3'05Ch,0A2h,0A2h,082h,044h
'4'008h,0FEh,048h,028h,018h
'5'09Ch,0A2h,0A2h,0A2h,0E4h
'6'04Ch,092h,092h,092h,07Ch
'7'0C0h,0A0h,090h,08Eh,080h
'8'06Ch,092h,092h,092h,06Ch
'9'07Ch,092h,092h,092h,064h
':'000h,06Ch,06Ch,000h,000h
';'000h,06Ch,06Ah,000h,000h
'<'082h,044h,028h,010h,000h
'='014h,014h,014h,014h,014h
'>'010h,028h,044h,082h,000h
'?060h,090h,08Ah,080h,040h
'@'07Ch,082h,09Eh,092h,04Ch
'A'03Eh,048h,088h,048h,03Eh
'B'06Ch,092h,092h,092h,0FEh
'C'044h,082h,082h,082h,07Ch
'D'07Ch,082h,082h,082h,0FEh
'E'082h,082h,092h,092h,0FEh
'F'080h,090h,090h,090h,0FEh
'G'05Eh,092h,092h,082h,07Ch
'H'0FEh,010h,010h,010h,0FEh
'I'000h,082h,0FEh,082h,000h
'J'080h,0FCh,082h,002h,004h
'K'082h,044h,028h,010h,0FEh
'L'002h,002h,002h,002h,0FEh
'M'0FEh,040h,030h,040h,0FEh
'N'0FEh,008h,010h,020h,0FEh
'O'07Ch,082h,082h,082h,07Ch
'P'060h,090h,090h,090h,0FEh
'Q'07Ah,084h,08Ah,082h,07Ch
'R'062h,094h,098h,090h,0FEh
'S'04Ch,092h,092h,092h,064h
'T'080h,080h,0FEh,080h,080h
'U'0FCh,002h,002h,002h,0FCh
'V'0F8h,004h,002h,004h,0F8h
'W'0FCh,002h,01Ch,002h,0FCh
'X'0C6h,028h,010h,028h,0C6h
'Y'0E0h,010h,00Eh,010h,0E0h
'Z'0C2h,0A2h,092h,08Ah,086h
 '['000h,082h,082h,0FEh,000h
 '\004h,008h,010h,020h,040h
 ']'000h,0FEh,082h,082h,000h
 '^020h,040h,080h,040h,020h
 '_002h,002h,002h,002h,002h

```



```

'~'000h,020h,040h,080h,000h
'a'01Eh,02Ah,02Ah,02Ah,004h
'b'01Ch,022h,022h,012h,0FEh
'c'004h,022h,022h,022h,01Ch
'd'0FEh,012h,022h,022h,01Ch
'e'018h,02Ah,02Ah,02Ah,01Ch
'f'040h,080h,090h,07Eh,010h
'g'07Ch,04Ah,04Ah,04Ah,030h
'h'01Eh,020h,020h,010h,0FEh
'i'000h,002h,0BEh,022h,000h
'j'000h,0BCh,022h,002h,004h
'k'000h,022h,014h,008h,0FEh
'l'000h,002h,0FEh,082h,000h
'm'01Eh,020h,018h,020h,03Eh
'n'01Eh,020h,020h,010h,03Eh
'o'01Ch,022h,022h,022h,01Ch
'p'020h,050h,050h,050h,07Eh
'q'07Eh,030h,050h,050h,020h
'r'010h,020h,020h,010h,03Eh
's'004h,02Ah,02Ah,02Ah,012h
't'004h,002h,012h,07Ch,010h
'u'03Eh,004h,002h,002h,03Ch
'v'038h,004h,002h,004h,038h
'w'03Ch,002h,00Ch,002h,03Ch
'x'022h,014h,008h,014h,022h
'y'03Ch,00Ah,00Ah,00Ah,030h
'z'022h,032h,02Ah,026h,022h
'{'082h,082h,06Ch,010h,000h
'|'000h,000h,0FEh,000h,000h
'}'000h,010h,06Ch,082h,082h
'~'080h,040h,0C0h,080h,040h

```

7. Se desea diseñar un display formado por 5 matrices de Leds de 7x5 que sea capaz de almacenar mensajes de 70 caracteres. El mensaje a desplegar debe ser introducido via serial o por medio de un teclado.
8. Diseñar un circuito que permita detectar la tecla oprimida en un teclado:

Cada vez que se oprime una tecla en in teclado IBM el teclado envía un código, este código es único para cada tecla. Por ejemplo si se oprime la tecla 'A' se enviará el código 1CH. Si se mantiene oprimida esta tecla el código se envía continuamente hasta que se suelte o se oprima otra tecla. Cuando se suelta una tecla el teclado envía el código FOH para indicar que se liberó una tecla y a continuación envía el código de la tecla liberada en nuestro ejemplo cuando se libera la tecla 'A' el teclado envía los códigos FOH 1CH.

El teclado siempre envía el mismo código para cada letra, por lo tanto se debe tener en cuenta que cada vez que se oprima la tecla 'SHIFT' la siguiente letra debe cambiarse de mayúsculas a minúsculas o viceversa. También se debe tener en cuenta el estado de la tecla 'CAPS'.

Host Commands

ED	Set Status LED's - This command can be used to turn on and off the Num Lock, Caps Lock & Scroll Lock LED's. After Sending ED, keyboard will reply with ACK (FA) and wait for another byte which determines their Status. Bit 0 controls the Scroll Lock, Bit 1 the Num Lock and Bit 2 the Caps lock. Bits 3 to 7 are ignored.
EE	Echo - Upon sending a Echo command to the Keyboard, the keyboard should reply with a Echo (EE)

F0	Set Scan Code Set. Upon Sending F0, keyboard will reply with ACK (FA) and wait for another byte, 01-03 which determines the Scan Code Used. Sending 00 as the second byte will return the Scan Code Set currently in Use
F3	Set Typematic Repeat Rate. Keyboard will Acknowledge command with FA and wait for second byte, which determines the Typematic Repeat Rate.
F4	Keyboard Enable - Clears the keyboards output buffer, enables Keyboard Scanning and returns an Acknowledgment.
F5	Keyboard Disable - Resets the keyboard, disables Keyboard Scanning and returns an Acknowledgment.
FE	Resend - Upon receipt of the resend command the keyboard will re- transmit the last byte sent.
FF	Reset - Resets the Keyboard.

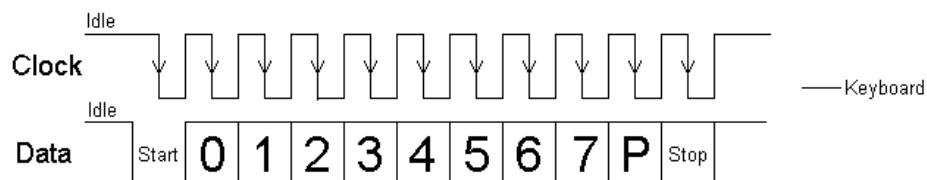
Commands

Now if the Host Commands are send from the host to the keyboard, then the keyboard commands must be sent from the keyboard to host. If you think this way, you must be correct. Below details some of the commands which the keyboard can send.

FA	Acknowledge
AA	Power On Self Test Passed (BAT Completed)
EE	See Echo Command (Host Commands)
FE	Resend - Upon receipt of the resend command the Host should re-transmit the last byte sent.
00	Error or Buffer Overflow
FF	Error or Buffer Overflow

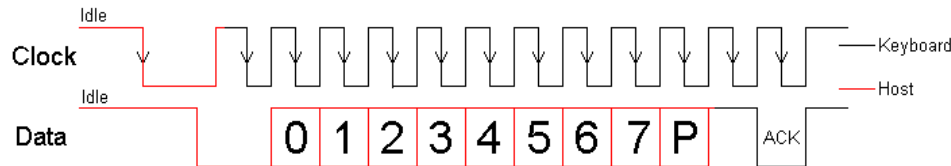
Sca

El teclado riene una interfaz PS2 la cual cuenta con la disposición de pines y el diagrama de tiempos indicados en la siguiente figura:



La figura anterior muestra la comunicación entre el teclado y el Host. Como puede verse se trata de una comunicaci3n serial sincr3nica en la cual el teclado genera las se1aales Data y Clock. Solo durante el flanco de bajado se debe leer el estado de la se1aal de datos.

La comunicaci3n entre el Host y el teclado se muestra en la siguiente figura. Como puede verse el Host debe iniciar la transmisi3n generando un flanco de bajada en la se1aal clock y un tiempo despu3s generar el Bit de Start, despu3s de esto el teclado genera la se1aal de reloj y el Host debe enviar serialmente el comando deseado, si el teclado recibe correctamente la trama responde con ACK.



El conector PS2 tiene 6 pines los cuales tiene las siguientes funciones:



PS/2

1. KBD Clock
2. GND
3. KBD Data
4. N/C
5. +5V (VCC)
6. N/C

Se desea que el driver de teclado tenga una salida ASCII, para poder ser utilizado con el Display inteligente del ejercicio 6. La siguiente tabla da una relación entre el código de la tecla y el código ASCII de la letra o carácter correspondiente.

NC			F9		NC		F5		F3		F1		F2		F12		NC		F10		F8		F6		F4		Tab				NC		
00	FF		01	FF	02	FF	03	FF	04	FF	05	FF	06	FF	07	FF	08	FF	09	FF	0A	FF	0B	FF	0C	FF	0D	09	0E	7C	0F		
NC		Alt			SHL		NC		CTL		q		l		NC		NC		NC		z		a			w		2				NC	
10	FF	11	FF	12	FF	13	FF	14	FF	15	71	16	31	17	FF	18	FF	19	FF	1A	7A	1B	73	1C	61	1D	77	1E	32	1F			
NC		c		x		d		e		4		3		NC		NC				v		f		t		r		5				NC	
20		21		22		23		24		25		26		27		28		29		2A		2B		2C		2D		2E		2F			
NC		n		b		h		g		y		6		NC		NC		NC		m		j		u		7		8				NC	
30		31		32		33		34		35		36		37		38		39		3A		3B		3C		3D		3E		3F			
NC				k		i		o		0		9		NC		NC				-		l		n		p		'				NC	
40		41		42		43		44		45		46		47		48		49		4A		4B		4C		4D		4E		4F			
NC		NC		£		NC		7		£		NC		NC		CAP		SHR		ENT		+				NC		NC				NC	
50		51		52		53		54		55		56		57		58		59		5A		5B		5C		5D		5E		5F			
NC		<		NC		NC		NC		NC		BKS		NC		NC		1		NC		4		7		NC		NC				NC	
60		61		62		63		64		65		66		67		68		69		6A		6B		6C		6D		6E		6F			
70		.		2		5		6		8		ESC		NUM		F11		+		3		-		*		9		BD				NC	
0		71		72		73		74		75		76		77		78		79		7A		7B		7C		7D		7E		7F			

;SHIFT + KEY

```
;00 NC 01 F9 02 NC 03 F5 04 F3 05 F1 06 F2 07 F12 08 NC 09 F10 0A F8 0B F6 0C F4 0D TAB 0E ° 0F NC
db 0FFH, 04300H, 0FFH, 03F00H, 03D00H, 03B00H, 03C00H, 08600H, 0FFH, 04400H, 04200H, 04000H, 03E00H,
009H, 0A7H, 0FFH
```

```
;10 NC 11 ALT 12 SHL 13 NC 14 CTL 15 Q 16 ! 17 NC 18 NC 19 NC 1A Z 1B S 1C A 1D W 1E @ 1F NC
db 0FFH, 0FFH, 0FFH, 0FFH, 0FFH, 051H, 021H, 0FFH, 0FFH, 0FFH, 05AH, 053H, 041H, 057H, 040H, 0FFH
```

```
;20 NC 21 C 22 X 23 D 24 E 25 $ 26 # 27 NC 28 NC 29 SPC 2A V 2B F 2C T 2D R 2E % 2F NC
db 0FFH, 043H, 058H, 044H, 045H, 024H, 023H, 0FFH, 0FFH, 0FFH, 056H, 046H, 054H, 052H, 025H, 0FFH
```

```
;30 NC 31 N 32 B 33 H 34 G 35 Y 36 & 37 NC 38 NC 39 NC 3A M 3B J 3C U 3D / 3E ( 3F NC
db 0FFH, 0FFH, 042H, 048H, 047H, 059H, 026H, 0FFH, 0FFH, 0FFH, 04DH, 04AH, 055H, 02FH, 028H, 0FFH
```

```
;40 NC 41 ; 42 K 43 I 44 O 45 = 46 ) 47 NC 48 NC 49 : 4A _ 4B L 4C Ñ 4D P 4E ? 4F NC
db 0FFH, 03BH, 04BH, 049H, 04FH, 03DH, 029H, 0FFH, 0FFH, 03AH, 05FH, 04CH, 0A5H, 050H, 03FH, 0FFH
```

```
;50 NC 51 NC 52 [ 53 NC 54 ` 55 ; 56 NC 57 NC 58 CAP 59 SHR 5A ENT 5B * 5C NC 5D ] 5E NC 5F NC
db 0FFH, 0FFH, 05BH, 0FFH, 0B0H, 0ADH, 0FFH, 0FFH, 0FFH, 0FFH, 013H, 02AH, 0FFH, 05DH, 0FFH, 0FFH
```

```
;60 NC 61 > 62 NC 63 NC 64 NC 65 NC 66 BKS 67 NC 68 NC 69 END 6A NC 6B <- 6C HM 6D NC 6E NC 6F
NC
```

```
db 0FFH, 03EH, 0FFH, 0FFH, 0FFH, 0FFH, 08H, 0FFH, 0FFH, 0FFH, 0FFH, 0FFH, 0FFH, 0FFH, 0FFH, 0FFH
;70 0 71 DEL 72 AD 73 5 74 -> 75 AUP 76 ESC 77 NUM 78 F11 79 + 7A PDN 7B - 7C * 7D PUP 7E B D 7F NC
db 030H, 0FFH, 0FFH, 035H, 0FFH, 0FFH, 0FFH, 0FFH, 0FFH, 02BH, 0FFH, 02DH, 02AH, 0FFH, 0FFH, 0FFH
```

```
;83 F7
```

```
;E011 ALT GR E014 CTRR E069 END E06B LEFT E06C HOME E070 INS E071 DEL
```

```
;E072 DOWN E074 RIGHT E075 UP E07A P DN E07D P UP E05A INTRO
```

```
;E11477E1F014F077 PAUSE
```

```
;E012E07C PRINT SCREEN
```

Referencias

1. Dominic Rath. *Open On-Chip Debugger*. PhD thesis, University of Applied Sciences Augsburg, 2005.
2. I. Bowman, S. Siddiqi, and M. Tanuan. Concrete Architecture of the Linux Kernel. <http://docs.huihoo.com/linux/kernel/a2/index.html>, 12 February 1998.
3. I. Bowman. Conceptual Architecture of the Linux Kernel. <http://docs.huihoo.com/linux/kernel/a1/>, 1998.
4. Texas Instruments. IEEE Std 1149.1 (JTAG) Testability. *1997 Semiconductor Group*, 1996.
5. Michael L. Haungs. The Executable and Linking Format (ELF). <http://www.cs.ucdavis.edu/haungs/paper/node1.html>, 21 September 1998.
6. C. Hallinan. *Embedded Linux Premiere A Practical Real-World Approach*. Prentice Hall, 18 September 2006.
7. Karim Yaghmour, Jon Masters, Gilad Ben-Yossef, and Philippe Gerum. *Building Embedded Linux Systems*. O'REILLY, 2008.
8. J. Corbet, A. Rubini, and G. Kroah-Hartman. *Linux Device Drivers, Third Edition*. O'Reilly, 2005.